การพัฒนาเกมห้องหลบหนีด้วยการออกแบบซอฟต์แวร์แบบมอดูลาร์

Development of an Escape Room Game Using Modular Software Design

นายอวกาศ ดีคำ

โครงงานสหกิจศึกษานี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
ปริญญาวิศวกรรมศาสตรบัณฑิต สาขาวิศวกรรมคอมพิวเตอร์และปัญญาประดิษฐ์
คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีไทย - ญี่ปุ่น
ปีการศึกษา 2569

ชื่อโครงงาน	การพัฒนาเกมห้องหลบหนีด้วยการออกแบบซอฟต์แวร์แบบมอดูลาร์
ผู้เขียน	นายอวกาศ ดีคำ
สาขาวิชา	วิศวกรรมคอมพิวเตอร์และปัญญาประดิษฐ์
อาจารย์ที่ปรึกษาสหกิจศึกษา	อาจารย์ ดร. อัดนา เซนโต๊ะ

คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีไทย-ญี่ปุ่น อนุมัติให้โครงงานสหกิจศึกษาฉบับนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

..... ประธานสหกิจศึกษาสาขา

(.....)

วันที่ ..... เดือน ..... พ.ศ. .....

คณะกรรมการสอบโครงงานสหกิจ

..... ประธานกรรมการ

(.....)

..... กรรมการ

(.....)

..... อาจารย์ที่ปรึกษาสหกิจ

(.....)

ลิขสิทธิ์ของสถาบันเทคโนโลยีไทย – ญี่ปุ่น

ชื่อโครงงาน	การพัฒนาเกมห้องหลบหนีด้วยการออกแบบซอฟต์แวร์แบบมอดูลาร์
ผู้เขียน	นายอวกาศ ดีคำ
คณะ	วิศวกรรมศาสตร์
สาขาวิชา	วิศวกรรมคอมพิวเตอร์และปัญญาประดิษฐ์
อาจารย์ที่ปรึกษาสหกิจ	อาจารย์ ดร. อัดนา เซนโต๊ะ
พนักงานที่ปรึกษา	นายธนพงศ์ เคหะเจริญ
ชื่อบริษัท	บริษัท Lumas corp co., LTD.

บทสรุป

โครงงานนี้มีวัตถุประสงค์เพื่อฝึกฝนและศึกษากระบวนการพัฒนาเกมสามมิติ โดยกำหนดขอบเขตของโครงงานเป็นเกมแนวห้องหลบหนี ซึ่งมีการกำหนดรูปแบบการควบคุมและเงื่อนไขการเล่นไว้ตามข้อกำหนดที่ได้รับ ในส่วนของปริศนาภายในเกมเปิดโอกาสให้นักศึกษาเป็นผู้ออกแบบและพัฒนาด้วยตนเอง เพื่อให้ได้เรียนรู้และลงมือปฏิบัติผ่านกระบวนการพัฒนาเกมอย่างครบถ้วน ตั้งแต่การออกแบบแนวคิดและเนื้อหา การออกแบบระบบเกม การพัฒนาโปรแกรม ไปจนถึงการทดสอบการทำงานของเกม

ในการพัฒนาโครงงาน นักศึกษาได้กำหนดแนวทางการออกแบบซอฟต์แวร์โดยยึดหลักการออกแบบแบบมอดูลาร์ โดยออกแบบให้วัตถุแต่ละชิ้นเกิดจากการประกอบคอมโพเนนต์ย่อย ซึ่งแต่ละคอมโพเนนต์รับผิดชอบหน้าที่เฉพาะด้าน แทนการสร้างคลาสโปรแกรมแยกสำหรับวัตถุแต่ละประเภท เพื่อให้สามารถนำส่วนประกอบเดิมกลับมาใช้ซ้ำและเพิ่มวัตถุใหม่ได้โดยไม่จำเป็นต้องแก้ไขโปรแกรมส่วนเดิม ด้วยแนวทางดังกล่าว นักศึกษาจึงเลือกออกแบบปริศนาทั้งห้าองก์โดยมีนาฬิกาเป็นองค์ประกอบหลัก เนื่องจากนาฬิกามีส่วนประกอบพื้นฐานร่วมกัน ได้แก่ หน้าปัดและกลไกการทำงาน แต่สามารถแสดงพฤติกรรมที่แตกต่างกันได้ เช่น การเดินตรง การเดินช้า การหยุดเดิน และการตีบอกเวลา จึงเหมาะสำหรับแสดงให้เห็นถึงประสิทธิภาพและความยืดหยุ่นของการออกแบบซอฟต์แวร์แบบมอดูลาร์ได้อย่างชัดเจน

ผลจากการพัฒนา ได้เกมที่สามารถเล่นจนจบได้ ประกอบด้วยห้าห้องที่เรียงลำดับเป็นเส้นทางเดียว โดยแต่ละห้องมีปริศนาประจำห้อง และมีนาฬิกาภายในเกมทั้งหมด 46 เรือน ซึ่งสร้างขึ้นจากการใช้หน้าปัดและกลไกพื้นฐานร่วมกัน โดยปรับเปลี่ยนเฉพาะคอมโพเนนต์ที่นำมาประกอบเพิ่มเติม เช่น ลานสปริง ลูกตุ้ม และกลไกตีบอกเวลา ทำให้สามารถสร้างนาฬิกาหรือวัตถุปริศนารูปแบบใหม่ได้ด้วยการเลือกและประกอบคอมโพเนนต์ที่มีอยู่ โดยไม่จำเป็นต้องเขียนโปรแกรมเพิ่มเติม

Title of Project	Development of an Escape Room Game Using Modular Software Design
Writer / Submitted by	Mr. Awakas Deecome
Faculty	Engineering
Major	Computer Engineering and Artificial Intelligence
Advisor's	Dr. Adna Sento
Job Supervisor	Mr. Thanapong Khehacharoen
Company's name	Lumas corp co., LTD.

Summary

This project aims to gain practical experience in the 3D game development process by developing an Escape Room game whose control scheme and gameplay conditions follow the assigned requirements. The student independently designed and developed the puzzles, covering concept and content design, game system design, programming, and testing.

The student adopted Modular Software Design from the beginning. Each object is built by combining components that each have a specific responsibility, rather than creating a separate class for each type of object, so existing components can be reused and new objects added without modifying existing code. Clocks were chosen as the central element of all five acts because they share a clock face and a mechanism yet behave differently: keeping time, running slowly, stopping, and chiming. They therefore demonstrate the flexibility of modular design clearly.

The result is a fully playable game of five rooms in a single route, each with its own puzzle, and 46 clocks built from shared clock face and mechanism components, varied by additional components such as mainsprings, pendulums, and chiming mechanisms. New clocks and puzzle objects can be created by selecting and combining existing components, without additional programming.

กิตติกรรมประกาศ

ขอขอบพระคุณบริษัท Lumas corp co., LTD. ที่ได้ให้โอกาสในการฝึกงานและสนับสนุนการจัดทำโครงงานสหกิจนี้เป็นอย่างดี ทั้งในด้านการให้ข้อมูลที่จำเป็น การสนับสนุนด้านเทคนิค รวมถึงความร่วมมือในทุกขั้นตอนของการวางแผน การพัฒนาเกม และทดสอบการทำงานของเกม ซึ่งความช่วยเหลือเหล่านี้มีส่วนสำคัญที่ทำให้โครงงานนี้สามารถดำเนินไปได้อย่างราบรื่นและสำเร็จลุล่วงตามเป้าหมาย

ขอขอบพระคุณอาจารย์ที่ปรึกษาสหกิจ อาจารย์ ดร.อัดนา เซนโต๊ะ ที่ให้คำแนะนำอย่างใกล้ชิดและเป็นประโยชน์ตลอดระยะเวลาการทำงาน รวมถึงนายธนพงศ์ เคหะเจริญ พนักงานที่ปรึกษา ที่ให้การสนับสนุนในด้านวิชาการและเทคนิคด้วยความเอาใจใส่และอดทน นอกจากนี้ยังขอขอบคุณเพื่อนร่วมโครงการทุกคนที่ร่วมแรงร่วมใจและทุ่มเททำงานอย่างเต็มที่จนโครงงานนี้สำเร็จสมบูรณ์

อวกาศ ดีคำ

สารบัญ

	หน้า
บทสรุปภาษาไทย	ค
บทสรุปภาษาอังกฤษ	ง
กิตติกรรมประกาศ	ง
สารบัญ	ฉ
สารบัญตาราง	ช
สารบัญภาพ	ญ

บทที่

บทที่ 1 บทนำ	1
1.1 ชื่อและที่ตั้งของสถานประกอบการ	1
1.2 ลักษณะธุรกิจของสถานประกอบการ หรือการให้บริการหลักขององค์กร	2
1.3 รูปแบบการจัดองค์กรและการบริหารองค์กร	2
1.4 ตำแหน่งและหน้าที่งานที่นักศึกษาได้รับมอบหมาย	2
1.5 พนักงานที่ปรึกษา และ ตำแหน่งของพนักงานที่ปรึกษา	3
1.6 ระยะเวลาที่ปฏิบัติงาน	3
1.7 ที่มาและความสำคัญของปัญหา	3
1.8 วัตถุประสงค์หรือจุดมุ่งหมายของโครงงาน	4
1.9 ผลที่คาดว่าจะได้รับจากการปฏิบัติงานหรือโครงงานที่ได้รับมอบหมาย	4
1.10 นิยามศัพท์เฉพาะ	4
บทที่ 2 ทฤษฎีหรือเทคโนโลยีที่ใช้ในการปฏิบัติงาน	7
2.1 ทฤษฎีการออกแบบซอฟต์แวร์แบบมอดูลาร์	7
2.2 หลักการออกแบบเชิงวัตถุที่ใช้ในโครงงาน	11
2.3 ทฤษฎีความถูกต้องของการนับเวลาในเกม	18
2.4 ทฤษฎีการทดสอบซอฟต์แวร์	22
2.5 เทคโนโลยีที่ใช้ในการปฏิบัติงาน	25
บทที่ 3 แผนงานการปฏิบัติงานและขั้นตอนการดำเนินงาน	35
3.1 แผนการทำโครงงาน	35
3.2 รายละเอียดขั้นตอนการทำโครงงาน	38

3.3 ขั้นตอนการดำเนินงานที่นักศึกษาปฏิบัติงานหรือโครงงาน 40
บทที่ 4 ผลการดำเนินงาน 49
4.1 ผลการดำเนินงาน 49
4.2 ผลการวิเคราะห์ข้อมูล 60
4.3 วิจารณ์ข้อมูลโดยเปรียบเทียบผลที่ได้รับกับวัตถุประสงค์และจุดมุ่งหมายของการปฏิบัติงานหรือการจัดทำโครงงาน 82
บทที่ 5 บทสรุปและข้อเสนอแนะ 89
5.1 สรุปผลการดำเนินโครงงาน 89
5.2 ปัญหาและแนวทางการแก้ไข 90
5.3 ข้อเสนอแนะจากการดำเนินงาน 94
เอกสารอ้างอิง 98
ภาคผนวก 103
ประวัติผู้จัดทำโครงงานสหกิจศึกษา 106

สารบัญตาราง

ตารางที่		หน้า
ตารางที่ 2.1	Interface ที่ประกาศในโครงงานและจำนวนผู้ implement	12
ตารางที่ 2.2	ผลการประเมินตามหลักการเปิด–ปิด	14
ตารางที่ 2.3	การใช้ Observer Pattern ในโครงงาน	17
ตารางที่ 2.4	ระยะห่างระหว่างเลขที่แทนได้ ณ ค่าสะสม 280 นาที (เวลา 4:40)	20
ตารางที่ 2.5	ไฟล์ทดสอบและคุณลักษณะเชิงสถาปัตยกรรมที่แต่ละไฟล์พิสูจน์	23
ตารางที่ 2.6	ข้อตกลงการอ่านข้อมูลนำเข้าตามชนิดของค่า	31
ตารางที่ 2.7	เครื่องมือสร้างเนื้อหาด้วยกระบวนวิธีในโครงงาน	32
ตารางที่ 3.1	แผนการฝึกสหกิจศึกษา ระหว่างวันที่ 1 กรกฎาคม ถึง 31 ตุลาคม พ.ศ. 2569	36
ตารางที่ 3.2	ข้อกำหนดจากโจทย์และส่วนของโปรแกรมที่รองรับ	36
ตารางที่ 3.3	ขอบเขตเนื้อหาของเกมที่นักศึกษากำหนดขึ้นเอง	37
ตารางที่ 3.4	เครื่องมือที่ใช้ในการพัฒนาโครงงาน	38
ตารางที่ 3.5	ทรัพยากรที่นำเข้าจากภายนอกโครงงาน	39
ตารางที่ 3.6	แนวคิดและเหตุผลการออกแบบปริศนาแต่ละองก์	41
ตารางที่ 3.7	ชิ้นส่วนหลักของระบบนาฬิกาและหน้าที่	43
ตารางที่ 3.8	ไฟล์ที่เปลี่ยนจากรับ PlayerInteractor เป็นรับ IActor	45
ตารางที่ 3.9	คลาสเฉพาะของปริศนาแต่ละองก์	47
ตารางที่ 4.1	ปริมาณผลลัพธ์ของโครงงานที่วัดได้จากไฟล์ในโปรเจกต์	49
ตารางที่ 4.2	คำตอบของปริศนาแต่ละองก์และหลักฐานที่ยืนยัน	53
ตารางที่ 4.3	กลไกที่แต่ละองก์ใช้ร่วมกันและส่วนที่เขียนขึ้นเฉพาะ	54
ตารางที่ 4.4	ชิ้นส่วนที่ประกอบเป็นนาฬิกาทั้ง 46 เรือน	58
ตารางที่ 4.5	จำนวนบรรทัดของโปรแกรมแยกตามบทบาท	59
ตารางที่ 4.6	โมดูลทั้งเก้ากลุ่มและขนาดของแต่ละกลุ่ม	60
ตารางที่ 4.7	การพึ่งพาระหว่างโมดูลที่วัดได้จากโปรแกรม	62
ตารางที่ 4.8	การเทียบขอบเขตที่ประกาศไว้กับที่โปรแกรมใช้จริง	64
ตารางที่ 4.9	ส่วนต่อประสานทั้งสิบสองรายการและจำนวนคลาสที่นำไปใช้	65
ตารางที่ 4.10	หลักฐานการนำกลับมาใช้ที่วัดได้	67
ตารางที่ 4.11	ผลการนับรูปแบบการเขียนโปรแกรมในโปรแกรมส่วนที่ทำงานในเกม 63 ไฟล์	69
ตารางที่ 4.12	ผลการประเมินความสามารถในการขยายเก้าสถานการณ์	70

ตารางที่ 4.13 ชุดทดสอบทั้ง 15 กรณีและสิ่งที่แต่ละกรณียืนยัน 73
ตารางที่ 4.14 ความครอบคลุมของชุดทดสอบแยกตามโมดูล 76
ตารางที่ 4.15 ผลการสังเกตจากการทดลองเล่นของเพื่อน 5 คน 78
ตารางที่ 4.16 สถานะของโปรเจกต์และข้อความที่พบใน Console 80
ตารางที่ 4.17 ผลการดำเนินงานเทียบกับวัตถุประสงค์ของโครงงาน 83
ตารางที่ 4.18 ผลเทียบกับขอบเขตที่กำหนดไว้ในหัวข้อ 3.1 84
ตารางที่ 4.19 ผลการประเมินคุณภาพของโครงสร้างแปดเกณฑ์ 85
ตารางที่ 4.20 ข้อจำกัดที่พบ เรียงตามผลกระทบ 87
ตารางที่ 5.1 กลไกสี่อย่างที่ใช้แก้ปัญหาความถูกต้องของการนับเวลา 91
ตารางที่ 5.2 ผลที่วัดได้หลังเปลี่ยนมาใช้ส่วนต่อประสาน IActor 93
ตารางที่ 5.3 ลำดับความสำคัญของข้อเสนอแนะ 96

สารบัญภาพ

ภาพที่		หน้า
ภาพที่ 1.1	แผนที่ที่ตั้งบริษัท Lumas corp co., LTD.	1
ภาพที่ 2.1	ส่วนติดต่อผู้ใช้ของ Unity Editor ในโครงงาน	25
ภาพที่ 2.2	ข้อมูลผังบ้านที่เก็บในรูปแอสเซ็ต ScriptableObject	28
ภาพที่ 2.3	การประกาศขอบเขตการอ้างอิงของโมดูลนาฬิกาด้วยไฟล์ Assembly Definition	30
ภาพที่ 3.1	โจทย์ของโครงงานที่ได้รับจากพนักงานที่ปรึกษา	35
ภาพที่ 3.2	ขั้นตอนการดำเนินงานสี่ขั้นของโครงงาน	40
ภาพที่ 3.3	เมนูเครื่องมือสร้างเนื้อหาที่พัฒนาขึ้นในโครงงาน	46
ภาพที่ 3.4	ผังบ้านทั้งห้าห้องที่สร้างจากไฟล์ HousePlan.asset	46
ภาพที่ 4.1	ห้องหน้าร้าน องก์ที่หนึ่ง	50
ภาพที่ 4.2	ห้องช่าง องก์ที่สอง	51
ภาพที่ 4.3	ห้องนั่งเล่น องก์ที่สาม	51
ภาพที่ 4.4	ห้องนอน องก์ที่สี่	52
ภาพที่ 4.5	ห้องหลังตู้นาฬิกา องก์ที่ห้า	52
ภาพที่ 4.6	ผนังนาฬิกาเจ็ดเรือนขององก์ที่หนึ่ง ให้เห็นลูกตุ้มที่แกว่งและที่หยุด	55
ภาพที่ 4.7	การยกนาฬิกาขึ้นดูและแป้นเลขที่ปรากฏเมื่อไขลาน	56
ภาพที่ 4.8	นาฬิกาพกและนาฬิกาทรายที่ชูขึ้นข้างกันขณะจับเวลาในองก์ที่สอง	56
ภาพที่ 4.9	ภาพถ่ายเก่าและป้ายเลขหลังของในองก์ที่สี่	57
ภาพที่ 4.10	จุดแสงหกจุดบนจอก่อนและหลังหมุนปริซึมถูกต้อง ในองก์ที่ห้า	57
ภาพที่ 4.11	ทิศทางการพึ่งพาระหว่างโมดูลของโครงงาน	63
ภาพที่ 4.12	ไฟล์ PlankClock.asset ใน Inspector ให้เห็นรายการ Fittings ที่ไม่แยกส่วนแสดงเวลาจากชิ้นส่วนกลไก	68
ภาพที่ 4.13	ผลการรันชุดทดสอบใน Test Runner แสดงทั้ง 15 กรณีและเวลาที่ใช้	76
ภาพที่ 4.14	หน้าต่าง Console ของ Unity แสดงสถานะการคอมไพล์และข้อความที่พบ	82

บทที่ 1
บทนำ

1.1 ชื่อและที่ตั้งของสถานประกอบการ

สถานประกอบการที่ผู้เขียนเข้าปฏิบัติงานสหกิจศึกษาคือ บริษัท Lumas corp co., LTD. ซึ่งเป็นสตูดิโอพัฒนาเกม ตั้งอยู่เลขที่ 818/2 ถนนอุดมสุข แขวงบางนาเหนือ เขตบางนา กรุงเทพมหานคร รหัสไปรษณีย์ 10260

ปัจจุบันดำเนินกิจการด้านการพัฒนาและเผยแพร่เกมคอมพิวเตอร์ โดยมีผลงานเกมที่เปิดเผยต่อสาธารณะคือ Disordered เกมแนวสืบสวนสอบสวนแบบวิชวลโนเวล (Visual Novel) ที่วาดด้วยมือ ฉากอยู่ในประเทศไทยช่วงทศวรรษ 1990 ซึ่งบริษัทเป็นทั้งผู้พัฒนาและผู้เผยแพร่บนแพลตฟอร์ม Steam ที่ตั้งของบริษัทเป็นดังภาพที่ 1.1



ภาพที่ 1.1 แผนที่ที่ตั้งบริษัท Lumas corp co., LTD.

ที่มา : ปรับจากแผนที่ Google Maps

1.2 ลักษณะธุรกิจของสถานประกอบการ หรือการให้บริการหลักขององค์กร

บริษัท Lumas corp co., LTD. ดำเนินธุรกิจด้านการพัฒนาและเผยแพร่เกมคอมพิวเตอร์ในรูปแบบสตูดิโอเกม โดยผลิตภัณฑ์ของบริษัทเผยแพร่ให้ผู้เล่น ผ่านแพลตฟอร์ม Steam

การให้บริการหลักของบริษัทประกอบด้วย

1. การพัฒนาเกมคอมพิวเตอร์ ซึ่งมีผลงานที่เปิดเผยต่อสาธารณะ คือเกม Disordered
2. การออกแบบเกมและการผลิตงานกราฟิกและโมเดลสามมิติสำหรับเกม
3. การเผยแพร่เกมของบริษัทผ่านแพลตฟอร์ม Steam

ส่วนงานที่เกี่ยวข้องกับโครงงานในรายงานฉบับนี้คือแผนกพัฒนาเกม ซึ่งเป็นงานพัฒนาซอฟต์แวร์เกม ตรงกับตำแหน่งที่ผู้เขียนได้รับมอบหมายตามหัวข้อ 1.4

1.3 รูปแบบการจัดองค์กรและการบริหารองค์กร

บริษัท Lumas corp co., LTD. แบ่งโครงสร้างการบริหารออกเป็นแผนกตามหน้าที่ความรับผิดชอบ ดังนี้

1. แผนกบริหารจัดการทั่วไป (Administration Department)

   ดูแลด้านการบริหารงานภายในองค์กร
2. แผนกพัฒนาเกม
3. แผนกออกแบบเกม
4. แผนกกราฟฟิคและปั้นโมเดลเกม

การบริหารองค์กรเน้นการทำงานร่วมกันเป็นทีม (Teamwork) โดยมีผู้บริหารระดับสูงเป็นผู้กำหนดนโยบายและเป้าหมายหลักของบริษัท และมีการประชุมประจำเพื่อวางแผน ติดตามผล และแก้ไขปัญหาอย่างมีประสิทธิภาพ นอกจากนี้บริษัทส่งเสริมการพัฒนาศักยภาพ ของพนักงานอย่างต่อเนื่อง เพื่อให้สามารถตอบสนองต่อการเปลี่ยนแปลงของตลาดและเทคโนโลยีได้อย่างรวดเร็ว

1.4 ตำแหน่งและหน้าที่งานที่นักศึกษาได้รับมอบหมาย

นักศึกษาได้รับมอบหมายให้ปฏิบัติงานในตำแหน่ง **ผู้ช่วยพัฒนาเกม (Game Developer intern)** ภายใต้แผนกพัฒนาเกม ของบริษัท Lumas corp co., LTD.

หน้าที่และความรับผิดชอบหลักของนักศึกษา ได้แก่

1. สนับสนุนการออกแบบและพัฒนาเกม ของบริษัท
2. เขียนและปรับปรุงโค้ดโปรแกรมด้วยภาษาเว็บ เช่น C# และ HTML
3. ทำงานร่วมกับทีมเพื่อทดสอบและแก้ไขข้อผิดพลาดของระบบ (Debugging)

4. ออกแบบหน้า UI ของเกม โดยใช้เครื่องมือออกแบบ เช่น Figma ตามแนวทางที่ทีมกำหนด
5. จัดเตรียมเอกสารประกอบการพัฒนาและคู่มือการใช้งานเบื้องต้น
6. เข้าร่วมประชุมและรายงานความก้าวหน้าของงานกับผู้จัดการโครงการและทีมงาน
7. ศึกษาและเรียนรู้เทคโนโลยีใหม่ ๆ ที่เกี่ยวข้องกับการพัฒนาเกม

นักศึกษามุ่งมั่นในการเรียนรู้และพัฒนาทักษะอย่างต่อเนื่อง เพื่อให้สามารถสนับสนุนการทำงานของทีมและส่งมอบงานที่มีคุณภาพตรงตามความต้องการของบริษัท

1.5 พนักงานที่ปรึกษา และ ตำแหน่งของพนักงานที่ปรึกษา

นายธนพงศ์ เคหะเจริญ ดำรงตำแหน่ง Project Manager/ Game Developer ทำหน้าที่เป็นพนักงานที่ในโครงการสหกิจศึกษานี้ โดยมีบทบาทในการให้คำแนะนำทางเทคนิคและการบริหารจัดการโครงการ รวมถึงสนับสนุนการพัฒนาระบบเกมและประสานงานระหว่างทีมงานต่าง ๆ เพื่อให้โครงการดำเนินไปอย่างมีประสิทธิภาพและสำเร็จลุล่วงตามเป้าหมาย

1.6 ระยะเวลาที่ปฏิบัติงาน

นักศึกษาได้เข้าปฏิบัติงานในโครงการสหกิจศึกษาที่บริษัท Lumas corp co., LTD. ตั้งแต่วันที่ 1 กรกฎาคม พ.ศ. 2569 ถึงวันที่ 31 ตุลาคม พ.ศ. 2569 รวมระยะเวลาทั้งสิ้น 4 เดือน โดยได้ร่วมปฏิบัติงานในแผนกพัฒนาเกม เพื่อเรียนรู้และปฏิบัติจริงในงานพัฒนาระบบเกม

1.7 ที่มาและความสำคัญของปัญหา

ในการพัฒนาเกมที่มีวัตถุหลากหลายประเภทและมีพฤติกรรมการทำงานแตกต่างกัน การออกแบบโครงสร้างซอฟต์แวร์เป็นสิ่งสำคัญ หากสร้างคลาสโปรแกรมแยกสำหรับวัตถุแต่ละประเภท อาจทำให้เกิดความซ้ำซ้อนของโค้ด และเมื่อมีการเพิ่มวัตถุหรือปรับเปลี่ยนพฤติกรรมการทำงาน จะต้องแก้ไขโปรแกรมหลายส่วน ส่งผลให้การพัฒนาและบำรุงรักษาระบบทำได้ยากขึ้น

เพื่อแก้ไขปัญหาดังกล่าว จึงนำหลักการออกแบบซอฟต์แวร์แบบมอดูลาร์ (Modular Software Design) มาใช้ โดยแบ่งการทำงานออกเป็นคอมโพเนนต์ย่อย (Component) ที่มีหน้าที่เฉพาะด้าน และสามารถนำมาประกอบกันเพื่อสร้างวัตถุที่มีคุณสมบัติและพฤติกรรมแตกต่างกัน ช่วยลดความซ้ำซ้อนและเพิ่มความยืดหยุ่นในการพัฒนาระบบ

โครงงานนี้จึงเลือกใช้นาฬิกาเป็นองค์ประกอบหลักในการออกแบบปริศนาทั้งห้าองก์ เนื่องจากนาฬิกามีส่วนประกอบพื้นฐานร่วมกัน ได้แก่ หน้าปัดและกลไกการทำงาน แต่สามารถแสดงพฤติกรรมที่แตกต่างกันได้ เช่น การเดินตรง การเดินช้า การหยุดเดิน และการตีบอกเวลา จึงเหมาะ

สำหรับแสดงให้เห็นถึงการนำคอมโพเนนต์กลับมาใช้ซ้ำ และการสร้างวัตถุรูปแบบใหม่ด้วยการประกอบส่วนประกอบเดิมโดยไม่จำเป็นต้องเขียนโปรแกรมเพิ่มเติม

1.8 วัตถุประสงค์หรือจุดมุ่งหมายของโครงงาน

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาเกมห้องหลบหนีด้วยการออกแบบซอฟต์แวร์แบบมอดูลาร์ เพื่อให้โครงสร้างโปรแกรมมีความยืดหยุ่นและรองรับการพัฒนาต่อยอดได้ โดยมีวัตถุประสงค์ดังนี้

1. เพื่อศึกษาและประยุกต์ใช้หลักการออกแบบซอฟต์แวร์แบบมอดูลาร์ในการพัฒนาเกม
2. เพื่อออกแบบและพัฒนาคอมโพเนนต์ที่สามารถนำกลับมาใช้ซ้ำและประกอบเป็นวัตถุที่มีพฤติกรรมแตกต่างกันได้
3. เพื่อพัฒนาเกมห้องหลบหนีสามมิติที่ประกอบด้วยปริศนาทั้งห้าองก์ตามแนวคิดที่ออกแบบไว้
4. เพื่อทดสอบการทำงานของระบบและตรวจสอบความถูกต้องของโครงสร้างซอฟต์แวร์ที่พัฒนาขึ้น

1.9 ผลที่คาดว่าจะได้รับจากการปฏิบัติงานหรือโครงงานที่ได้รับมอบหมาย

1. ได้รับความรู้และประสบการณ์เกี่ยวกับกระบวนการพัฒนาเกมสามมิติ ตั้งแต่การออกแบบ การพัฒนาโปรแกรม ไปจนถึงการทดสอบเกม
2. ได้เรียนรู้และเข้าใจการประยุกต์ใช้หลักการออกแบบซอฟต์แวร์แบบมอดูลาร์ (Modular Software Design) ในการพัฒนาเกม
3. ได้โครงสร้างซอฟต์แวร์ที่สามารถนำคอมโพเนนต์กลับมาใช้ซ้ำ และรองรับการสร้างวัตถุที่มีคุณสมบัติและพฤติกรรมแตกต่างกันโดยไม่ต้องเขียนโปรแกรมเพิ่มเติม
4. ได้เกมห้องหลบหนีสามมิติที่สามารถเล่นได้จนจบ พร้อมปริศนาทั้งห้าองก์ที่พัฒนาขึ้นตามข้อกำหนดของโครงงาน

1.10 นิยามศัพท์เฉพาะ

1. **เกมห้องหลบหนี (Escape Room Game)** หมายถึง เกมที่ผู้เล่นอยู่ในพื้นที่ปิด ต้องสำรวจสิ่งของในฉาก เก็บเบาะแส และแก้ปริศนาตามลำดับ เพื่อเปิดทางออกไปยังพื้นที่ถัดไป
2. **องก์ (Act)** หมายถึง หน่วยของปริศนาหนึ่งชุดที่อยู่ในห้องหนึ่งห้อง ในรายงานฉบับนี้ใช้เรียกปริศนาแต่ละชุด ที่ผู้เล่นต้องแก้เรียงตามลำดับ

3. **เอนจินเกม (Game Engine)** หมายถึง ซอฟต์แวร์ที่รวมเครื่องมือพื้นฐานสำหรับสร้างเกมไว้ เช่น การแสดงภาพสามมิติ ระบบฟิสิกส์ และการรับค่าจากอุปกรณ์ควบคุม โครงงานนี้ใช้เอนจิน Unity
4. **ภาษา C# (C-Sharp)** หมายถึง ภาษาโปรแกรมเชิงวัตถุ ใช้เขียนโปรแกรมควบคุมการทำงานของวัตถุในเกม บนเอนจิน Unity
5. **ภาษา HTML (HyperText Markup Language)** หมายถึง ภาษามาร์กอัปสำหรับกำหนดโครงสร้างของเนื้อหา บนหน้าเว็บ
6. **ส่วนติดต่อผู้ใช้ (User Interface: UI)** หมายถึง ส่วนของซอฟต์แวร์ที่ผู้ใช้มองเห็นและใช้โต้ตอบ เช่น ปุ่ม เมนู และข้อความบนหน้าจอ
7. **ฟิกมา (Figma)** หมายถึง เครื่องมือออกแบบส่วนติดต่อผู้ใช้ ที่ทำงานผ่านเว็บเบราว์เซอร์ ใช้วาดโครงร่างและต้นแบบของหน้าจอ ก่อนนำไปพัฒนาจริง
8. **การดีบัก (Debugging)** หมายถึง กระบวนการค้นหาสาเหตุ และแก้ไขข้อผิดพลาดในโปรแกรม
9. **การออกแบบซอฟต์แวร์แบบมอดูลาร์ (Modular Software Design)** หมายถึง แนวทางออกแบบซอฟต์แวร์ที่แบ่งระบบออกเป็นส่วนย่อย ที่มีหน้าที่ชัดเจน แต่ละส่วนแก้ไขหรือแทนที่ได้ โดยไม่กระทบส่วนอื่นมากนัก ในรายงานฉบับนี้ คำว่า "เชิงโมดูล" มีความหมายเดียวกัน
10. **คอมโพเนนต์ (Component)** หมายถึง หน่วยของโปรแกรมที่นำไปติดกับวัตถุในฉากได้ ทำหน้าที่อย่างเดียว และทำงานร่วมกับคอมโพเนนต์อื่น ที่ติดอยู่บนวัตถุเดียวกันได้ ในเอนจิน Unity คือคลาสที่สืบทอดจาก MonoBehaviour
11. **คลาส (Class)** หมายถึง แม่แบบในภาษาโปรแกรมเชิงวัตถุ ที่กำหนดข้อมูลและพฤติกรรมของวัตถุชนิดหนึ่ง
12. **การประกอบแทนการสืบทอด (Composition over Inheritance)** หมายถึง แนวทางที่สร้างพฤติกรรมของวัตถุ จากการนำคอมโพเนนต์หลายตัวมาประกอบกัน แทนการสร้างคลาสลูกที่สืบทอดจากคลาสแม่
13. **ส่วนต่อประสาน (Interface)** หมายถึง ข้อตกลงที่ระบุว่าคลาสที่นำไปใช้ ต้องมีความสามารถอะไรบ้าง โดยไม่ระบุว่าทำอย่างไร ทำให้โปรแกรมเรียกใช้ความสามารถนั้นได้ โดยไม่ต้องรู้จักคลาสที่ทำงานจริง
14. **การนำกลับมาใช้ซ้ำ (Reusability)** หมายถึง คุณสมบัติที่ทำให้ส่วนของโปรแกรมที่เขียนไว้แล้ว นำไปใช้กับงานอื่นได้ โดยไม่ต้องเขียนใหม่

15. **การเกาะเกี่ยว (Coupling)** หมายถึง ระดับความขึ้นต่อกันระหว่างโมดูล ยิ่งต่ำ ยิ่งแก้ไขหรือแทนที่โมดูลหนึ่งได้ โดยไม่กระทบโมดูลอื่น
16. **การเกาะกลุ่ม (Cohesion)** หมายถึง ระดับความเกี่ยวเนื่องกันของหน้าที่ ภายในโมดูลเดียวกัน ยิ่งสูง ยิ่งเข้าใจและทดสอบได้ง่าย
17. **แอสเซมบลีดีฟินิชัน (Assembly Definition)** หมายถึง ไฟล์ตั้งค่าของ Unity ที่กำหนดให้โฟลเดอร์หนึ่งถูกคอมไพล์เป็นแอสเซมบลีแยก และระบุได้ว่าแอสเซมบลีนั้นอ้างถึงแอสเซมบลีใดได้บ้าง ทำให้ขอบเขตระหว่างโมดูล เป็นข้อบังคับระดับการคอมไพล์
18. **พรีแฟบ (Prefab)** หมายถึง แม่แบบของวัตถุในฉาก ที่บันทึกไว้เป็นไฟล์ เมื่อนำไปวางในฉากจะได้สำเนาที่ยังเชื่อมกับแม่แบบ การแก้แม่แบบจึงมีผลกับทุกสำเนา
19. **ฉาก (Scene)** หมายถึง ไฟล์ที่เก็บวัตถุทั้งหมดในพื้นที่เล่นหนึ่งพื้นที่ พร้อมตำแหน่งและค่าที่ตั้งไว้
20. **หน้าปัดและกลไกนาฬิกา (Dial and Clock Movement)** หมายถึง ในรายงานฉบับนี้ หน้าปัดคือส่วนที่แสดงเวลาด้วยเข็มบนหน้านาฬิกา ส่วนกลไกคือส่วนที่นับเวลาและกำหนดว่านาฬิกาเดินหรือไม่ นาฬิกาแต่ละเรือน ประกอบจากสองส่วนนี้ ร่วมกับชิ้นส่วนอื่น เช่น ลานสปริงและลูกตุ้ม

บทที่ 2
ทฤษฎีหรือเทคโนโลยีที่ใช้ในการปฏิบัติงาน

ในการปฏิบัติงานสหกิจศึกษาครั้งนี้ เป็นการนำความรู้ทางด้านทฤษฎีวิศวกรรมซอฟต์แวร์และเทคโนโลยีการพัฒนาเกมมาใช้ในการปฏิบัติงานตลอดการดำเนินโครงงาน ซึ่งเป็นทั้งการนำความรู้ที่เคยเรียนมาประยุกต์ใช้ และการศึกษาเรียนรู้สิ่งใหม่ ๆ ที่ได้จากการปฏิบัติงานจริง เนื่องจากโครงงานนี้มีลักษณะเป็นระบบซอฟต์แวร์ที่ประกอบด้วยส่วนย่อยจำนวนมากทำงานร่วมกัน ทฤษฎีที่นำมาใช้จึงแบ่งได้เป็นสี่กลุ่ม คือ ทฤษฎีการออกแบบซอฟต์แวร์เชิงโมดูล หลักการออกแบบเชิงวัตถุ ทฤษฎีความถูกต้องของการนับเวลาในเกม และทฤษฎีการทดสอบซอฟต์แวร์ ร่วมกับเทคโนโลยีและเครื่องมือที่ใช้ในการพัฒนา โดยในแต่ละหัวข้อจะระบุตำแหน่งในซอร์สโค้ดของโครงงานที่ทฤษฎีนั้นถูกนำไปใช้จริงกำกับไว้ เพื่อให้สามารถตรวจสอบย้อนกลับได้

2.1 ทฤษฎีการออกแบบซอฟต์แวร์แบบมอดูลาร์

ในการพัฒนาเกมที่มีวัตถุหลากหลายประเภท หากเขียนคลาสโปรแกรมแยกสำหรับวัตถุแต่ละประเภท โค้ดที่ทำหน้าที่คล้ายกันจะถูกเขียนซ้ำในหลายคลาส และเมื่อต้องเพิ่มวัตถุใหม่หรือเปลี่ยนพฤติกรรม ก็ต้องแก้ไขโปรแกรมหลายส่วน ซึ่ง Fowler จัดให้ความซ้ำซ้อนของโค้ด (Duplicated Code) และการต้องแก้หลายจุดพร้อมกันเป็นอาการที่ควรแก้ก่อนอาการอื่น [1] ปัญหานี้เป็นปัญหาเชิงโครงสร้างของซอฟต์แวร์ ซึ่งทฤษฎีวิศวกรรมซอฟต์แวร์ตอบด้วยการออกแบบซอฟต์แวร์แบบมอดูลาร์ (Modular Software Design) หรือที่เรียกว่าการออกแบบซอฟต์แวร์เชิงโมดูล คือการแบ่งระบบออกเป็นหน่วยย่อยที่มีหน้าที่เฉพาะแล้วนำมาประกอบกันเป็นระบบ หัวข้อนี้จึงเริ่มจากหลักการแบ่งระบบเป็นโมดูล ตัวชี้วัดคุณภาพของการแบ่ง สถาปัตยกรรมเชิงองค์ประกอบ การประกอบแทนการสืบทอด และการนำกลับมาใช้ซ้ำ ตามลำดับ

2.1.1 การแยกระบบออกเป็นโมดูลและการซ่อนข้อมูล

การแยกระบบออกเป็นโมดูล (Modular Decomposition) คือการแบ่งซอฟต์แวร์ขนาดใหญ่ออกเป็นหน่วยย่อยที่พัฒนา ทดสอบ และแก้ไขได้โดยอิสระจากกัน Parnas เสนอไว้ว่าเกณฑ์ที่ควรใช้ในการแบ่งโมดูลไม่ใช่การแบ่งตามลำดับขั้นตอนการทำงานของโปรแกรม แต่ควรแบ่งตาม **สิ่งที่แต่ละโมดูลต้องปิดซ่อนไว้ไม่ให้โมดูลอื่นรู้** (Information Hiding) เพราะการเปลี่ยนแปลงในอนาคตมักเกิดขึ้นกับรายละเอียดภายใน ถ้ารายละเอียดนั้นถูกซ่อนไว้ การเปลี่ยนแปลงก็จะไม่ลุกลามออกไปนอกโมดูล [2]

ในโครงงานนี้ ซอร์สโค้ดถูกแบ่งออกเป็นโฟลเดอร์ตาม **โดเมนของระบบ** ไม่ใช่ตามชนิดของอ็อบเจกต์ในเอนจิน รวม 9 กลุ่ม ผลการแบ่งโมดูลพร้อมจำนวนไฟล์และจำนวนบรรทัดของแต่ละกลุ่มแสดงไว้ในตารางที่ 4.6 ของบทที่ 4

สิ่งที่ทำให้การแบ่งนี้เป็นการแบ่งโมดูลจริงตามความหมายของ Parnas ไม่ใช่เพียงการจัดระเบียบโฟลเดอร์ คือ **5 โมดูลแรกมีไฟล์ Assembly Definition (.asmdef) กำกับ** (และชุดทดสอบมีอีกหนึ่งไฟล์ รวมหกไฟล์) ซึ่งทำให้ขอบเขตระหว่างโมดูลกลายเป็นข้อบังคับระดับคอมไพเลอร์ ไม่ใช่เพียงข้อตกลงในการตั้งชื่อ รายละเอียดของกลไกนี้อยู่ในหัวข้อ 2.5.7

ข้อสังเกตที่ควรบันทึก คือโครงงานนี้ **ไม่ใช้ namespace เลยแม้แต่ที่เดียว** ทุกคลาสอยู่ใน global namespace และทุกไฟล์ .asmdef ตั้งค่า "rootNamespace": "" การแบ่งโมดูลที่นี่จึงใช้ .asmdef เป็นเครื่องมือแทน namespace ซึ่งเข้มงวดกว่า เพราะ namespace เพียงจัดกลุ่มชื่อและข้ามได้ด้วยคำสั่ง using ขณะที่ .asmdef ปิดกั้นการอ้างถึงในระดับการคอมไพล์ ต้นทุนที่ตามมาคือหากนำแอสเซ็ตจากภายนอกที่มีคลาสชื่อซ้ำเข้ามา โปรเจกต์จะคอมไพล์ไม่ผ่านทันที

2.1.2 การเกาะเกี่ยว (Coupling) และการเกาะกลุ่ม (Cohesion)

Coupling และ Cohesion เป็นตัวชี้วัดคุณภาพของการแบ่งโมดูลที่ Stevens, Myers และ Constantine เสนอไว้ตั้งแต่ปี ค.ศ. 1974 [3] โดย

- **Coupling** คือระดับความขึ้นต่อกันระหว่างโมดูล ยิ่งต่ำยิ่งดี เพราะโมดูลที่ขึ้นต่อกันน้อยสามารถแก้ไขหรือแทนที่ได้โดยไม่กระทบโมดูลอื่น
- **Cohesion** คือระดับความเกี่ยวเนื่องกันของหน้าที่ภายในโมดูลเดียว ยิ่งสูงยิ่งดี เพราะโมดูลที่ทำหน้าที่เดียวย่อมเข้าใจและทดสอบได้ง่ายกว่าโมดูลที่ทำหลายหน้าที่ปนกัน

เป้าหมายของการออกแบบจึงมักสรุปสั้น ๆ ว่า **Low Coupling, High Cohesion** และในมาตรฐาน ISO/IEC 25010 ความสามารถเช่นนี้จัดอยู่ในคุณลักษณะคุณภาพด้าน Maintainability โดยมี Modularity เป็นคุณลักษณะย่อย [4] อาการของ Coupling ที่สูงเกินไปที่ Fowler เรียกว่า code smell ได้แก่การที่การแก้ไขหนึ่งจุดบังคับให้ต้องแก้ไขอีกหลายจุดตามไปด้วย (Shotgun Surgery) และการที่คลาสหนึ่งต้องรู้รายละเอียดภายในของอีกคลาสหนึ่งมากเกินจำเป็น [1]

ตำแหน่งที่พบในโครงงาน — วัดโดยลบคอมเมนต์ออกจากทุกบรรทัดก่อน แล้วค้นหาชื่อชนิดข้อมูลที่ประกาศในโมดูลอื่น ผลที่ได้มีทั้งด้านที่ดีและด้านที่ยังเป็นข้อจำกัด

ผลการวัดทิศทางการพึ่งพาระหว่างโมดูล พร้อมตัวอย่างหลักฐานในโปรแกรม แสดงไว้ในตารางที่ 4.7 ของบทที่ 4

ด้านที่เป็นไปตามทฤษฎี คือ **โมดูล Clock และ Puzzle ไม่อ้างถึงชนิดข้อมูลใดในโมดูล Player หรือ UI แม้แต่ครั้งเดียว** (ยืนยันโดยการค้นหาคำว่า PlayerInteractor, PlayerHud,

PlayerMode, InspectView, InputReader, InventoryUI ในโฟลเดอร์ Clock/ ได้ผลลัพธ์ศูนย์รายการ) และโมดูล Items กับ World ไม่พึ่งโมดูลใดเลย ทำให้ย้ายไปใช้ในโปรเจกต์อื่นได้ทันที

ด้านที่ยังเป็นข้อจำกัด มีสามเรื่องที่ต้องระบุตามจริง

1. **มีวงจรการพึ่งพาระหว่าง Player และ UI** คือ Player อ้าง UI (PlayerInteractor.cs:31-33) และ UI อ้าง Player กลับ (InventoryUI.cs:18-19) วงจรนี้เป็นสาเหตุทางเทคนิคที่ทำให้สองโมดูลนี้ใส่ .asmdef ไม่ได้ (ดูหัวข้อ 2.5.7)
2. **มีการพึ่งพาข้ามชั้นหนึ่งจุด** คือ Player ⟶ Clock ที่ HeldSandglass.cs:12 ซึ่งถือ [SerializeField] private Sandglass glass; เป็นการอ้างถึงคลาสรูปธรรมในโมดูล Clock โดยตรง
3. **มีการอ้างถึงคลาสรูปธรรมผ่าน [SerializeField] รวม 215 จุดใน 53 ไฟล์** การพึ่งพาเหล่านี้ไม่ปรากฏในซอร์สโค้ดว่าปลายทางคืออ็อบเจกต์ใด ต้องเปิดดูหน้าต่าง Inspector ของ Unity จึงจะทราบ

ด้าน Cohesion โค้ดส่วนรันไทม์ 63 ไฟล์มีความยาวเฉลี่ยประมาณ 55 บรรทัดต่อไฟล์ ซึ่งสะท้อนว่าแต่ละคลาสรับผิดชอบหน้าที่แคบ ตัวอย่างคลาสที่ Cohesion สูงชัดเจน เช่น ClockDial.cs (32 บรรทัด ทำหน้าที่แปลงจำนวนนาทีเป็นมุมของเข็มอย่างเดียว) และ Pendulum.cs (61 บรรทัด) ส่วนคลาสที่ Cohesion ต่ำที่สุดคือ InspectView.cs ซึ่งยาว 317 บรรทัด (นับบรรทัดที่ไม่ว่าง) และรับผิดชอบงานที่นับได้ 8 อย่างในคลาสเดียว รายละเอียดอยู่ในบทที่ 5 หัวข้อ 5.3

2.1.3 สถาปัตยกรรมเชิงองค์ประกอบ (Component-Based Architecture)

สถาปัตยกรรมเชิงองค์ประกอบ หรือที่ในวงการพัฒนาเกมเรียกว่า Component Pattern คือการนิยามว่าอ็อบเจกต์หนึ่งในเกม "เป็นอะไร" ด้วย **ชุดของส่วนประกอบที่ถูกนำมาแปะไว้บนอ็อบเจกต์นั้น** แทนการนิยามด้วยคลาสที่สืบทอดกันเป็นลำดับชั้น Nystrom อธิบายว่าแบบแผนนี้เกิดขึ้นเพื่อแก้ปัญหาที่พบบ่อยในเกม คือเมื่อใช้การสืบทอด อ็อบเจกต์ที่ต้องการคุณสมบัติจากหลายสายพันธุ์พร้อมกันจะไม่มีที่ลงในลำดับชั้นเดียว และสุดท้ายโค้ดจะถูกยกขึ้นไปรวมกันในคลาสฐานจนคลาสฐานบวมผิดรูป [5] Gregory อธิบายโครงสร้างนี้ในบริบทของเอนจินเกมเชิงพาณิชย์ว่าเป็นโมเดลที่เอนจินยุคปัจจุบันส่วนใหญ่เลือกใช้ [6]

Unity ใช้โมเดลนี้เป็นสถาปัตยกรรมพื้นฐานของเอนจิน คือ GameObject เป็นเพียงภาชนะที่ไม่มีพฤติกรรมของตัวเอง พฤติกรรมทั้งหมดมาจาก Component ที่นำมาแปะ และสคริปต์ที่ผู้พัฒนาเขียนขึ้นจะกลายเป็น Component ได้ด้วยการสืบทอดจากคลาส MonoBehaviour [7]

ตำแหน่งที่พบในโครงงาน ตัวอย่างที่ชัดที่สุดคือระบบนาฬิกา ซึ่งเป็นระบบที่ใหญ่ที่สุดในโครงงาน (23 ไฟล์ 1,334 บรรทัด) นาฬิกาทั้ง 46 เรือนในฉากไม่ได้ถูกเขียนเป็นคลาสละเรือน แต่

ประกอบขึ้นจากส่วนประกอบที่ทำหน้าที่ละอย่าง ได้แก่ ClockMovement (เก็บเวลา), ClockDial (แสดงเวลาด้วยเข็ม), Mainspring (แหล่งพลังงานที่ไขลานได้), Pendulum (ลูกตุ้มที่ถ้าถูกหยุดก็หยุดนาฬิกา), ClockWinder (ก้านไขลาน) และ WeightChain (โซ่ตุ้มถ่วง) ชนิดของนาฬิกาจึงเป็น **ผลรวมของส่วนประกอบที่แปะอยู่** ไม่ใช่คลาสลูกที่สืบทอดมา เช่น นาฬิกาที่ไม่มี Mainspring คือนาฬิกาที่ไม่มีวันหมดพลัง ซึ่งถูกต้องสำหรับนาฬิกาที่ใช้ตุ้มถ่วงหรือไฟบ้าน

ระบบสิ่งที่เปิดได้ใช้แนวทางเดียวกัน คือ Openable ถือสถานะเปิดหรือปิดและไม่ขยับอะไรเลย ส่วนการเคลื่อนไหวแยกเป็น SwingMotion (บานพับ) และ SlideMotion (เลื่อนตรง) ดังนั้น ประตู ลิ้นชัก ฝาตู้โชว์ และผนังลับจึงใช้ Openable ตัวเดียวกันทั้งหมด

2.1.4 การประกอบแทนการสืบทอด (Composition over Inheritance)

หลักการ Composition over Inheritance ระบุว่าควรสร้างพฤติกรรมใหม่โดยการ **ประกอบ** อ็อบเจกต์ที่มีอยู่เข้าด้วยกัน แทนการ **สืบทอด** จากคลาสฐาน เหตุผลหลักคือการสืบทอดเป็นการผูกชนิดข้อมูลไว้ตั้งแต่ตอนคอมไพล์และสร้างความขึ้นต่อกันที่แน่นที่สุดแบบหนึ่งระหว่างคลาสลูกกับคลาสแม่ ขณะที่การประกอบเปลี่ยนได้ตอนรันไทม์และกระทบเฉพาะจุดที่ประกอบ [8] Gamma และคณะสรุปหลักนี้ไว้เป็นหนึ่งในสองหลักพื้นฐานของหนังสือ Design Patterns ว่า *"favor object composition over class inheritance"* [8]

ตำแหน่งที่พบในโครงงาน โครงงานนี้ปฏิบัติตามหลักนี้อย่างเคร่งครัดในระดับที่วัดได้เป็นตัวเลข ผลการนับตัวชี้วัดที่ยืนยันการใช้ Composition แทน Inheritance แสดงไว้ในตารางที่ 4.11 ของบทที่ 4

ความหมายเชิงวิศวกรรมของตารางนี้อยู่ที่ค่าที่เป็นศูนย์มากกว่าค่าที่ไม่เป็นศูนย์ คือ **ไม่มีลำดับชั้นการสืบทอดใดในโครงงานเลย** ทั้ง 51 คลาสสืบทอดจาก MonoBehaviour ตรง ๆ ในระดับเดียวกันทั้งหมด และไม่มีการใช้กลไกที่ผูกอ็อบเจกต์เข้าหากันแบบทั่วถึง (global) อย่าง Singleton, FindObjectOfType หรือ SendMessage ซึ่งเป็นแบบแผนที่ทำให้ Coupling สูงโดยที่คอมไพเลอร์ตรวจไม่พบ

กลไกที่โครงงานใช้แทนในการประกาศความขึ้นต่อกันระหว่างส่วนประกอบ คือแอตทริบิวต์ [RequireComponent] ซึ่งพบใน 15 คลาส และ [DisallowMultipleComponent] ซึ่งพบในเกือบทุก MonoBehaviour แอตทริบิวต์แรกทำให้ Unity เพิ่มส่วนประกอบที่จำเป็นให้อัตโนมัติและห้ามลบออก จึงเป็นการประกาศความขึ้นต่อกันที่เอนจินบังคับให้เป็นจริง และช่วยลดปัญหา "ลืมแปะส่วนประกอบ" ซึ่งเป็นผลข้างเคียงหลักของการใช้ Composition

2.1.5 การนำกลับมาใช้ซ้ำ (Reusability)

การนำกลับมาใช้ซ้ำ (Reusability) คือคุณสมบัติที่ทำให้ส่วนของโปรแกรมที่สร้างไว้แล้วถูกนำไปใช้ในที่อื่นได้โดยไม่ต้องเขียนใหม่ มาตรฐาน ISO/IEC 25010 จัดให้เป็นคุณลักษณะย่อยของ Maintainability ร่วมกับ Modularity [4] ทั้งสองเกี่ยวข้องกันโดยตรง เพราะส่วนประกอบที่มีหน้าที่เฉพาะและขึ้นต่อสิ่งอื่นน้อยตามหลัก Low Coupling, High Cohesion จึงนำไปประกอบใช้ซ้ำได้ง่าย ส่วนการสืบทอดผูกคลาสลูกไว้กับคลาสแม่ตั้งแต่ตอนคอมไพล์ ทำให้ใช้ซ้ำได้เฉพาะภายในสายการสืบทอดเดียวกัน [5, 8]

ตำแหน่งที่พบในโครงงาน นาฬิกาทุกเรือนในฉากใช้หน้าปัดจาก ClockDial.prefab ร่วมกัน และใช้ลานสปริงจาก Mainspring.prefab ร่วมกัน (หัวข้อ 2.5.5) ชนิดของนาฬิกาที่ต่างกัน เช่น เดินตรง เดินช้า หยุดเดิน หรือตีบอกเวลา เกิดจากการเลือกว่าจะประกอบส่วนประกอบใดเพิ่มเข้าไป ไม่ได้เกิดจากการเขียนคลาสใหม่ ผลการนับการนำกลับมาใช้ซ้ำนี้อยู่ในบทที่ 4

2.2 หลักการออกแบบเชิงวัตถุที่ใช้ในโครงงาน

2.2.1 หลักการผกผันการพึ่งพา (Dependency Inversion Principle)

Dependency Inversion Principle (DIP) เป็นหนึ่งในหลัก SOLID ที่ Martin เสนอ มีใจความสองข้อ คือ (1) โมดูลระดับสูงต้องไม่ขึ้นต่อโมดูลระดับล่าง ทั้งสองควรขึ้นต่อสิ่งที่เป็นนามธรรม และ (2) สิ่งที่เป็นนามธรรมต้องไม่ขึ้นต่อรายละเอียด รายละเอียดต้องขึ้นต่อสิ่งที่เป็นนามธรรม [9] ผลที่ได้คือทิศทางการพึ่งพาถูก "ผกผัน" จากที่โค้ดไหลไปตามลำดับการเรียกใช้ กลายเป็นว่าทั้งผู้เรียกและผู้ถูกเรียกชี้เข้าหา interface ตัวกลางเดียวกัน Martin เรียกเส้นที่ interface นั้นสร้างขึ้นว่าเป็น **เส้นแบ่งเขตของสถาปัตยกรรม** (architectural boundary) [10]

ตำแหน่งที่พบในโครงงาน โครงงานนี้ใช้ DIP ผ่านรูปแบบที่เรียกได้ว่า *interface-based discovery* คือตัวควบคุมไม่ถือการอ้างอิงไปยังผู้ร่วมงานของตน แต่ค้นหาเอาเองตอนเริ่มทำงานด้วยเมธอดตระกูล GetComponents<T>() โดยระบุเป็น interface ไม่ใช่คลาสรูปธรรม จุดที่ใช้รูปแบบนี้มี 4 จุด เรียกว่า *seam* ของสถาปัตยกรรม ได้แก่ IClockDisplay, IClockPart, ILock และ IPowerStore

ตัวอย่างที่ชัดที่สุดคือ ClockMovement ซึ่งเป็นคลาสที่เก็บเวลาของนาฬิกา

```
// Assets/Scripts/Clock/ClockMovement.cs:112-116
_displays = GetComponentsInChildren<IClockDisplay>(true);
// Inactive ones included: a part that is switched off is one the player has not
// fitted yet, and it has to be in the list so it can start saying no.
```

```
_parts = GetComponentsInChildren<IClockPart>(true);
```

คลาสนี้ **ไม่มีฟิลด์ชื่อ dial** และไม่รู้จักคลาส ClockDial เลย สิ่งที่มันรู้คือมีอะไรบางอย่างที่ implement IClockDisplay อยู่ภายใต้ตัวมัน และมันจะส่งจำนวนนาทีให้ทุกตัวในรายการนั้น ผลคือ ClockMovement (ระดับสูง) ไม่ขึ้นต่อ ClockDial (ระดับล่าง) แต่ทั้งสองขึ้นต่อ IClockDisplay ตรงตามใจความข้อแรกของ DIP

รูปแบบเดียวกันพบที่ Openable ซึ่งถามกลอนทุกตัวที่แปะอยู่กับตัวมัน

```
// Assets/Scripts/Interaction/Openable.cs:20-35
private ILock[] Locks => _locks ?? (_locks = GetComponents<ILock>());
private ILock BlockingLock { get { foreach (ILock fitted in Locks) if (fitted.IsLocked)
return fitted; return null; } }
```

และที่ IPowerStore ซึ่งถูกค้นหาโดยคลาสสามคลาสที่ไม่รู้จักกัน ด้วยบรรทัดที่เหมือนกันทั้งสามที่

```
// ClockWinder.cs:59 · WeightChain.cs:36 · ClockStriker.cs:73
_power = movement.GetComponentInChildren<IPowerStore>(true);
```

ข้อสำคัญคือ interface เหล่านี้มีผู้ implement มากกว่าหนึ่งรายจริง ซึ่งเป็นสิ่งที่แยกการใช้ interface อย่างมีประโยชน์ออกจากการใช้ interface เกินความจำเป็น ดังตารางที่ 2.1

ตารางที่ 2.1 Interface ที่ประกาศในโครงงานและจำนวนผู้ implement

Interface	โมดูล	จำนวน	รายชื่อผู้ implement
IInteractable	Interaction	9	Openable, PickupItem, ClockWinder, ClockSetter, WeightChain, ClockPartSocket, TestRun, CodeWheel, Prism
ILock	Interaction	6	KeyLock, SealedLock, TimeLock, RunningLock, CodeLock, StarLock
IClockPart	Clock	3	Mainspring, Pendulum, TestRun
IItemReceiver	Interaction	2	KeyLock, ClockPartSocket

IClockDisplay	Clock	2	ClockDial, DigitalReadout
IActor	Interaction	1	PlayerInteractor (และ RecordingActor ในชุดทดสอบ)
IInspectable	Interaction	1	Inspectable
IReadable	Interaction	1	WrittenPage
IPowerStore	Clock	1	Mainspring
ICodeInput	Puzzle	1	CodeWheel
IInHandOnly	Interaction	1	TestRun (marker interface ไม่มีสมาชิก)
ITimedTest	Interaction	1	TestRun
รวม		**29 ความสัมพันธ์**	จาก **22 คลาส**

ข้อจำกัดของการใช้งานข้อนี้คือ ICodeInput ยังใช้งานตามเจตนาของมันได้ไม่เต็มที่ เพราะฟิลด์ใน CodeLock ที่ควรเป็น ICodeInput[] ถูกประกาศเป็นคลาสรูปธรรม CodeWheel[] (Assets/Scripts/Puzzle/CodeLock.cs:17) สาเหตุคือ Unity ไม่สามารถ serialize interface ได้โดยตรง รายละเอียดและแนวทางปรับปรุงอยู่ในบทที่ 5 หัวข้อ 5.3

2.2.2 หลักการแยกส่วนต่อประสาน (Interface Segregation Principle)

Interface Segregation Principle (ISP) ระบุว่าไม่ควรบังคับให้ผู้ใช้งานต้องขึ้นต่อเมธอดที่ตนไม่ได้ใช้ interface ที่ใหญ่เกินไปควรถูกแยกออกเป็น interface เล็ก ๆ ตามกลุ่มผู้ใช้งาน [9]

ตำแหน่งที่พบในโครงงาน กรณีที่ชัดเจนที่สุดคือ IActor ซึ่งเป็นสัญญาที่กำหนดว่า "ผู้ที่มาโต้ตอบกับวัตถุ" สามารถถูกขอให้ทำอะไรได้บ้าง ทั้ง interface มีเพียงสามเมธอด

```
// Assets/Scripts/Interaction/IActor.cs:15-26
public interface IActor
{
    void Say(string message);
    void OfferBag(IItemReceiver to);
```

```
    void Take(ItemData item);
}
```

และ PlayerInteractor implement ทั้งสามข้อด้วยโค้ดสามบรรทัด

```
// Assets/Scripts/Player/PlayerInteractor.cs:105-109
public void Say(string message) => hud.ShowMessage(message);
public void OfferBag(IItemReceiver to) => inventoryUI.Open(to);
public void Take(ItemData item) => Inventory.Add(item);
```

ที่สำคัญคือเมธอด IInteractable.Interact() รับพารามิเตอร์เป็น IActor ไม่ใช่ PlayerInteractor (Assets/Scripts/Interaction/Interactions.cs:13) ดังนั้นวัตถุทุกชิ้นในเกม เช่น นาฬิกา ลิ้นชัก วงล้อรหัส จึงมองเห็นเพียงสามเมธอดที่มันต้องใช้ ไม่ได้มองเห็นระยะเอื้อม ความเร็วเดิน หรือระบบการยกของขึ้นดูของผู้เล่น ผลที่วัดได้มีสองอย่าง คือ (1) โมดูล Clock และ Puzzle สามารถใส่ .asmdef ที่ไม่อ้างถึงโมดูล Player ได้ และ (2) ทดสอบได้โดยไม่ต้องเปิดฉากและไม่ต้องมีผู้เล่นอยู่จริง ดังหลักฐานในหัวข้อ 2.4.2

2.2.3 หลักการเปิด–ปิด (Open–Closed Principle)

Open–Closed Principle (OCP) ซึ่ง Meyer เสนอไว้ครั้งแรก [11] และ Martin นำมาขยายในบริบทของ SOLID [9] ระบุว่าหน่วยซอฟต์แวร์ควร **เปิดให้ขยายได้ แต่ปิดการแก้ไข** กล่าวคือ การเพิ่มความสามารถใหม่ควรทำได้ด้วยการเขียนโค้ดใหม่ ไม่ใช่การแก้โค้ดที่ทำงานอยู่แล้ว เพราะการแก้โค้ดเดิมมีความเสี่ยงที่จะทำให้สิ่งที่เคยทำงานได้เสียหาย

ตำแหน่งที่พบในโครงงาน เมื่อประเมินตามเกณฑ์นี้ โครงงานผ่าน 6 สถานการณ์จาก 9 สถานการณ์ ดังตารางที่ 2.2

ตารางที่ 2.2 ผลการประเมินตามหลักการเปิด–ปิด

สิ่งที่ต้องการเพิ่ม	ต้องเขียนใหม่	ต้องแก้ไฟล์เดิมหรือไม่	ผล
วิธีแสดงเวลาแบบใหม่	1 คลาสที่ implement IClockDisplay	ไม่ต้อง	ผ่าน

ตารางที่ 2.2 ผลการประเมินตามหลักการเปิด–ปิด (ต่อ)

สิ่งที่ต้องการเพิ่ม	ต้องเขียนใหม่	ต้องแก้ไฟล์เดิมหรือไม่	ผล
ส่วนประกอบนาฬิกาแบบใหม่	1 คลาสที่ implement IClockPart	ไม่ต้อง	ผ่าน
แหล่งพลังงานแบบใหม่	1 คลาสที่ implement IPowerStore	ไม่ต้อง	ผ่าน
กลอนแบบใหม่	1 คลาสที่ implement ILock	ไม่ต้อง	ผ่าน
วัตถุที่โต้ตอบได้แบบใหม่	1 คลาสที่ implement IInteractable	ไม่ต้อง	ผ่าน
ชนิดนาฬิกาใหม่ทั้งเรือน	ไม่ต้องเขียนโค้ด — สร้างไฟล์ ClockRecipe.asset	ไม่ต้อง	ผ่าน
อุปกรณ์ใส่รหัสแบบใหม่	1 คลาสที่ implement ICodeInput	**ต้องแก้ CodeLock.cs**	ไม่ผ่าน
วิธีมองวัตถุแบบใหม่	—	**ต้องเพิ่มค่าใน enum InspectStyle และแก้ InspectView.cs**	ไม่ผ่าน
สภาพนาฬิกาแบบที่ 6	—	**ต้องเพิ่มค่าใน enum ClockCondition และแก้ ClockMovement.cs**	ไม่ผ่าน

สามสถานการณ์ที่ไม่ผ่านเกิดจากการเลือกใช้ enum ร่วมกับคำสั่ง switch แทนการใช้ polymorphism ซึ่งเป็น **การแลกเปลี่ยนที่ตัดสินใจโดยรู้ตัว** ไม่ใช่ความพลาด เกณฑ์ที่ใช้ตัดสินคือ มิติที่คาดว่ามีค่าจำกัดและรู้ครบแล้ว (closed set) ใช้ enum ส่วนมิติที่คาดว่าจะเพิ่มขึ้นเรื่อย ๆ (open set) ใช้ interface ซึ่งเป็นเกณฑ์ที่ยอมรับได้ในทางวิศวกรรมซอฟต์แวร์ ยกเว้นกรณี ICodeInput ที่เจตนาเดิมต้องการให้เป็น open set แต่ผลลัพธ์ในโค้ดไม่เป็นเช่นนั้น

2.2.4 การแยกความกังวล (Separation of Concerns)

Separation of Concerns เป็นหลักที่ Dijkstra อธิบายไว้ว่าเป็นการพิจารณาเรื่องหนึ่ง ๆ แยกจากเรื่องอื่นในแต่ละครั้ง เพื่อให้สามารถใช้ความคิดกับเรื่องนั้นได้อย่างลึกซึ้งโดยไม่ถูกรบกวนจากเรื่องที่ไม่เกี่ยวข้อง [12] ในทางปฏิบัติของการออกแบบซอฟต์แวร์ หลักนี้แปลเป็นการแยกส่วนที่รับผิดชอบเรื่องต่างกันออกเป็นหน่วยต่างกัน

ตำแหน่งที่พบในโครงงาน โครงงานนี้ใช้หลักนี้อย่างสม่ำเสมอในสามคู่ที่สำคัญ

1. **แยกสถานะออกจากการเคลื่อนไหว** — Openable ถือสถานะเปิดหรือปิด ตรวจสอบกลอน และส่งสัญญาณเหตุการณ์ โดย **ไม่เขียนค่า transform เลยแม้แต่บรรทัดเดียว** (Openable.cs:1-78) ส่วน SwingMotion และ SlideMotion อ่านค่า IsOpen แล้วขยับวัตถุ โดยไม่รู้ว่าเหตุใดจึงเปิด (SwingMotion.cs:25-31, SlideMotion.cs:25-30)
2. **แยกเวลาออกจากการแสดงเวลา** — ClockMovement เก็บจำนวนนาทีและส่งค่าให้ผู้แสดงผล (ClockMovement.cs:155-159) ส่วน ClockDial แปลงนาทีเป็นมุมของเข็ม (ClockDial.cs:22-37) และ DigitalReadout แปลงนาทีเดียวกันเป็นตัวเลขสี่หลัก (DigitalReadout.cs:44-57) โดยสองคลาสหลังนี้ **ไม่ใช้โค้ดร่วมกันเลยแม้แต่บรรทัดเดียว**
3. **แยกตรรกะออกจากเสียง** — TestRun ควบคุมช่วงเวลาของการทดสอบนาฬิกา ส่วน RunTicking เล่นเสียงติ๊กโดยอ่านค่า _run.Allows เท่านั้นและ **ไม่มีตัวจับเวลาของตัวเอง** (RunTicking.cs:39) ทำให้เสียงไม่สามารถคลาดจากการทดสอบได้ ไม่ว่าจะตั้งความยาวของการทดสอบไว้เท่าใด

การแยกนี้ทำได้สม่ำเสมอในโมดูล Clock และ Interaction แต่ยังไม่ได้ทำในสองจุด คือ Horologium.Update() ขยับ transform ของจุดแสงด้วยตัวเอง (Horologium.cs:94-106) และ InspectView เรียกใช้ส่วนแสดงผลโดยตรง (InspectView.cs:166,197,364-365) รายละเอียดอยู่ในบทที่ 5 หัวข้อ 5.3

2.2.5 รูปแบบผู้สังเกตการณ์ (Observer Pattern)

Observer Pattern เป็นแบบแผนเชิงพฤติกรรมที่ Gamma และคณะนิยามไว้ว่า เป็นการกำหนดความสัมพันธ์แบบหนึ่งต่อหลายระหว่างอ็อบเจกต์ เพื่อให้เมื่ออ็อบเจกต์หนึ่งเปลี่ยนสถานะ อ็อบเจกต์ที่ขึ้นต่อทั้งหมดได้รับการแจ้งและปรับตัวเองอัตโนมัติ โดยที่ผู้แจ้งไม่จำเป็นต้องรู้ว่าใครเป็นผู้รับฟัง [8] ในภาษา C# แบบแผนนี้มีกลไกรองรับอยู่ในภาษาโดยตรงด้วยคำสั่ง event [13]

ตำแหน่งที่พบในโครงงาน โครงงานใช้ event Action ของ C# จำนวน 3 คู่ ดังตารางที่ 2.3 และ **ไม่ใช้ UnityEvent เลย**

ตารางที่ 2.3 การใช้ Observer Pattern ในโครงงาน

ผู้ประกาศเหตุการณ์	ผู้รับฟัง	ความหมายในเกม	หลักฐาน
Openable.Opened	GameManager.HandleEscaped	ประตูทางออกเปิด จึงจบเกม	Openable.cs:37,76 · GameManager.cs:25-26,42-47
IPowerStore.Filled	ClockStriker.Strike	มีผู้เติมพลังงานให้นาฬิกานกจึงออกมาขัน	IPowerStore.cs:19 · Mainspring.cs:25,46 · ClockStriker.cs:76,106,110
ICodeInput.Changed	CodeLock.CheckWheels	มีการหมุนวงล้อ จึงตรวจรหัสทันที	CodeWheel.cs:30,63 · CodeLock.cs:43,49,65-76

ตัวอย่างที่แสดงประโยชน์เชิงสถาปัตยกรรมของแบบแผนนี้ได้ชัดที่สุดคือคู่ที่สอง ClockStriker รับฟังเหตุการณ์ว่า "ถูกเติมพลังงาน" ไม่ได้รับฟังว่า "มีคนหมุนก้านไขลาน" ดังนั้นเมื่อนาฬิกาคุกคูถูกเปลี่ยนจากการไขลานด้วย ClockWinder ไปเป็นการดึงโซ่ตุ้มถ่วงด้วย WeightChain คลาส ClockStriker **ไม่ต้องถูกแก้เลยแม้แต่บรรทัดเดียว** เพราะทั้งสองคลาสเรียกเมธอด IPowerStore.Fill() เหมือนกัน (ClockWinder.cs:100, WeightChain.cs:55)

ข้อที่ควรบันทึกไว้ในแง่ความถูกต้องของการใช้งาน คือเหตุการณ์ทั้งสามคู่มีการยกเลิกการรับฟังใน OnDisable ครบทุกคู่ (GameManager.cs:26, ClockStriker.cs:106, CodeLock.cs:46-50) ซึ่งเป็นสิ่งจำเป็นเพื่อป้องกันการรั่วของหน่วยความจำและการเรียกเมธอดบนอ็อบเจกต์ที่ถูกทำลายไปแล้ว

```
// Assets/Scripts/Game/GameManager.cs:25-26 — subscribe และ unsubscribe เป็นคู่
private void OnEnable() => wayOut.Opened += HandleEscaped;
private void OnDisable() => wayOut.Opened -= HandleEscaped;
```

2.2.6 การออกแบบที่ขับเคลื่อนด้วยข้อมูล (Data-Driven Design)

Data-Driven Design คือการย้ายสิ่งที่เคยเป็นค่าคงที่หรือตรรกะในซอร์สโค้ดออกมาเป็นข้อมูลภายนอก เพื่อให้เปลี่ยนแปลงได้โดยไม่ต้องคอมไพล์ใหม่ และเพื่อให้ผู้ที่ไม่ได้เขียนโปรแกรม

สามารถปรับแต่งเนื้อหาได้ Gregory ระบุว่าแนวทางนี้เป็นสิ่งที่เอนจินเกมเชิงพาณิชย์ต้องมี เพราะงานออกแบบเกมมีการปรับค่าซ้ำจำนวนมากและรอบการคอมไพล์เป็นคอขวด [6]

ตำแหน่งที่พบในโครงงาน โครงงานใช้ ScriptableObject ของ Unity (ดูหัวข้อ 2.5.6) เป็นรูปแบบของข้อมูลภายนอก รวม 3 คลาส 12 ไฟล์แอสเซ็ต ได้แก่

- ItemData — นิยามว่าของชิ้นหนึ่งคืออะไร (10 ไฟล์ใน Assets/Scenes/Items/)
- HousePlan — ผังบ้านทั้งหลังในรูปข้อมูล คือสี่เหลี่ยมห้าห้องและรายการแนวผนังพร้อมจุดกลางของช่องเปิด (Assets/Recipes/HousePlan.asset)
- ClockRecipe — สูตรประกอบนาฬิกาหนึ่งชนิด (Assets/Recipes/PlankClock.asset)

กรณี ClockRecipe เป็นกรณีที่น่าสนใจที่สุด เพราะเป็นจุดที่ Data-Driven Design กับ Component-Based Architecture มาบรรจบกัน

```
// Assets/Scripts/Clock/ClockRecipe.cs:19-37 — มีรายการเดียว ไม่แยกว่าอะไรคือจอแสดงผล อะไรคือส่วนประกอบ
[System.Serializable]
public struct Fitting { public GameObject Prefab; public Vector3 At; public Vector3 Turn; [Min(0.01f)] public float Scale; }

[SerializeField] private GameObject caseModel;   // ตัวเรือนเปล่า เป็นรูปทรงเท่านั้น
[SerializeField] private Fitting[] fittings;     // ทุกอย่างที่แปะลงบนเรือน เรียงลำดับใดก็ได้
```

สิ่งสำคัญของโครงสร้างนี้อยู่ที่สิ่งที่มัน **ไม่มี** คือไม่มีรายการแยกสำหรับจอแสดงผลและสำหรับส่วนประกอบ ทั้งสองเป็นเพียง "สิ่งที่นำมาแปะ" เหมือนกัน และ ClockMovement เป็นผู้แยกเองตอน Awake ด้วยการถามว่าสิ่งนั้น implement interface ใด ผลคือ **ชนิดของนาฬิกากลายเป็นไฟล์ข้อมูล ไม่ใช่เมธอดในโค้ด** ซึ่งพิสูจน์แล้วด้วยชุดทดสอบ RecipeSeamTests.cs (ดูหัวข้อ 2.4.2)

2.3 ทฤษฎีความถูกต้องของการนับเวลาในเกม

2.3.1 วงรอบของเกมและความเป็นอิสระจากอัตราเฟรม

เกมแบบเรียลไทม์ทำงานด้วยโครงสร้างที่เรียกว่า **วงรอบของเกม (Game Loop)** คือวงวนที่ทำงานซ้ำไปเรื่อย ๆ โดยในแต่ละรอบจะรับข้อมูลนำเข้า ปรับสถานะของโลกในเกมให้ก้าวหน้าไปหนึ่งขั้น แล้ววาดภาพออกจอหนึ่งเฟรม Nystrom ชี้ว่าประเด็นสำคัญของวงรอบนี้คือวงวนทำงานด้วย

ความเร็วที่ไม่แน่นอน ขึ้นกับความสามารถของเครื่องและปริมาณงานในเฟรมนั้น ดังนั้นการปรับสถานะของโลกจึง **ต้องอ้างตามเวลาจริงที่ผ่านไป ไม่ใช่ตามจำนวนรอบของวงวน** [5]

ถ้าโค้ดปรับสถานะด้วยค่าคงที่ต่อหนึ่งเฟรม เช่น เพิ่มตำแหน่งขึ้นทีละ 0.1 หน่วยทุกเฟรม ผลลัพธ์จะผูกกับอัตราเฟรมของเครื่องที่รัน เครื่องที่วาดได้ 120 เฟรมต่อวินาทีจะเคลื่อนที่เร็วเป็นสองเท่าของเครื่องที่วาดได้ 60 เฟรมต่อวินาที อาการนี้เรียกว่าโค้ดที่ **ขึ้นกับอัตราเฟรม** (frame-rate dependent) และเป็นข้อผิดพลาดที่พบบ่อยที่สุดอย่างหนึ่งในการพัฒนาเกม วิธีแก้มาตรฐานคือคูณอัตราต่อวินาทีด้วยช่วงเวลาจริงของเฟรมนั้น ซึ่งใน Unity คือค่า Time.deltaTime ที่นิยามไว้ว่า *"ช่วงเวลาในหน่วยวินาทีจากเฟรมก่อนหน้ามาถึงเฟรมปัจจุบัน"* [14]

ตำแหน่งที่พบในโครงงาน การนับเวลาทุกจุดในโครงงานใช้ Time.deltaTime เป็นฐาน ตัวอย่างเช่น

```
// Assets/Scripts/Clock/ClockMovement.cs:136
double advance = (double)minutesPerSecond * Rate * Time.deltaTime;
```

```
// Assets/Scripts/Clock/Sandglass.cs:107 — นาฬิกาทรายนับเวลาที่ผ่านไป
_run = Mathf.Min(_run + Time.deltaTime, seconds);
```

```
// Assets/Scripts/Clock/TestRun.cs:77
_ran += Time.deltaTime;
```

หลักฐานยืนยันเชิงลบที่มีน้ำหนัก คือการค้นหาคำว่า Time.frameCount ทั้งโครงงาน **ไม่พบการใช้งานเลยแม้แต่ที่เดียว** กล่าวคือไม่มีจุดใดในโครงงานที่นับเวลาด้วยจำนวนเฟรม

2.3.2 ความแม่นยำของเลขทศนิยมและความคลาดเคลื่อนสะสม

เลขทศนิยมแบบจุดลอยตัว (floating-point) ตามมาตรฐาน IEEE 754 เก็บค่าในรูปของแมนทิสซาคูณด้วยสองยกกำลังเลขชี้กำลัง ผลคือ **ระยะห่างระหว่างเลขที่แทนได้สองค่าที่อยู่ติดกันไม่คงที่ แต่ใหญ่ขึ้นตามขนาดของค่า** ระยะห่างนี้เรียกว่า ULP (Unit in the Last Place) [15] Goldberg อธิบายไว้ว่าผลสำคัญของคุณสมบัตินี้คือ เมื่อนำเลขขนาดใหญ่มากบวกกับเลขขนาดเล็กมาก ส่วนของเลขเล็กที่ตกอยู่ใต้ ULP ของผลลัพธ์จะถูกปัดทิ้งไปทั้งหมด ปรากฏการณ์นี้เรียกว่าการสูญเสียนัยสำคัญ (loss of significance) และจะสะสมตัวขึ้นเมื่อเกิดซ้ำหลายพันครั้ง [16]

ชนิดข้อมูล float ตามมาตรฐาน IEEE 754 binary32 มีแมนทิสซา 24 บิต ส่วน double (binary64) มี 53 บิต [15] เมื่อนำมาคำนวณกับค่าที่เกิดขึ้นจริงในโครงงานนี้ จะได้ผลตามตารางที่ 2.4

ตารางที่ 2.4 ระยะห่างระหว่างเลขที่แทนได้ ณ ค่าสะสม 280 นาที (เวลา 4:40)

ชนิดข้อมูล	แมนทิสซา	ULP ณ ค่า 280	การคำนวณ
float (binary32)	24 บิต	$\approx 3.05 \times 10^{-5}$ นาที	$2^{8} \times 2^{-23} = 2^{-15}$
double (binary64)	53 บิต	$\approx 5.68 \times 10^{-14}$ นาที	$2^{8} \times 2^{-52} = 2^{-44}$

นาฬิกาในเกมนี้ทำงานโดยนำจำนวนนาทีที่เพิ่มขึ้นในหนึ่งเฟรมไปบวกกับจำนวนนาทีสะสมที่มีอยู่แล้ว ซึ่งเป็นรูปแบบของการอินทิเกรตค่าที่ละนิดเข้าสู่ผลรวมที่ใหญ่ขึ้นเรื่อย ๆ นับพันครั้งต่อการเล่นหนึ่งครั้ง จากตารางที่ 2.4 ULP ของ float ณ ค่า 280 อยู่ที่ประมาณ 3×10^{-5} นาที ขณะที่นาฬิกาในห้องโถงถูกตั้งค่า minutesPerSecond ไว้ที่ 1/60 ซึ่งที่อัตรา 60 เฟรมต่อวินาทีหมายถึงค่าที่เพิ่มขึ้นต่อเฟรมประมาณ 2.8×10^{-4} นาที กล่าวคือค่าที่นำมาบวกอยู่ห่างจาก ULP ของผลรวมเพียงประมาณหนึ่งลำดับความสำคัญเท่านั้น ซึ่งเป็นช่วงที่การปัดทิ้งเริ่มให้ผลที่วัดได้ และเลวร้ายลงอีกเมื่อนาฬิกาเดินช้า เพราะค่าที่เพิ่มต่อเฟรมยิ่งเล็กลง

หลักทั่วไปที่สรุปได้จากเรื่องนี้ และเป็นหลักที่ใช้ได้กับทุกระบบที่สะสมค่า คือ **ผลรวมที่กำลังสะสมและค่าที่นำมาบวกเข้าไป ต้องอยู่ภายในช่วงความแม่นยำของชนิดข้อมูลนั้นเมื่อเทียบกันเอง** ระบบใดที่อินทิเกรตค่าเล็กเข้าสู่ผลรวมใหญ่ต่อเนื่องหลายพันเฟรม ย่อมเผชิญปัญหาเดียวกัน

ตำแหน่งที่พบในโครงงาน ClockMovement เก็บจำนวนนาทีสะสมไว้ใน double ไม่ใช่ float

```
// Assets/Scripts/Clock/ClockMovement.cs:50
private double _minutes;
```

โดยส่วนต่อประสานที่เปิดให้ภายนอกยังคงเป็น float (เมธอด Minutes, Advance, Show) เพราะการแปลงชนิดหนึ่งครั้งที่ปลายทางมีผลต่อความคลาดเคลื่อนเพียงประมาณ 0.01 องศาและเกิดครั้งเดียว ส่วนที่ทำใน float ไม่ได้คือ **การสะสม** ดังนั้นทุกอย่างที่บวกเข้ากับ _minutes จึงต้องอยู่ในรูป double ตลอดทาง รวมถึงการวนรอบ 12 ชั่วโมงด้วย เพราะเมธอด Mathf.Repeat ของ Unity รับเฉพาะ float จึงต้องทำการวนรอบในเมธอด OnDial แทน (ClockMovement.cs:163-166)

2.3.3 ลำดับการทำงานของฟังก์ชันเหตุการณ์

ใน Unity สคริปต์ที่สืบทอดจาก MonoBehaviour จะถูกเอนจินเรียกใช้ผ่านฟังก์ชันเหตุการณ์ (event function) ตามลำดับที่กำหนดไว้ในวงรอบของเฟรม โดยฟังก์ชันที่เกี่ยวข้องกับโครงงานนี้คือ Awake (เรียกครั้งเดียวเมื่อสร้างอ็อบเจกต์), Update (เรียกทุกเฟรม) และ LateUpdate (เรียกทุกเฟรม **หลังจาก** Update ของทุกสคริปต์ทำงานครบแล้ว) [17]

ประเด็นที่สำคัญต่อความถูกต้องของโปรแกรม และเป็นสิ่งที่เอกสารของ Unity ระบุไว้เป็นข้อจำกัดอย่างชัดเจน คือ **ลำดับการเรียกฟังก์ชันเหตุการณ์เดียวกันระหว่างอ็อบเจกต์ต่างกันนั้นไม่รับประกัน** เอกสารใช้ถ้อยคำว่า *"โดยทั่วไป ไม่สามารถเชื่อถือลำดับที่ฟังก์ชันเหตุการณ์เดียวกันถูกเรียกใช้สำหรับ GameObject ต่างกันได้ ยกเว้นในกรณีที่ลำดับนั้นถูกระบุไว้ในเอกสารหรือสามารถตั้งค่าได้"* และระบุต่อว่าลำดับระหว่างสคริปต์ต่างชนิดกันสามารถตั้งค่าได้ด้วย Script Execution Order แต่ลำดับระหว่างอินสแตนซ์ของสคริปต์เดียวกันนั้นตั้งไม่ได้เลย [17]

ผลที่ตามมาคือ หากสองส่วนประกอบที่ต้องทำงานเรียงลำดับกันเขียนโค้ดไว้ใน Update ทั้งคู่ โดยไม่ได้ตั้งค่า Script Execution Order ผลลัพธ์ของโปรแกรมจะไม่แน่นอน และอาจต่างกันในแต่ละเครื่องหรือแต่ละครั้งที่รัน วิธีแก้ที่ไม่ต้องพึ่งการตั้งค่าในโปรเจกต์ คือย้ายส่วนที่ต้องทำงานทีหลังไปอยู่ใน LateUpdate เพราะเอนจินรับประกันว่า LateUpdate ทำงานหลัง Update ของทุกสคริปต์

ตำแหน่งที่พบในโครงงาน คลาส TestRun ซึ่งควบคุมช่วงเวลาของการทดสอบนาฬิกา นับเวลาใน LateUpdate ขณะที่ ClockMovement เดินเข็มใน Update

```
// Assets/Scripts/Clock/TestRun.cs:72-81
private void LateUpdate()
{
    if (!_running) return;
    _ran += Time.deltaTime;
    if (_ran >= Length) _running = false;
}
```

นอกจากนี้ยังมีกลไกที่เกี่ยวข้องอีกสองอย่างในโครงงาน คือ

1. **การคิดเฟรมสุดท้ายเป็นเศษส่วน** — TestRun คืนค่าตัวคูณอัตราเป็นสัดส่วนของเวลาที่เหลือต่อความยาวของเฟรมที่กำลังจะใช้ไป ทำให้เฟรมที่คร่อมเส้นสิ้นสุดถูกนับเพียงเท่าที่เหลือจริง เข็มจึงหยุดตรงขีดเดิมเสมอไม่ว่าเฟรมจะยาวเท่าใด

```
// Assets/Scripts/Clock/TestRun.cs:52
```

```
  public float RateScale => _running ? Mathf.Clamp01((Length - _ran) /
Time.deltaTime) : 1f;
```

2. **การตั้งลำดับการทำงานของ InputReader** — คลาสนี้ใช้แอตทริบิวต์ [DefaultExecutionOrder(-100)] (InputReader.cs:8) เพื่อให้แน่ใจว่าอ็อบเจกต์ PlayerControls ถูกสร้างขึ้นก่อนที่คลาสอื่นจะเข้ามาลงทะเบียนรับฟังเหตุการณ์ใน OnEnable ของตนเอง ซึ่งเป็นการใช้กลไกตั้งลำดับอย่างตรงจุดในที่ที่จำเป็นจริง

2.4 ทฤษฎีการทดสอบซอฟต์แวร์

2.4.1 ระดับและชนิดของการทดสอบ

การทดสอบซอฟต์แวร์แบ่งตามขอบเขตของสิ่งที่ถูกทดสอบได้เป็นหลายระดับ โดยสองระดับที่เกี่ยวข้องกับโครงงานนี้คือ

- **Unit test** — ทดสอบหน่วยย่อยที่สุดของโค้ด เช่น เมธอดหรือคลาสเดียว โดยแยกออกจากสิ่งแวดล้อมภายนอกทั้งหมด Meszaros ระบุคุณสมบัติที่ unit test ที่ดีควรมีไว้ว่าต้องทำงานเร็ว ทำงานได้โดยไม่ต้องพึ่งระบบภายนอก และให้ผลเหมือนกันทุกครั้งที่รัน [18]
- **Integration test** — ทดสอบว่าหลายหน่วยทำงานร่วมกันได้ถูกต้อง ซึ่งจำเป็นต้องมีสิ่งแวดล้อมบางส่วนอยู่จริง

ใน Unity ความแตกต่างนี้สะท้อนออกมาเป็นสองโหมดของ Unity Test Framework คือ **EditMode test** ซึ่งรันในกระบวนการของตัวแก้ไข (Editor) โดยไม่ต้องเริ่มเกม และ **PlayMode test** ซึ่งรันในสภาพแวดล้อมเวลาทำงานจริงของเอนจิน จึงสามารถใช้ฟังก์ชันเหตุการณ์และให้เฟรมเดินไปข้างหน้าได้ [19] ความแตกต่างนี้สำคัญต่อการเขียนรายงานให้ถูกต้อง เพราะการทดสอบที่ต้องอาศัยการเดินของเฟรมนั้นไม่ใช่ unit test ตามความหมายที่เคร่งครัด

ชุดทดสอบของโครงงานนี้เป็นการทดสอบแบบ PlayMode ซึ่งรันภายในเอนจินเท่านั้น ไม่มี EditMode test และไม่มี unit test ที่รันนอกเอนจิน รายงานฉบับนี้จึงเรียกชุดทดสอบนี้ว่า PlayMode test ทุกครั้ง

2.4.2 การทดสอบความเหมาะสมของสถาปัตยกรรม (Architecture Fitness Function)

Ford, Parsons และ Kua เสนอแนวคิด **Architecture Fitness Function** ว่าเป็นการทดสอบอัตโนมัติชนิดหนึ่งที่ไม่ได้ตรวจว่าฟังก์ชันคืนค่าถูกต้องหรือไม่ แต่ตรวจว่า **คุณลักษณะเชิงสถาปัตยกรรมที่ออกแบบไว้ยังเป็นจริงอยู่หรือไม่** ตัวอย่างเช่นการตรวจว่าโมดูลหนึ่งไม่อ้างถึงอีก

โมดูลหนึ่ง หรือการตรวจว่าจุดต่อที่ออกแบบให้ขยายได้นั้นขยายได้จริง ประโยชน์ของการทดสอบชนิดนี้คือทำให้ข้ออ้างทางสถาปัตยกรรมกลายเป็นสิ่งที่ตรวจสอบได้โดยอัตโนมัติ ไม่ใช่เพียงข้อตกลงที่เขียนไว้ในเอกสารแล้วค่อย ๆ ถูกละเมิดไปตามเวลา [20]

ตำแหน่งที่พบในโครงงาน โครงงานมี PlayMode test รวม **15 กรณีใน 3 ไฟล์** และลักษณะของการทดสอบเหล่านี้ตรงกับนิยามของ Architecture Fitness Function มากกว่านิยามของ unit test ทั่วไป ดังตารางที่ 2.5

ตารางที่ 2.5 ไฟล์ทดสอบและคุณลักษณะเชิงสถาปัตยกรรมที่แต่ละไฟล์พิสูจน์

ไฟล์	บรรทัด	กรณี	พิสูจน์ว่า
ActorSeamTests.cs	198	6	IActor ตัดการพึ่งพาคลาส PlayerInteractor ได้จริง
ClockSeamTests.cs	124	6	IClockDisplay และ IClockPart เป็นจุดต่อที่ขยายได้จริง
RecipeSeamTests.cs	71	3	นาฬิกาที่ประกอบจากไฟล์ข้อมูลทำงานเหมือนที่เขียนด้วยโค้ด

กรณีทดสอบที่แสดงลักษณะนี้ได้ชัดที่สุดสองกรณี คือ

กรณีที่หนึ่ง คลาสจอแสดงผลถูกประกาศขึ้นใหม่ **ภายในไฟล์ทดสอบ** ซึ่งหมายความว่าคลาสนี้ยังไม่มีอยู่ในขณะที่ ClockMovement ถูกคอมไพล์ หากการทดสอบผ่าน ย่อมพิสูจน์ได้ว่า ClockMovement ไม่ได้ผูกกับคลาสจอแสดงผลใดเป็นการเฉพาะ ซึ่งคือข้ออ้างของ DIP ในหัวข้อ 2.2.1 นั่นเอง

```
// Assets/Tests/PlayMode/ClockSeamTests.cs:41-67
private class CountingDisplay : MonoBehaviour, IClockDisplay
{
    public int Shown; public float Last;
    public void Show(float minutes) { Shown++; Last = minutes; }
}

[UnityTest]
public IEnumerator A_display_written_after_the_movement_still_gets_the_time()
```

```
{
    GameObject go = Clock(out ClockMovement movement);
    var display = go.AddComponent<CountingDisplay>();
    Set(movement, "startHour", 4); Set(movement, "startMinute", 40);
    go.SetActive(true);
    yield return null;
    Assert.Greater(display.Shown, 0, "the movement never called Show");
    Assert.AreEqual(4 * 60 + 40, display.Last, 1f, "it was handed the wrong time");
}
```

กรณีที่สอง สัญญา IActor ทั้งสัญญาถูก implement ด้วยคลาส C# ธรรมดาความยาวประมาณสิบบรรทัด ที่ไม่ใช่ MonoBehaviour และไม่ต้องอยู่ในฉากใดเลย แล้วนำคลาสนั้นไปเปิดตู้ไขกุญแจ และประกอบชิ้นส่วนนาฬิกาได้สำเร็จ ซึ่งพิสูจน์ข้ออ้างของ ISP ในหัวข้อ 2.2.2

```
// Assets/Tests/PlayMode/ActorSeamTests.cs:18-27
private class RecordingActor : IActor
{
    public readonly List<string> Said = new List<string>();
    public readonly List<IItemReceiver> BagOfferedTo = new List<IItemReceiver>();
    public readonly List<ItemData> Taken = new List<ItemData>();
    public void Say(string message) => Said.Add(message);
    public void OfferBag(IItemReceiver to) => BagOfferedTo.Add(to);
    public void Take(ItemData item) => Taken.Add(item);
}
```

ข้อจำกัดของชุดทดสอบนี้คือครอบคลุมเฉพาะโมดูล Items, Interaction และ Clock ไม่มีกรณีทดสอบใดครอบคลุมโมดูล Puzzle, World, Player, UI หรือ Game และในกรณีของสามโมดูลหลัง เหตุผลเป็นข้อจำกัดทางเทคนิคของ Unity ที่ชุดทดสอบซึ่งมี .asmdef อ้างถึงโมดูลใน predefined assembly ไม่ได้ (ดูหัวข้อ 2.5.7) รายละเอียดอยู่ในบทที่ 4 และข้อเสนอแนะอยู่ในบทที่ 5 หัวข้อ 5.3

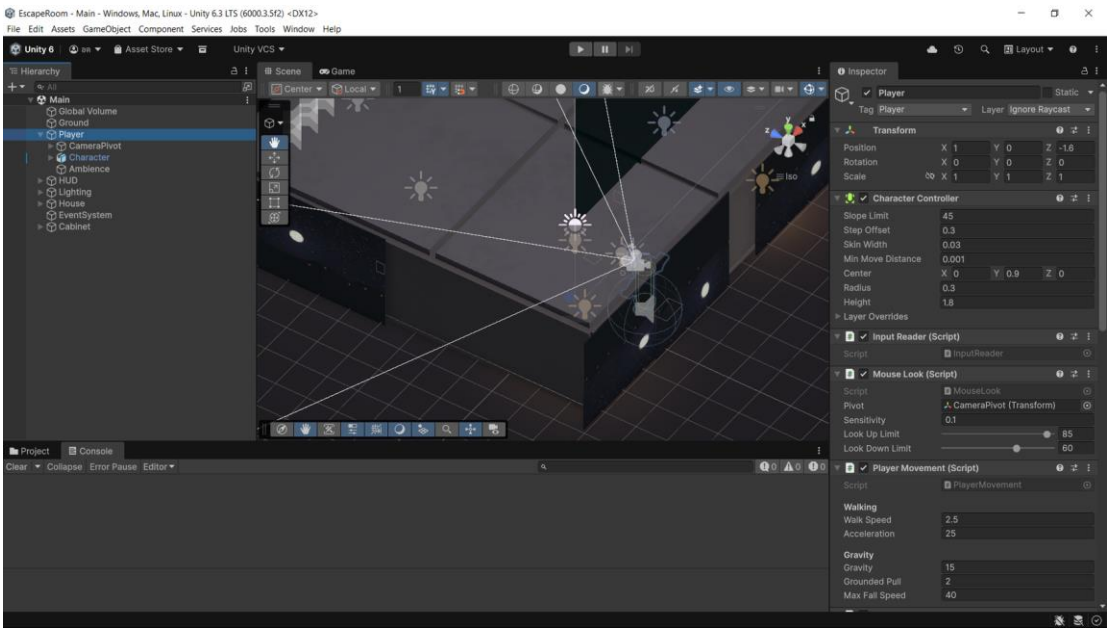
2.5 เทคโนโลยีที่ใช้ในการปฏิบัติงาน

2.5.1 Unity Engine

Unity เป็นเอนจินเกมสำหรับสร้างเนื้อหาสองมิติและสามมิติที่รองรับการส่งออกไปหลายแพลตฟอร์ม โครงสร้างพื้นฐานของเอนจินใช้โมเดล GameObject–Component ตามที่อธิบายไว้ในหัวข้อ 2.1.3 โดยหน่วยที่ใช้จัดองค์ประกอบของโลกในเกมเรียกว่า Scene และหน่วยที่ใช้เก็บแม่แบบของอ็อบเจกต์ที่นำกลับมาใช้ซ้ำเรียกว่า Prefab

โครงงานนี้ใช้ **Unity เวอร์ชัน 6000.3.5f2** โดยทั้งเกมอยู่ใน Scene เพียงหนึ่งฉาก คือ Assets/Scenes/Main.unity ขนาดประมาณ 2.35 เมกะไบต์ และเป็นฉากเดียวที่อยู่ในรายการ Build Settings ยืนยันจากไฟล์ EditorBuildSettings.asset ซึ่งมีรายการ m_Scenes เพียงรายการเดียว และจากเมธอด GameManager.Restart() (Assets/Scripts/Game/GameManager.cs:30) ที่เรียก SceneManager.LoadScene ด้วยหมายเลขของฉากที่กำลังทำงานอยู่ กล่าวคือโหลดฉากเดิมซ้ำ เพราะไม่มีฉากอื่นให้ไป

ผลของการใช้ฉากเดียวคือไม่ต้องมีระบบคงสถานะข้ามฉาก (ยืนยันจากการค้นหา DontDestroyOnLoad ได้ผลลัพธ์ศูนย์รายการ) แต่ก็หมายความว่าไม่มีระบบบันทึกและโหลดเกม และเมนูหลักเป็นแผงส่วนติดต่อผู้ใช้ภายในฉากเดียวกัน ไม่ใช่ฉากแยก



ภาพที่ 2.1 ส่วนติดต่อผู้ใช้ของ Unity Editor ในโครงงาน

จากภาพที่ 2.1 หน้าต่าง Hierarchy แสดงรายการวัตถุทั้งหมดในฉาก ส่วนหน้าต่าง Inspector แสดงส่วนประกอบที่แปะอยู่บนวัตถุที่เลือก ซึ่งเป็นการแสดงให้เห็นสถาปัตยกรรมเชิงองค์ประกอบตามหัวข้อ 2.1.3 ว่าวัตถุหนึ่งชิ้นคือชุดของส่วนประกอบที่นำมาแปะรวมกัน

2.5.2 Universal Render Pipeline (URP)

Universal Render Pipeline เป็นไปป์ไลน์การเรนเดอร์สำเร็จภาพที่สร้างขึ้นบน Scriptable Render Pipeline ของ Unity เอกสารของ Unity อธิบายไว้ว่า URP *"ให้ลำดับงานที่เป็นมิตรต่อศิลปิน ซึ่งช่วยให้สร้างกราฟิกที่ถูกปรับแต่งประสิทธิภาพแล้วได้อย่างรวดเร็วและง่ายดาย ครอบคลุมแพลตฟอร์มตั้งแต่อุปกรณ์พาหะไปจนถึงเครื่องเล่นเกมและเครื่องคอมพิวเตอร์ระดับสูง"* [21] ความแตกต่างที่สำคัญในทางปฏิบัติคือ URP ใช้เชดเดอร์และคุณสมบัติของวัสดุชุดของตนเอง ซึ่งไม่เข้ากันกับเชดเดอร์มาตรฐานของไปป์ไลน์แบบเดิม

โครงงานนี้ใช้ **URP เวอร์ชัน 17.3.0** ยืนยันจากไฟล์ Packages/manifest.json และจากไฟล์ตั้งค่า Assets/Settings/UniversalRenderPipelineGlobalSettings.asset และยืนยันในระดับการใช้งานจริงจากสองจุด คือวัสดุในโครงงานใช้คุณสมบัติชื่อ _BaseMap และ _EmissionMap ซึ่งเป็นชื่อคุณสมบัติของ URP และ Assets/Scripts/Game/RoomLighting.cs:62 เรียกใช้ RenderSettings.reflectionIntensity

2.5.3 ภาษา C#

C# เป็นภาษาโปรแกรมเชิงวัตถุที่มีการตรวจชนิดข้อมูลตอนคอมไพล์ (statically typed) และทำงานบนสภาพแวดล้อมที่มีการจัดการหน่วยความจำอัตโนมัติ ข้อกำหนดของภาษาถูกรับรองเป็นมาตรฐาน ECMA-334 [13] Unity ใช้ C# เป็นภาษาเดียวสำหรับการเขียนสคริปต์

คุณสมบัติของภาษาที่โครงงานนี้ใช้อย่างมีนัยสำคัญต่อสถาปัตยกรรมมีสามข้อ

1. **interface** — ใช้เป็นสัญญากลางที่ทำให้เกิด DIP และ ISP ตามหัวข้อ 2.2.1 และ 2.2.2 รวม 12 interface
2. **event** — ใช้เป็นกลไกของ Observer Pattern ตามหัวข้อ 2.2.5 โครงงานใช้ event Action ของเนมสเปซ System และ **ไม่ประกาศ delegate ของตัวเองเลย** (ยืนยันจากการค้นหาคำว่า delegate ได้ผลลัพธ์ศูนย์รายการ)
3. **double กับ float** — ใช้แยกบทบาทกันตามหัวข้อ 2.3.2 คือใช้ double สำหรับการสะสมค่า และ float สำหรับส่วนต่อประสานที่เชื่อมกับเอนจิน

2.5.4 MonoBehaviour และวงจรชีวิตของส่วนประกอบ

MonoBehaviour เป็นคลาสฐานที่สคริปต์ของ Unity จำนวนมากสืบทอดมา เอกสารอ้างอิงของ Unity ระบุว่าคลาสนี้ *"มีฟังก์ชันวงจรชีวิตที่ทำให้การพัฒนาด้วย Unity ง่ายขึ้น"* และอ็อบเจกต์ของคลาสนี้ดำรงอยู่ในฐานะ Component ของ GameObject เสมอ [7] ฟังก์ชันวงจรชีวิตที่สำคัญและลำดับการเรียกใช้อธิบายไว้ในหัวข้อ 2.3.3

โครงงานนี้มีคลาสที่สืบทอดจาก MonoBehaviour ทั้งหมด **51 คลาสใน 51 ไฟล์** โดยทุกคลาสสืบทอดโดยตรงในระดับเดียวกัน ไม่มีคลาสใดสืบทอดจากคลาสอื่นในโครงงาน และลำดับของสมาชิกในแต่ละคลาสถูกจัดเรียงตามแบบแผนเดียวกันทุกไฟล์ คือค่าคงที่ ตามด้วยฟิลด์ที่ serialize ได้ ตามด้วยฟิลด์สถานะภายใน ตามด้วยเหตุการณ์และคุณสมบัติสาธารณะ ตามด้วยฟังก์ชันวงจรชีวิตเรียงตามลำดับการเรียกใช้ ตามด้วยเมธอดสาธารณะและเมธอดภายใน

2.5.5 Prefab

Prefab คือระบบของ Unity สำหรับเก็บ GameObject พร้อมส่วนประกอบ ค่าคุณสมบัติ และอ็อบเจกต์ลูกทั้งหมด ไว้ในรูปของแอสเซ็ตที่นำกลับมาใช้ซ้ำได้ เอกสารของ Unity อธิบายว่าแอสเซ็ต Prefab ทำหน้าที่เป็นแม่แบบที่ใช้สร้างอินสแตนซ์ของ Prefab ขึ้นในฉาก [22] และการแก้ไขที่แม่แบบจะส่งผลไปยังทุกอินสแตนซ์ที่ยังเชื่อมโยงอยู่

โครงงานนี้ใช้ Prefab เป็นหน่วยของการนำกลับมาใช้ซ้ำอย่างกว้างขวาง ตัวอย่างที่ชัดเจนที่สุดคือ Assets/Prefabs/Clocks/ClockDial.prefab ซึ่งเป็นหน้าปัดนาฬิกาที่ใช้ร่วมกันโดยนาฬิกาทุกเรือน พบอินสแตนซ์ของคลาส ClockDial ในฉากรวม **46 อินสแตนซ์** และ Assets/Prefabs/Clocks/Parts/Mainspring.prefab พบ 45 อินสแตนซ์

จำนวนนาฬิกาในเกมคือ **46 เรือน** นับจากจำนวนอินสแตนซ์ของส่วนประกอบ ClockMovement ในฉากจริง

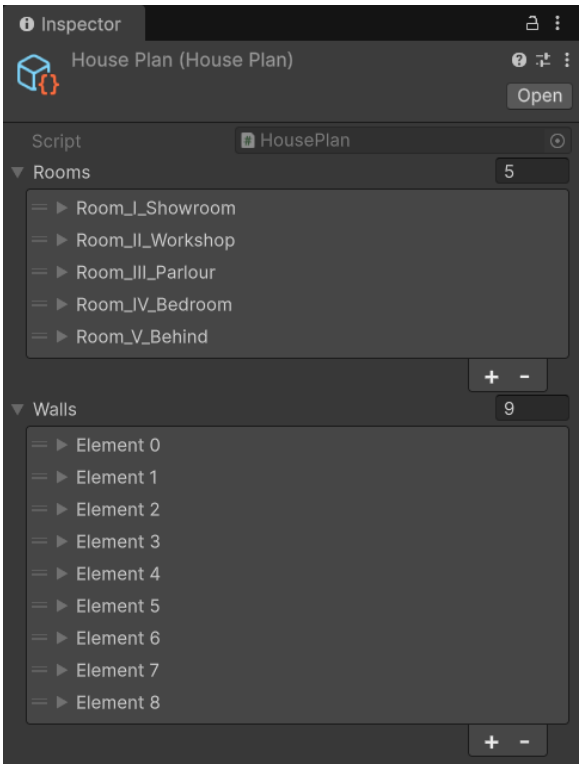
กลไกของ Prefab ยังเป็นที่มาของข้อผิดพลาดประเภทหนึ่งที่พบในโครงงานนี้ คือ **ค่าเริ่มต้นที่เขียนไว้ในซอร์สโค้ดของฟิลด์ที่มี [SerializeField] จะถูกเขียนทับด้วยค่าที่ถูก serialize ไว้ในแอสเซ็ตเสมอตอนที่ Unity โหลดอ็อบเจกต์** ดังนั้นการแก้ค่าเริ่มต้นในโค้ดจึงไม่มีผลต่อ Prefab ที่ถูกสร้างไปก่อนหน้านั้นแล้ว กรณีที่พบจริงในโครงงานและผลกระทบอยู่ในบทที่ 5 หัวข้อ 5.2

2.5.6 ScriptableObject

ScriptableObject เป็นชนิดข้อมูลของ Unity ที่สืบทอดจาก UnityEngine.Object และ serialize ได้ ต่างจาก MonoBehaviour ที่ว่า **อ็อบเจกต์ของมันไม่ได้แปะอยู่กับ GameObject ในฐานะ Component แต่ดำรงอยู่ในโปรเจกต์ในฐานะแอสเซ็ต เป็นอิสระจาก GameObject**

เอกสารของ Unity ระบุว่าคุณค่าหลักของ ScriptableObject คือการเป็นที่เก็บข้อมูล และการใช้งานที่พบบ่อยคือการเป็นภาชนะสำหรับข้อมูลที่อ็อบเจกต์หลายตัวใช้ร่วมกันตอนทำงาน ซึ่งช่วยลดการใช้หน่วยความจำโดยหลีกเลี่ยงการทำสำเนาค่าเดียวกันหลายชุด [23]

โครงงานนี้ใช้ ScriptableObject เป็นกลไกของ Data-Driven Design ตามหัวข้อ 2.2.6 รวม 3 คลาส และ 12 ไฟล์แอสเซ็ต ได้แก่ ItemData, HousePlan และ ClockRecipe



ภาพที่ 2.2 ข้อมูลผังบ้านที่เก็บในรูปแอสเซ็ต ScriptableObject

จากภาพที่ 2.2 ไฟล์ HousePlan.asset เก็บรายการห้องและแนวผนังทั้งหมดของบ้านไว้เป็นข้อมูล การย้ายห้องหรือขยายห้องจึงเป็นการแก้ตัวเลขในไฟล์นี้ แล้วสั่งสร้างโครงบ้านใหม่ ไม่ต้องแก้โปรแกรม

2.5.7 Assembly Definition

Assembly Definition (ไฟล์นามสกุล .asmdef) เป็นกลไกของ Unity ที่ใช้แบ่งสคริปต์ในโปรเจกต์ออกเป็นแอสเซมบลีหลายชุด แทนที่จะคอมไพล์รวมอยู่ในแอสเซมบลีที่เอนจินสร้างให้โดยปริยาย (predefined assembly) ซึ่งมีชื่อว่า Assembly-CSharp ประโยชน์คือลดเวลาคอมไพล์เมื่อแก้โค้ด และ **ทำให้ขอบเขตการพึ่งพาระหว่างกลุ่มโค้ดกลายเป็นข้อบังคับระดับคอมไพเลอร์**

กฎการอ้างอิงที่สำคัญสามข้อซึ่งเอกสารของ Unity ระบุไว้อย่างชัดเจน คือ [24]

1. โดยปริยาย **predefined assembly อ้างอิงแอสเซมบลีอื่นทั้งหมด** รวมถึงแอสเซมบลีที่สร้างด้วย Assembly Definition เอกสารใช้ถ้อยคำว่า *"คลาสใน predefined assembly สามารถใช้ชนิดข้อมูลทั้งหมดที่นิยามไว้ในแอสเซมบลีอื่นใดในโปรเจกต์ได้"*

2. Unity **ไม่อนุญาต** ให้แอสเซมบลีที่สร้างด้วย Assembly Definition อ้างอิงไปยัง predefined assembly

3. Unity **ไม่อนุญาต** การอ้างอิงแบบวงจร (cyclical reference) คือกรณีที่สองแอสเซมบลีอ้างถึงกันไปกลับ เอกสารระบุว่าหากพบข้อผิดพลาดนี้ ต้องปรับโครงสร้างโค้ดเพื่อตัดวงจรออก หรือย้ายคลาสที่อ้างถึงกันไปอยู่ในแอสเซมบลีเดียวกัน

กฎข้อ 1 และ 2 ร่วมกันให้ผลที่มีประโยชน์เชิงสถาปัตยกรรมอย่างมาก คือ **การวางโค้ดชั้นล่างไว้ในไฟล์ .asmdef และวางโค้ดชั้นบนไว้ใน Assembly-CSharp เป็นการบังคับทิศทางการพึ่งพาให้เป็นทางเดียวโดยอัตโนมัติ** โดยไม่ต้องอาศัยวินัยของผู้เขียนโค้ด

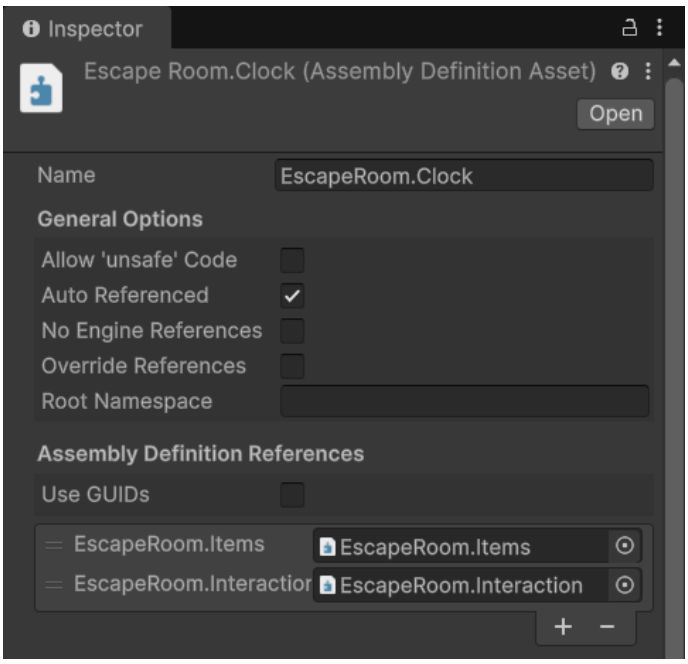
ตำแหน่งที่พบในโครงงาน โครงงานใช้กลไกนี้ตรงตามรูปแบบที่กล่าวมา คือโมดูลชั้นล่างห้าโมดูลมี .asmdef ของตนเอง (EscapeRoom.Items, EscapeRoom.Interaction, EscapeRoom.Clock, EscapeRoom.Puzzle, EscapeRoom.World) ส่วนโมดูลชั้นบนสามโมดูล (Player, UI, Game) ถูกปล่อยไว้ใน Assembly-CSharp ผลคือการเขียนคำสั่งที่อ้างถึงคลาสของผู้เล่นจากภายในโมดูลนาฬิกา เช่น GetComponent<PlayerMovement>() ภายใน ClockDial **จะคอมไพล์ไม่ผ่าน** ตารางการพึ่งพาในหัวข้อ 2.1.2 จึงไม่ใช่เพียงข้อตกลง แต่เป็นข้อผิดพลาดของคอมไพเลอร์

เหตุผลที่ไม่แบ่ง .asmdef ให้ครบทั้งเก้าโมดูล มีข้อจำกัดทางเทคนิคบังคับอยู่สองข้อ

1. **มีวงจร Player ⇄ UI อยู่จริง** ซึ่งขัดกับกฎข้อ 3 โดยตรง สองโมดูลนี้จึงใส่ .asmdef แยกกันไม่ได้ ต้องตัดวงจรก่อน หรือรวมเป็นแอสเซมบลีเดียว

2. **ไฟล์ PlayerControls.cs อยู่ที่ Assets/ ไม่ได้อยู่ใต้ Assets/Scripts/** หากสร้าง EscapeRoom.Player.asmdef ขึ้น คลาสในนั้นจะมองไม่เห็นคลาส PlayerControls เพราะอยู่ต่างแอสเซมบลีกัน และตามกฎข้อ 2 แอสเซมบลีใหม่นั้นจะอ้างถึง predefined assembly ที่ PlayerControls อยู่ไม่ได้ ต้องย้ายไฟล์ก่อน

ข้อจำกัดที่พบเพิ่มเติมคือ ไฟล์ EscapeRoom.Puzzle.asmdef ประกาศการอ้างอิงไปยัง EscapeRoom.Items และ EscapeRoom.Clock แต่โค้ดในโฟลเดอร์ Puzzle/ ไม่ได้ใช้ชนิดข้อมูลจากสองโมดูลนั้นเลย (รายการเดียวที่พบคือคำว่า IClockDisplay ในคอมเมนต์ที่ ICodeInput.cs:5) และ EscapeRoom.Tests.asmdef ประกาศอ้าง EscapeRoom.Puzzle ที่ไม่ได้ใช้เช่นกัน การประกาศเกินนี้ไม่กระทบการทำงาน แต่ทำให้ขอบเขตของโมดูลหลวมกว่าที่โค้ดต้องการจริง รายละเอียดอยู่ในบทที่ 5 หัวข้อ 5.3



ภาพที่ 2.3 การประกาศขอบเขตการอ้างอิงของโมดูลนาฬิกาด้วยไฟล์ Assembly Definition

จากภาพที่ 2.3 ไฟล์ EscapeRoom.Clock.asmdef ระบุรายการแอสเซมบลีที่โมดูลนาฬิกาอ้างถึงได้ คือ EscapeRoom.Items และ EscapeRoom.Interaction เท่านั้น การอ้างถึงโมดูลอื่นนอกรายการนี้จะทำให้โปรเจกต์คอมไพล์ไม่ผ่าน

2.5.8 Unity Input System

Unity Input System เป็นแพ็กเกจระบบรับข้อมูลนำเข้ารุ่นใหม่ของ Unity ที่แยกการกระทำในเกม (action) ออกจากอุปกรณ์และปุ่มที่ใช้สั่งการกระทำนั้น (binding) โดยนิยามไว้ในไฟล์ .inputactions และ Unity จะสร้างคลาส C# ให้อัตโนมัติจากไฟล์นั้น นอกจากนี้ยังมีแนวคิดของ **Action Map** คือกลุ่มของการกระทำที่เปิดหรือปิดพร้อมกันเป็นชุด [25]

โครงงานนี้ใช้ **Input System เวอร์ชัน 1.17.0** โดยคลาสที่ถูกสร้างอัตโนมัติคือ Assets/PlayerControls.cs และสถาปัตยกรรมการรับข้อมูลนำเข้าของโครงงานมีสองข้อตกลงที่สำคัญ

ข้อแรก **คลาส InputReader เป็นเจ้าของอ็อบเจกต์ PlayerControls เพียงตัวเดียวของเกมทั้งเกม** สร้างใน Awake และคืนทรัพยากรด้วย Dispose() ใน OnDestroy (InputReader.cs:19-20) เหตุผลคือเกมมีการสลับ Action Map สามชุด หากมีคลาสอื่นสร้างอ็อบเจกต์ PlayerControls ขึ้นอีกตัว Action Map ที่ตัวควบคุมโหมดเข้าใจว่าปิดไปแล้วจะยังเปิดอยู่

ข้อสอง **วิธีอ่านข้อมูลนำเข้าถูกเลือกตามชนิดของค่า** ดังตารางที่ 2.6

ตารางที่ 2.6 ข้อตกลงการอ่านข้อมูลนำเข้าตามชนิดของค่า

ชนิดของค่า	ตัวอย่างในเกม	วิธีอ่าน	ตำแหน่งในโค้ด
ค่าต่อเนื่อง	การเดิน การหันมอง	เรียก ReadValue() ทุกเฟรม (polling)	PlayerMovement.cs:43, MouseLook.cs:35
เหตุการณ์การกดครั้งเดียว	ปุ่ม E, F, Q, Submit, Navigate	ลงทะเบียนรับฟังเหตุการณ์ performed	PlayerInteractor.cs:51-52, InspectView.cs:121-123, InventoryUI.cs:43-45

เหตุผลเชิงวิศวกรรมของข้อตกลงนี้คือ ปุ่มที่อ่านด้วยการ polling ทุกเฟรมจะถูกอ่านซ้ำหรือถูกอ่านพลาด ขึ้นกับว่าเฟรมตกลงตรงจังหวะใดของการกด ขณะที่ค่าต่อเนื่องที่อ่านด้วยการรับฟังเหตุการณ์จะได้ค่าเฉพาะขณะที่ค่าเปลี่ยน ไม่ได้ค่าในขณะที่ผู้เล่นกดปุ่มค้างไว้

ข้อตกลงที่ตามมาอีกข้อหนึ่งคือ **การกระทำที่ถูกปิดจะคืนค่าโดยปริยาย (default) อยู่แล้ว จึงไม่ต้องล้างค่า แต่ค่าที่ถูกเก็บสะสมไว้ในฟิลด์ต้องถูกล้างใน OnDisable** มิฉะนั้นค่านั้นจะข้ามไปอยู่ในการเปิดใช้งานครั้งถัดไป โครงงานปฏิบัติตามข้อนี้จริงที่ PlayerMovement.cs:59-63 (ล้างค่าความเร็วและความเร็วการตก), HeadBob.cs:71-76, Flicker.cs:47-53, CuckooBird.cs:41-44 และ RunTicking.cs:51-56

2.5.9 Unity Test Framework และ NUnit

Unity Test Framework เป็นแพ็กเกจที่ใช้เขียนและรันการทดสอบอัตโนมัติใน Unity โดยสร้างอยู่บนเฟรมเวิร์ก **NUnit** ซึ่งเป็นเฟรมเวิร์กการทดสอบหน่วยสำหรับภาษาในตระกูล .NET [26] และรองรับการทดสอบสองโหมดตามที่อธิบายไว้ในหัวข้อ 2.4.1 คือ EditMode และ PlayMode [19]

โครงงานนี้ใช้ **Unity Test Framework เวอร์ชัน 1.6.0** ยืนยันจาก Packages/manifest.json และจากไฟล์ Assets/Tests/PlayMode/EscapeRoom.Tests.asmdef ซึ่งประกาศการอ้างอิงไปยัง UnityEngine.TestRunner, UnityEditor.TestRunner และ nunit.framework.dll พร้อมตั้งค่า defineConstraints เป็น ["UNITY_INCLUDE_TESTS"] เพื่อให้แอสเซมบลีนี้ถูกคอมไพล์เฉพาะเมื่อมีการรวมชุดทดสอบเท่านั้น

กรณีทดสอบในโครงงานใช้แอตทริบิวต์ [UnityTest] ร่วมกับเมธอดที่คืนค่าชนิด IEnumerator และคำสั่ง yield return null ซึ่งเป็นรูปแบบที่ทำให้เฟรมเดินไปข้างหน้าหนึ่งเฟรม

ระหว่างการทดสอบ จำเป็นสำหรับการทดสอบสิ่งที่ทำงานใน Update และ LateUpdate ส่วนการตรวจผลลัพธ์ใช้คลาส Assert ของ NUnit

2.5.10 การสร้างเนื้อหาด้วยกระบวนวิธีและการเขียนสคริปต์สำหรับตัวแก้ไข

การสร้างเนื้อหาด้วยกระบวนวิธี (Procedural Content Generation หรือ PCG) คือการสร้างเนื้อหาของเกม เช่น ภูมิประเทศ ด่าน พื้นผิว หรือรูปทรง ด้วยอัลกอริทึม แทนการสร้างด้วยมือทีละชิ้น Hendrikx และคณะจัดหมวดเนื้อหาที่สร้างด้วยวิธีนี้ไว้หลายระดับ ตั้งแต่ระดับพื้นผิวและรูปทรง ไปจนถึงระดับการวางผังด่านและกฎของเกม [27] และ Shaker, Togelius กับ Nelson อธิบายว่าประโยชน์หลักของแนวทางนี้คือการลดต้นทุนการผลิตเนื้อหา และการทำให้เนื้อหาเปลี่ยนแปลงได้จากการแก้พารามิเตอร์ [28]

ในกรณีของ Unity การสร้างเนื้อหาด้วยกระบวนวิธีมักทำผ่านการเขียนสคริปต์สำหรับตัวแก้ไข (Editor Scripting) คือสคริปต์ที่ทำงานในตัวแก้ไขเท่านั้นและไม่ถูกรวมไปในเกมที่ส่งออก โดยวางไว้ในโฟลเดอร์ชื่อ Editor ซึ่งทำให้ Unity คอมไพล์เข้าแอสเซมบลี Assembly-CSharp-Editor

ตำแหน่งที่พบในโครงงาน โครงงานนี้มีเครื่องมือสร้างเนื้อหาในโฟลเดอร์ Assets/Scripts/Editor/ รวม **11 ไฟล์ 5,772 บรรทัด** ซึ่งมากกว่าโค้ดส่วนรันไทม์ทั้งหมด (3,481 บรรทัด) โดยทุกตัวเลขบรรทัดในรายงานนี้นับเฉพาะบรรทัดที่ไม่ว่าง เครื่องมือเหล่านี้สร้างทั้งผังบ้าน รูปทรงของวัตถุ พื้นผิว และ Prefab โดยมีรายการตามตารางที่ 2.7

ตารางที่ 2.7 เครื่องมือสร้างเนื้อหาด้วยกระบวนวิธีในโครงงาน

เครื่องมือ	สิ่งที่สร้าง	ขอบเขตที่แก้ไข
HouseBuilder	พื้น ผนัง ฝ้า ช่องเปิด และคิ้ว จากข้อมูลใน HousePlan.asset	กลุ่มอ็อบเจกต์ House/Shell เท่านั้น
DressingBuilder	เครื่องเรือนและของตกแต่งในแต่ละห้อง	กลุ่ม Dressing ของแต่ละห้องเท่านั้น
ClockBuilder, ClockVariants	Prefab นาฬิกาและหน้าปัด	Assets/Prefabs/Clocks/
ClockAssembler	Prefab นาฬิกาจากไฟล์ ClockRecipe.asset	PlankClock.prefab

PuzzleBuilder, MissingThingsBuilder	วัตถุประกอบปริศนาแต่ละองก์	วัตถุเฉพาะตัว
NewspaperBuilder	หนังสือพิมพ์บนโต๊ะ	Room_I_Showroom/Newspaper
TextureKit	พื้นผิวทั้งหมดในเกม เขียนเป็นไฟล์ลง Assets/Textures	Assets/Textures/
BuildKit, InterfaceBuilder	ฟังก์ชันช่วยที่เครื่องมืออื่นเรียกใช้	—

ประเด็นทางวิศวกรรมที่สำคัญของแนวทางนี้คือ **ความเป็น idempotent ของตัวสร้างเนื้อหา เมื่อเทียบกับข้อมูลที่มนุษย์แก้ไขไว้** กล่าวคือเครื่องมือที่ทำงานโดยการลบของเดิมแล้วสร้างใหม่ จะกลืนงานที่คนแก้ไขไว้ด้วยมือไปด้วย โครงงานนี้แก้ปัญหานี้ด้วยสองวิธีที่พบหลักฐานในฉากจริง คือ (ก) การย้ายวัตถุที่คนแก้ไขออกไปอยู่นอกขอบเขตของเครื่องมือ ยืนยันจากตำแหน่งในฉากที่ Room_II_Workshop/Sandglass และ Room_I_Showroom/Newspaper เป็นอ็อบเจกต์ลูกโดยตรงของห้อง ไม่ได้อยู่ใต้กลุ่ม Dressing ขณะที่ของตกแต่งอื่นอยู่ใต้ Dressing ทั้งหมด และ (ข) การอ่านค่าที่คนเขียนไว้ออกมาเก็บก่อน แล้วใส่กลับหลังสร้างใหม่

2.5.11 เครื่องมือพัฒนาอื่น

เครื่องมือพัฒนาอื่นที่พบหลักฐานการใช้งานในโครงงาน ได้แก่

1. **แพ็กเกจ uGUI เวอร์ชัน 2.0.0** ใช้สร้างส่วนติดต่อผู้ใช้ ยืนยันจากคำสั่ง using UnityEngine.UI และการใช้ชนิดข้อมูล Text แบบเดิม (legacy), Graphic, Canvas และ RectTransform
2. **ส่วนขยายสำหรับสภาพแวดล้อมการพัฒนา** คือ com.unity.ide.rider เวอร์ชัน 3.0.38 และ com.unity.ide.visualstudio เวอร์ชัน 2.0.26 และพบโฟลเดอร์ .vscode/ ในโปรเจกต์

ไฟล์ Packages/manifest.json มีรายการ dependency รวม 46 รายการ แต่ส่วนใหญ่เป็นโมดูลที่ Unity ใส่มาตามแม่แบบของโปรเจกต์ ไม่ใช่การตัดสินใจทางเทคนิคของโครงงาน และบางส่วน เช่น com.unity.ai.navigation และ com.unity.timeline ไม่พบการใช้งานในซอร์สโค้ด

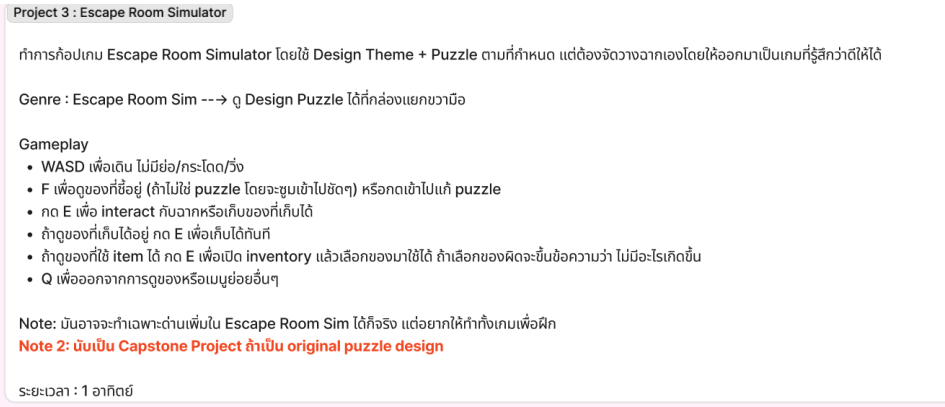
หรือในฉากเลย บทนี้จึงกล่าวถึงเฉพาะเทคโนโลยีที่พบหลักฐานการใช้งานจริงตามหัวข้อ 2.5.1 ถึง 2.5.11

บทที่ 3
แผนงานการปฏิบัติงานและขั้นตอนการดำเนินงาน

การปฏิบัติงานสหกิจศึกษา ณ บริษัท Lumas corp co., LTD. ในตำแหน่งผู้ช่วยพัฒนาเกม มีโครงงานหลักคือเกมห้องหลบหนีสามมิติมุมมองบุคคลที่หนึ่ง (Escape Room Simulator) พัฒนาด้วยโปรแกรม Unity โดยพนักงานที่ปรึกษากำหนดแนวเกม รูปแบบการควบคุม และเงื่อนไขการเล่นมาให้ ส่วนการออกแบบปริศนา การวางฉาก และการออกแบบโครงสร้างซอฟต์แวร์ เป็นงานที่นักศึกษาดำเนินการเอง เนื่องจากโครงงานนี้มีวัตถุประสงค์เพื่อฝึกกระบวนการพัฒนาเกมสามมิติ การดำเนินงานจึงแบ่งเป็นสี่ขั้นตามลำดับของกระบวนการพัฒนาเกม คือ การออกแบบแนวคิดและเนื้อหา การออกแบบระบบเกม การพัฒนาโปรแกรม และการทดสอบการทำงานของเกม ดังภาพที่ 3.2 ในหัวข้อ 3.3

3.1 แผนการทำโครงงาน

โครงงานเริ่มจากเอกสารโจทย์หนึ่งหน้าที่ได้รับจากพนักงานที่ปรึกษา ระบุชื่องานว่า Project 3 : Escape Room Simulator กำหนดให้พัฒนาเกมแนวห้องหลบหนีทั้งเกม มีรูปแบบการควบคุม คือ W A S D สำหรับเดิน F สำหรับดูวัตถุ E สำหรับโต้ตอบหรือเก็บของ และ Q สำหรับออกจากการดูวัตถุหรือเมนูย่อย ระบุว่างานจะนับเป็น Capstone Project หากการออกแบบปริศนาเป็นผลงานต้นฉบับ และกำหนดระยะเวลา 1 สัปดาห์สำหรับออกแบบธีมและปริศนาเพื่อนำเสนอแนวทางของเกม ดังภาพที่ 3.1



Project 3 : Escape Room Simulator

ทำการก้อปเกม Escape Room Simulator โดยใช้ Design Theme + Puzzle ตามที่กำหนด แต่ต้องจัดวางฉากเองโดยให้ออกมาเป็นเกมที่รู้สึกว่าดีให้ได้

Genre : Escape Room Sim ---> ดู Design Puzzle ได้ที่กล่องแยกขวามือ

Gameplay

- WASD เพื่อเดิน ไม่มีย่อ/กระโดด/วิ่ง
- F เพื่อดูของที่ซื้อยู่ (ถ้าไม่ใช่ puzzle โดยจะซูมเข้าไปชัดๆ) หรือกดเข้าไปแก้ puzzle
- กด E เพื่อ interact กับฉากหรือเก็บของที่เก็บได้
- ถ้าดูของที่เก็บได้อยู่ กด E เพื่อเก็บได้ทันที
- ถ้าดูของที่ใช้ item ได้ กด E เพื่อเปิด inventory แล้วเลือกของมาใช้ได้ ถ้าเลือกของผิดจะขึ้นข้อความว่า ไม่มีอะไรเกิดขึ้น
- Q เพื่อออกจากการดูของหรือเมนูย่อยอื่นๆ

Note: มันอาจจะทำเฉพาะด่านเพิ่มใน Escape Room Sim ได้ก็จริง แต่อยากให้ทำทั้งเกมเพื่อฝึก

Note 2: นับเป็น Capstone Project ถ้าเป็น original puzzle design

ระยะเวลา : 1 อาทิตย์

ภาพที่ 3.1 โจทย์ของโครงงานที่ได้รับจากพนักงานที่ปรึกษา

ที่มา : เอกสารโจทย์ Project 3 : Escape Room Simulator จากพนักงานที่ปรึกษา บริษัท Lumas corp co., LTD.

จากภาพที่ 3.1 เงื่อนไขข้อสุดท้ายของโจทย์มีผลโดยตรงต่อการวางแผน เพราะกลไกการเล่นที่โจทย์กำหนดมาแล้วสามารถลงมือพัฒนาได้ทันที แต่ปริศนาต้องใช้เวลาออกแบบก่อนจึงจะเขียนโปรแกรมได้ นักศึกษาจึงวางแผนการปฏิบัติงานตลอดระยะเวลาฝึกสหกิจศึกษา 4 เดือน เป็น 5 กิจกรรม ดังตารางที่ 3.1

ตารางที่ 3.1 แผนการฝึกสหกิจศึกษา ระหว่างวันที่ 1 กรกฎาคม ถึง 31 ตุลาคม พ.ศ. 2569

กิจกรรม	กรกฎาคม 2569				สิงหาคม 2569				กันยายน 2569				ตุลาคม 2569			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1. ศึกษาข้อมูลของโครงงาน	■	■	■	■	■	■	■									
2. วางแผนการทำโครงงาน						■	■	■								
3. ดำเนินโครงงาน								■	■	■						
4. ตรวจสอบโครงงานและแก้ไขปัญหาการดำเนินงาน										■	■	■	■			
5. จัดทำรายงานและเอกสารประกอบ													■	■	■	■

โครงงานนี้มีวัตถุประสงค์เพื่อฝึกกระบวนการพัฒนาเกมสามมิติ โดยใช้เกมห้องหลบหนีเป็นขอบเขตของงาน ขอบเขตแบ่งได้เป็นสองส่วน คือส่วนที่โจทย์กำหนดมา กับส่วนที่นักศึกษากำหนดขึ้นเอง

ส่วนที่โจทย์กำหนด คือรูปแบบการควบคุมและการโต้ตอบ ตามภาพที่ 3.1 ตารางที่ 3.2 เทียบข้อกำหนดแต่ละข้อกับส่วนของโปรแกรมที่รองรับ

ตารางที่ 3.2 ข้อกำหนดจากโจทย์และส่วนของโปรแกรมที่รองรับ

ข้อกำหนดจากโจทย์	ส่วนของโปรแกรมที่รองรับ	สถานะ
ใช้ปุ่ม WASD เพื่อเดิน ไม่มีการย่อ กระโดด หรือวิ่ง	PlayerMovement.cs มีเฉพาะการเดินและแรงโน้มถ่วง	ทำครบ
ปุ่ม F เพื่อดูวัตถุใกล้ ๆ โดยถ้าเป็นปริศนาจะซูมเข้าไป	InspectStyle กำหนดรูปแบบการดูไว้สามแบบ คือ Hold Turn และ Zoom	ทำครบและขยายเพิ่ม

ตารางที่ 3.2 ข้อกำหนดจากโจทย์และส่วนของโปรแกรมที่รองรับ (ต่อ)

ข้อกำหนดจากโจทย์	ส่วนของโปรแกรมที่รองรับ	สถานะ
ปุ่ม E เพื่อโต้ตอบกับฉากหรือเก็บของ	PlayerInteractor.cs ส่งคำสั่งให้วัตถุที่อยู่ใต้เป้าเล็งผ่านส่วนต่อประสาน IInteractable	ทำครบ
ถ้าเป็นของที่เก็บได้ กด E เพื่อเก็บทันที	PickupItem.cs ซึ่งเป็นหนึ่งในผู้ทำตาม IInteractable	ทำครบ
ถ้าเป็นสิ่งที่ใช้ไอเทมได้ กด E เพื่อเปิดกระเป๋าแล้วเลือกของ	IItemReceiver ร่วมกับ InventoryUI.cs	ทำครบ
ถ้าเลือกของผิดให้ขึ้นข้อความว่า "ไม่มีอะไรเกิดขึ้น"	InventoryUI.cs แสดงข้อความนี้เมื่อ IItemReceiver.Give คืนค่าเท็จ	ทำครบ
ปุ่ม Q เพื่อออกจากการดูของหรือเมนูย่อย	PlayerModeController.cs สลับชุดคำสั่งกลับสู่โหมดสำรวจ	ทำครบ
ทำทั้งเกม ไม่ใช่เพิ่มด่านในเกมที่มีอยู่แล้ว	โปรเจกต์สร้างขึ้นใหม่ทั้งหมด มีฉากเดียวคือ Main.unity	ทำครบ
การออกแบบปริศนาต้องเป็นผลงานต้นฉบับ	ปริศนาทั้งห้าองก์นักศึกษาออกแบบเอง	ทำครบ

ส่วนที่นักศึกษากำหนดขึ้นเอง คือเนื้อหาของเกม ได้แก่ ฉากเป็นร้านนาฬิกาเก่าห้าห้องเรียงต่อกันเป็นเส้นทางเดียว ผู้เล่นต้องแก้ปริศนาของห้องหนึ่งจึงจะเปิดประตูไปห้องถัดไปได้ และปริศนาทั้งห้าองก์ใช้นาฬิกาเป็นองค์ประกอบหลัก ดังตารางที่ 3.3

ตารางที่ 3.3 ขอบเขตเนื้อหาของเกมที่นักศึกษากำหนดขึ้นเอง

องก์	**ห้อง**	**ขนาดห้อง (เมตร)**	**แก่นของปริศนา**
1	หน้าร้าน	6.0 × 5.0	แยกนาฬิกาที่ยังเดินออกจากนาฬิกาที่เสีย แล้วอ่านตัวเลขตามลำดับที่กำหนด
2	ห้องช่าง	6.0 × 6.8	จับเวลานาฬิกาพกด้วยนาฬิกาทราย เพื่อหาเรือนที่เดินช้า

3	ห้องนั่งเล่น	6.8 × 6.8	นับเสียงร้องของนกสามเรือน
4	ห้องนอน	6.8 × 5.0	เรียงลำดับวัตถุสี่ชิ้นตามภาพถ่ายเก่า
5	หลังตู้นาฬิกา	9.2 × 7.5	หมุนปริซึมสามชิ้นให้จุดแสงตกตรงกับกลุ่มดาวบนฉาก

3.2 รายละเอียดขั้นตอนการทำโครงงาน

3.2.1 ประชากรและกลุ่มตัวอย่าง

โครงงานนี้เป็นการพัฒนาเกมเพื่อฝึกกระบวนการพัฒนาซอฟต์แวร์ ไม่ใช่การสำรวจความคิดเห็น จึงไม่มีประชากรเชิงสถิติ กลุ่มตัวอย่างมีเพียงผู้ทดลองเล่นเกม คือเพื่อนของนักศึกษาจำนวน 5 คน ซึ่งเล่นเกมจนจบคนละหนึ่งรอบ วิธีทดลองเล่นอยู่ในหัวข้อ 3.3.4 และผลการสังเกตอยู่ในหัวข้อ 4.2.8

3.2.2 เครื่องมือและอุปกรณ์ที่ใช้ในการทำโครงงาน

เครื่องมือที่ใช้ในการพัฒนา ได้แก่ เอนจินและแพ็กเกจที่ติดตั้งในโปรเจกต์ และเครื่องมือเขียนโปรแกรม ดังตารางที่ 3.4

ตารางที่ 3.4 เครื่องมือที่ใช้ในการพัฒนาโครงงาน

กลุ่ม	เครื่องมือ	รุ่น	ใช้ทำอะไร	อ้างอิง
เอนจิน	Unity Editor	6000.3.5f2	เอนจินหลักของเกม	[7]
ไปป์ไลน์เรนเดอร์	Universal Render Pipeline	17.3.0	ไปป์ไลน์การเรนเดอร์ของโปรเจกต์	[21]
ภาษาโปรแกรม	C#	ตามรุ่นที่ Unity 6 รองรับ	ภาษาที่ใช้เขียนทั้งโครงงาน	[13]

ระบบรับคำสั่ง	Unity Input System	1.17.0	กำหนดชุดคำสั่งสามชุดและสร้างคลาส PlayerControls ให้อัตโนมัติ	[25]
กรอบการทดสอบ	Unity Test Framework	1.6.0	รันชุดทดสอบแบบ PlayMode	[19]
ไลบรารีข้อยืนยัน	NUnit	มาพร้อม Test Framework	เขียนข้อยืนยันในชุดทดสอบ	[26]
ตัวแก้ไขโค้ด	Visual Studio Code และ JetBrains Rider	com.unity.ide.visualstudio 2.0.26 และ com.unity.ide.rider 3.0.38	เขียนและแก้ไขซอร์สโค้ด	—

ทรัพยากรที่ไม่ได้สร้างขึ้นในโครงงานมีสามกลุ่ม ดังตารางที่ 3.5

ตารางที่ 3.5 ทรัพยากรที่นำเข้าจากภายนอกโครงงาน

ทรัพยากร	จำนวน	ตำแหน่งในโปรเจกต์
โมเดลตัวละครพร้อมภาพพื้นผิวและแอนิเมชัน	73 ไฟล์ (ในจำนวนนี้เป็น .fbx 6 ไฟล์)	Assets/Character Models/
ไฟล์เสียง	4 ไฟล์	Assets/Sound/ คือ clock-bg.mp3, clock-bell.mp3, clock-cuckoo.mp3 และ clock-ticking-down.mp3

ไฟล์ที่มาพร้อมแม่แบบ Universal 3D ของ Unity	30 ไฟล์	Assets/Settings/ และ Assets/InputSystem_Actions .inputactions

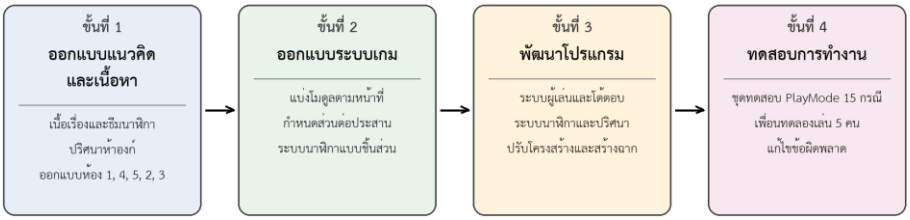
โมเดลตัวละครมาจากชุดทรัพยากรของ Zenva Academy (academy.zenva.com) ซึ่งนักศึกษาได้รับจากการซื้อคอร์สเรียน Unity และนำมาใช้ต่อในโครงงานนี้ ทรัพยากรอื่นทั้งหมดในเกมได้แก่ ภาพพื้นผิวทุกภาพ วัสดุ 115 ไฟล์ เรขาคณิตของฉากและวัตถุทุกชิ้น รวมถึงภาพในกรอบรูปภาพถ่ายเก่าที่เป็นคำใบ้ขององค์ที่ 4 และเป้าเล็งบนหน้าจอ ถูกสร้างขึ้นด้วยซอร์สโค้ดในโครงงานเอง

3.2.3 การเก็บรวบรวมข้อมูล

ข้อมูลที่เก็บในโครงงานนี้มีสามอย่าง อย่างแรกคือผลการรันชุดทดสอบอัตโนมัติ 15 กรณี อย่างที่สองคือข้อความใน Console ของ Unity ว่ามีข้อผิดพลาดหรือไม่ และอย่างที่สามคือข้อสังเกตจากการนั่งดูเพื่อน 5 คนเล่นเกมและความเห็นที่สอบถามหลังเล่นจบ นักศึกษาจดข้อมูลทั้งสามอย่างด้วยตนเอง วิธีเก็บข้อมูลอยู่ในหัวข้อ 3.3.4 และผลอยู่ในหัวข้อ 4.2.7 ถึง 4.2.9

3.3 ขั้นตอนการดำเนินงานที่นักศึกษาปฏิบัติงานหรือโครงงาน

การดำเนินงานแบ่งเป็นสี่ขั้นตามกระบวนการพัฒนาเกม ดังภาพที่ 3.2 ขั้นที่ 1 ถึง 2 เป็นการออกแบบ ขั้นที่ 3 เป็นการลงมือพัฒนา และขั้นที่ 4 เป็นการทดสอบ ในทางปฏิบัติขั้นเหล่านี้เหลื่อมกัน เช่น ขณะพัฒนาโปรแกรมก็ต้องกลับไปปรับการออกแบบระบบ แต่ในหัวข้อนี้เล่าแยกเป็นขั้นเพื่อให้เห็นว่าแต่ละขั้นทำอะไรและได้อะไร



ภาพที่ 3.2 ขั้นตอนการดำเนินงานสี่ขั้นของโครงงาน

3.3.1 ขั้นที่ 1 การออกแบบแนวคิดและเนื้อหา

เนื้อเรื่อง ช่างนาฬิกาเจ้าของร้านหายตัวไปประมาณ 5 วัน ผู้เล่นเป็นญาติห่าง ๆ ที่ได้รับมรดกเป็นบ้านหลังนี้ และต้องเข้าไปในบ้านเพื่อสืบหาสิ่งที่เกิดขึ้น บ้านประกอบด้วยห้าห้องเรียงต่อกันเป็นเส้นทางเดียว แต่ละห้องมีปริศนาหนึ่งชุด และผู้เล่นต้องแก้ปริศนาให้ได้กุญแจหรือกลไกที่เปิดทางไปห้องถัดไป

ธีมนาฬิกา นักศึกษากำหนดแนวทางการออกแบบซอฟต์แวร์แบบมอดูลาร์ไว้ก่อนเลือกธีมของปริศนา แล้วจึงเลือกนาฬิกามาเป็นองค์ประกอบหลักของปริศนาทั้งห้าองก์ เพราะนาฬิกามีส่วนประกอบพื้นฐานร่วมกัน คือหน้าปัดและกลไก แต่แสดงพฤติกรรมต่างกันได้ เช่น เดินตรง เดินช้า หยุดเดิน และตีบอกเวลา จึงทำให้เห็นการนำคอมโพเนนต์กลับมาใช้ซ้ำได้ชัดเจน ธีมนี้เข้ากับเนื้อเรื่องด้วย เพราะบ้านของช่างนาฬิกาย่อมเต็มไปด้วยนาฬิกา

การออกแบบปริศนา ปริศนาออกแบบทีละห้อง ตามลำดับ ห้อง 1 ห้อง 4 ห้อง 5 ห้อง 2 และห้อง 3 ห้อง 3 เป็นห้องสุดท้ายที่ออกแบบ เหตุผลการออกแบบของแต่ละองก์เป็นดังตารางที่ 3.6

ตารางที่ 3.6 แนวคิดและเหตุผลการออกแบบปริศนาแต่ละองก์

องก์	สิ่งที่ผู้เล่นต้องทำ	เหตุผลในการออกแบบ
1 หน้าร้าน	พบคู่มือดูแลนาฬิกาที่เขียนว่าต้องไขลานทุกสัปดาห์ จึงอนุมานได้ว่าช่างไขไว้เฉพาะบางเรือนก่อนหายตัวไป มีเรือนที่ยังเดินและเรือนที่หยุด ไขนาฬิกาแต่ละเรือนเพื่อให้ได้ตัวเลข แล้วเรียงตัวเลขของเรือนที่ยังเดินจากซ้ายไปขวาตามผนัง	ให้ผู้เล่นอนุมานจากเนื้อเรื่อง ไม่ใช่เดาเหรียญบนโต๊ะมีเข็มชี้ไปทางผนังด้านที่ต้องเริ่มนับ
2 ห้องช่าง	พบนาฬิกาพกหลายเรือน คู่มือบอกว่ามาตรฐานการจับเวลาคือนาฬิกาทราย 10 วินาที จึงไล่จับเวลาทุกเรือนจนพบเรือนที่เดินช้ากว่า แล้วนำเวลาบนหน้าปัดของเรือนนั้นไปตั้งที่กลไกบนโต๊ะ	ให้ผู้เล่นค้นพบคำตอบด้วยการทดลอง ธีมเวลาที่ช้าลงเป็นนัยของเรื่องว่าช่างอาจทดลองเกี่ยวกับเวลา

3 ห้องนั่งเล่น	ดึงโซ่ใต้นาฬิกานกกุ๊กกูสามเรือน นับว่านกโผล่พร้อมเสียงกี่ครั้ง เรือนกลางไม่มีนกและไม่มีเสียงจึงนับเป็น 0	ตั้งใจให้ผู้เล่นสับสน เพราะผู้เล่นผ่านมาสองห้องที่ต้องหาคำใบ้รอบตัว จึงมักไปหาคำใบ้ต่อ ทั้งที่คำใบ้อยู่ที่ประตูกรงนก ซึ่งมีล้อรหัสสามล้อ ข้อคิดที่ต้องการสื่อคือบางครั้งคำตอบที่ง่ายที่สุดอยู่ที่สิ่งที่ต้องไปตอบ
4 ห้องนอน	ภาพถ่ายบนโต๊ะแสดงของวางเรียงตามลำดับ ของจริงกระจายอยู่ทั่วห้อง ผู้เล่นหยิบมาพลิกดูตัวเลขที่ด้านหลัง แล้วเรียงตามลำดับในภาพ	เป็นปริศนาเรียบง่าย จุดประสงค์คือโชว์การประกอบแบบอื่น ผู้เล่นต้องนำตุ้มถ่วงและลูกตุ้มไปใส่ให้ครบ นาฬิกาตั้งพื้นจึงจะเดินได้
5 หลังตู้นาฬิกา	ใส่ปริซึมสามชิ้นที่เก็บมาตั้งแต่ห้องแรก แล้วหมุนปริซึมจนจุดแสงที่ฉายตรงกับจุดบนจอ จึงออกจากบ้านได้	เป็นฉากปิดท้ายของเดโม เปิดทางให้ต่อเนื้อเรื่องภายหลัง ปริซึมแต่ละชิ้นเปลี่ยนคุณสมบัติของแสงเพียงอย่างเดียว ผู้เล่นจึงสังเกตแล้วแก้ทีละชิ้นได้

ปริซึมทั้งสามชิ้นถูกซ่อนไว้ในสามห้องแรกที่ผู้เล่นเดินผ่าน คือปริซึมหกหน้าในลิ้นชักของตู้โชว์ห้องหน้าร้านซึ่งไม่ล็อก ปริซึมสี่หน้าในลิ้นชักโต๊ะช่าง และปริซึมสามหน้าในตู้เสื้อผ้าห้องนอน ผู้เล่นจึงเก็บชิ้นส่วนไว้ตลอดทางโดยยังไม่รู้ว่าใช้ทำอะไร ทำให้ปริศนาสุดท้ายเชื่อมกับทั้งเกม

เมื่อพิจารณาปริศนาทั้งห้าองก์ ผู้เล่นต้องใช้วิธีคิดต่างกันในแต่ละห้อง คือ อนุมานจากเนื้อเรื่อง ทดลองเปรียบเทียบ ตีความจากภาพย้อนกลับไปหาของจริง ค้นหาและประกอบชิ้นส่วน และจัดตำแหน่งของแสง นาฬิกาเป็นแกนของปริศนาโดยตรงในองก์ที่ 1 ถึง 3 ส่วนองก์ที่ 4 นาฬิกาตั้งพื้นเป็นกลไกที่เปิดทางออกเมื่อใส่ชิ้นส่วนครบ และองก์ที่ 5 ใช้นาฬิกาเป็นธีม เพราะกลุ่มดาวบนฉากคือกลุ่มดาวนาฬิกาลูกตุ้ม (Horologium) ผลการออกแบบนี้อธิบายต่อในบทที่ 4

3.3.2 ขั้นที่ 2 การออกแบบระบบเกม

เมื่อกำหนดเนื้อหาแล้ว นักศึกษาออกแบบระบบเกมโดยยึดหลักการออกแบบซอฟต์แวร์แบบมอดูลาร์ตามบทที่ 2 มีการตัดสินใจหลักห้าข้อ

1. **แบ่งโค้ดเป็นโมดูลตามโดเมนของระบบ ไม่ใช่ตามชนิดของวัตถุ** ได้แก่ Items Interaction Clock Puzzle World Player UI และ Game โดยกำหนดทิศ

ทางการพึ่งพาให้ไหลทางเดียวจากโมดูลล่างขึ้นบน คือ Items ไม่พึ่งโมดูลใด Interaction พึ่ง Items Clock และ Puzzle พึ่ง Interaction ส่วน Player UI และ Game อยู่ชั้นบนสุด

2. **วัตถุแต่ละชิ้นเกิดจากการประกอบคอมโพเนนต์ ไม่สร้างคลาสแยกต่อวัตถุ** ระบบสิ่งที่เปิดได้แยกเป็น Openable ที่เก็บสถานะเปิด–ปิด SwingMotion และ SlideMotion ที่ทำให้เคลื่อนที่ และกลอนชนิดต่าง ๆ ที่ผูกกับ Openable ด้วยส่วนต่อประสาน ILock ประตู ลิ้นชัก ฝาตู้ และผนังลับจึงใช้ Openable ตัวเดียวกัน
3. **ระบบนาฬิกาออกแบบเป็นชิ้นส่วน** ดังตารางที่ 3.7 นาฬิกาทุกเรือนมีหน้าปัดและกลไกเหมือนกัน และต่างกันที่ชิ้นส่วนที่ประกอบเพิ่ม ส่วนที่แสดงเวลา ที่เก็บกำลัง และเงื่อนไขของนาฬิกา แยกออกจากกันโดยกำหนดผ่านส่วนต่อประสาน IClockDisplay IClockPart และ IPowerStore

ตารางที่ 3.7 ชิ้นส่วนหลักของระบบนาฬิกาและหน้าที่

ชิ้นส่วน	หน้าที่
ClockMovement	เก็บเวลาของนาฬิกา และส่งเวลาให้ส่วนแสดงผลทุกชิ้นที่ติดอยู่ ก่อนเดินจะถามชิ้นส่วนทุกชิ้นว่าอนุญาตให้เดินหรือไม่
ClockDial	หน้าปัดแบบเข็ม เปลี่ยนเวลาเป็นมุมของเข็ม
DigitalReadout	ส่วนแสดงเวลาแบบตัวเลขที่เลื่อนแถบ เป็นส่วนแสดงผลชนิดที่สอง
ClockCondition	สภาพของนาฬิกา มีห้าแบบ คือ Running Slow Reversed Stopped และ Broken
Mainspring	ลานสปริง เป็นที่เก็บกำลังที่ไขลานได้ นาฬิกาที่ไม่มีลานสปริงไม่มีวันหมดกำลัง
Pendulum	ลูกตุ้ม ถ้าถูกหยุดจะทำให้นาฬิกาหยุด
ClockWinder และ WeightChain	ก้านไขลานและโซ่ตุ้มถ่วง เป็นสองวิธีใส่กำลังให้ลานสปริง
ClockStriker	เปิดและปิดประตูนกตามจำนวนชั่วโมงบนหน้าปัด

TestRun	ชิ้นส่วนที่ทำให้นาฬิกาเริ่มเดินเมื่อเริ่มการทดสอบ ใช้ในปริศนาองก์ที่ 2
ClockPartSocket	ช่องใส่ชิ้นส่วนที่ผู้เล่นเก็บมา ใช้ในองก์ที่ 4 และ 5

4. **การรับคำสั่งและการสลับโหมดผู้เล่นรวมอยู่ที่จุดเดียว** InputReader เป็นเจ้าของชุดคำสั่ง PlayerControls เพียงตัวเดียวของทั้งเกม และ PlayerModeController เป็นส่วนเดียวที่สลับโหมดของผู้เล่น คือสำรวจ ดูวัตถุ และเปิดกระเป๋า ปุ่มที่กดเป็นครั้ง ๆ เช่น E F และ Q ใช้การสมัครรับเหตุการณ์ ส่วนค่าต่อเนื่อง เช่น การเดินและการหันกล้อง ใช้การอ่านทุกเฟรม
5. **สร้างฉากและทรัพยากรจากโปรแกรม ไม่ปั้นด้วยโปรแกรมสามมิติ** เครื่องมือสำหรับตัวแก้ไขอ่านผังบ้านจากไฟล์ข้อมูลแล้วสร้างห้อง ผนัง เฟอร์นิเจอร์ นาฬิกา และวัตถุปริศนา การย้ายหรือขยายห้องจึงเป็นการแก้ตัวเลขในไฟล์ข้อมูล

3.3.3 ขั้นที่ 3 การพัฒนาโปรแกรม

1. เตรียมโปรเจกต์และระบบควบคุมผู้เล่น สร้างโปรเจกต์แม่แบบ Universal 3D ของ Unity รุ่น 6000.3.5f2 นำเข้าชุดโมเดลตัวละคร สร้างวัสดุชุดแรก 22 ไฟล์ และสร้างไฟล์กำหนดคำสั่ง PlayerControls.inputactions ของตนเองแทนไฟล์ที่มากับแม่แบบ วันถัดมาเริ่มเขียนซอร์สโค้ดของเกม ไฟล์แรก ๆ คือ InputReader.cs MouseLook.cs PlayerMovement.cs และ RoomLighting.cs ส่วนไฟล์ ClockCondition.cs ซึ่งกำหนดสภาพของนาฬิกาห้าแบบ

2. ระบบโต้ตอบกับวัตถุและโครงของระบบนาฬิกา สร้างกลไกพื้นฐานที่ปริศนาทุกองก์ใช้ร่วมกัน โดยแยกหน้าที่เป็นชิ้นเล็ก ๆ ชิ้นละหนึ่งหน้าที่ ได้แก่ SwingMotion และ SlideMotion สำหรับการเคลื่อนที่ของสิ่งที่เปิดได้ Inventory สำหรับของที่ผู้เล่นถืออยู่ BorrowedProp สำหรับจำสภาพเดิมของวัตถุที่ยกขึ้นดูเพื่อคืนสภาพเมื่อวางกลับ Inspectable และ InspectStyle สำหรับรูปแบบการดูวัตถุสามแบบ และ IClockPart ซึ่งเป็นส่วนต่อประสานตัวแรกของระบบนาฬิกา ตู้โชว์ของปริศนาองก์ที่ 1 เป็นวัตถุปริศนาชิ้นแรกที่สร้าง

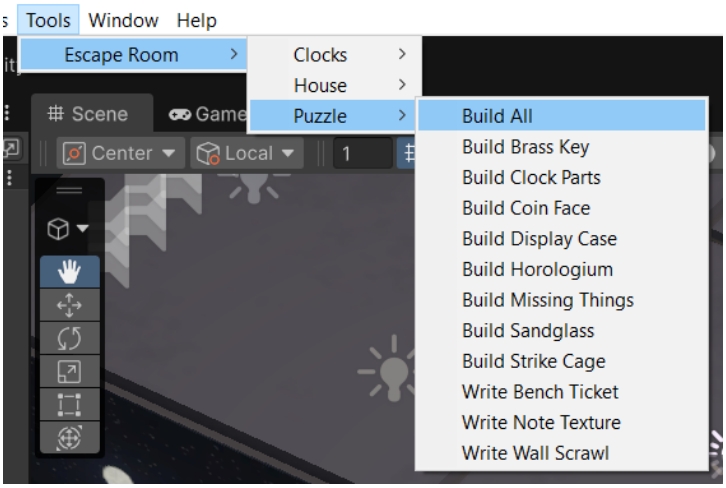
3. ปรับโครงสร้างและบังคับขอบเขตโมดูล แบ่งเป็นสามงาน งานแรกคือแยกส่วนต่อประสานของระบบนาฬิกาออกมา ได้แก่ IClockDisplay IPowerStore และ ICodeInput แล้วเขียน DigitalReadout เป็นผู้ทำตาม IClockDisplay รายที่สอง เพื่อพิสูจน์ว่าส่วนต่อประสานที่แยกออกมารองรับการแสดงเวลาที่ต่างรูปแบบกันโดยสิ้นเชิงได้ คือหน้าปัดแบบ

เข็มกับตัวเลขที่เลื่อนแถบ โดย ClockMovement ไม่ต้องรู้ว่าปลายทางเป็นแบบใด งานที่สองคือตัดการพึ่งพาที่คลาสของปริศนามีต่อคลาสของผู้เล่น โดยสร้างส่วนต่อประสาน IActor ที่มีเพียงสามเมธอด คือ Say สำหรับพูดข้อความ OfferBag สำหรับเปิดกระเป๋าให้เลือกของ และ Take สำหรับรับของเข้ากระเป๋า แล้วแก้ไฟล์ 10 ไฟล์ที่เดิมรับพารามิเตอร์เป็นคลาส PlayerInteractor ให้รับ IActor แทน ดังตารางที่ 3.8 ผลคือโมดูล Clock และ Puzzle ไม่อ้างถึงโมดูล Player อีกเลย งานที่สามคือบังคับขอบเขตของโมดูลด้วยไฟล์ Assembly Definition จำนวน 5 ไฟล์ คือ EscapeRoom.Items EscapeRoom.Interaction EscapeRoom.Clock EscapeRoom.Puzzle และ EscapeRoom.World พร้อมไฟล์ของชุดทดสอบอีก 1 ไฟล์ ทำให้ขอบเขตระหว่างโมดูลกลายเป็นข้อบังคับที่คอมไพเลอร์ตรวจ จากนั้นสร้างนาฬิกาเรือนแรกจากไฟล์ข้อมูลล้วน (PlankClock) โดยใช้ ClockRecipe และเครื่องมือ ClockAssembler ซึ่งไม่มีเมธอดในภาษา C# รองรับนาฬิกาเรือนนี้โดยเฉพาะ

ตารางที่ 3.8 ไฟล์ที่เปลี่ยนจากรับ PlayerInteractor เป็นรับ IActor

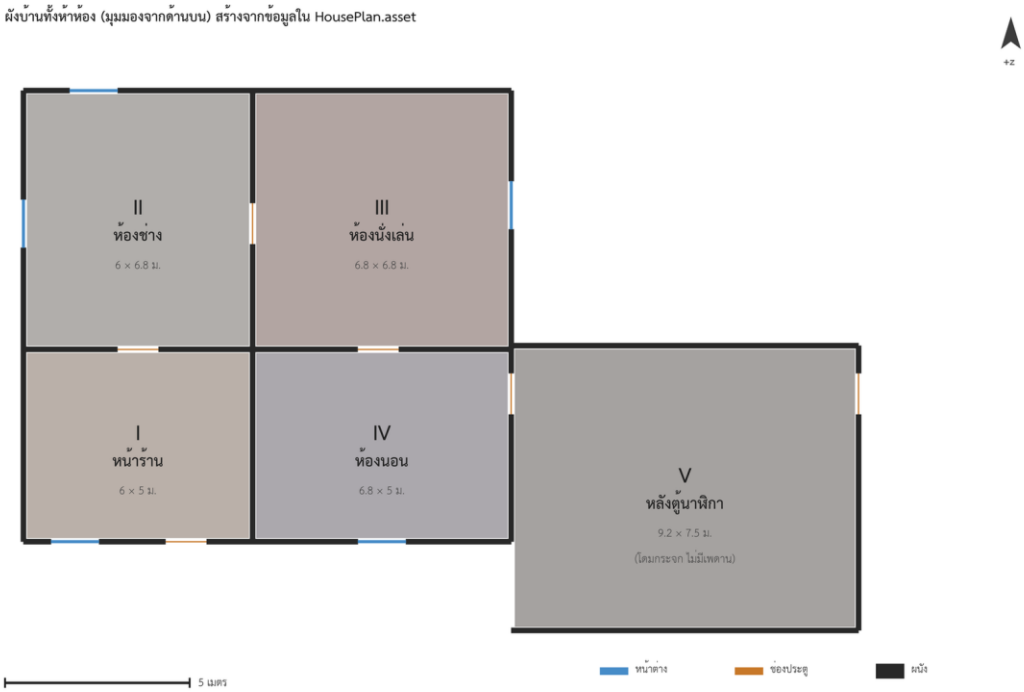
โมดูล	ไฟล์
Clock	ClockPartSocket.cs, ClockSetter.cs, ClockWinder.cs, RunningLock.cs, TimeLock.cs, WeightChain.cs
Interaction	KeyLock.cs, Openable.cs, PickupItem.cs
Puzzle	CodeWheel.cs

4. สร้างโลกของเกมด้วยโปรแกรม สร้างเครื่องมือสำหรับตัวแก้ไข 3 ตัวหลัก คือ TextureKit วาดภาพพื้นผิวทุกชนิดลงไฟล์ HousePlan เก็บผังบ้านเป็นไฟล์ข้อมูล คือ รายการห้องห้าห้องกับแนวผนังและตำแหน่งช่องประตู และ HouseBuilder อ่านผังแล้วสร้างพื้น ผนัง ฝ้า หน้าต่าง และคิ้วบัวทั้งหมด เครื่องมือแต่ละตัวแตะเฉพาะส่วนที่เป็นของตนเอง HouseBuilder สร้างใหม่เฉพาะกลุ่ม House/Shell และ DressingBuilder สร้างใหม่เฉพาะกลุ่ม Dressing ของแต่ละห้อง จึงสั่งสร้างฉากใหม่ได้ทุกเมื่อโดยไม่ทำลายงานที่วางด้วยมือ ส่วนวัตถุที่คนเป็นผู้เขียนเนื้อหา เช่น นาฬิกาทรายและหนังสือพิมพ์ ถูกวางไว้นอกขอบเขตของเครื่องมือ หรือเครื่องมืออ่านข้อความที่เขียนไว้ออกมาเก็บก่อนแล้วใส่กลับหลังสร้างใหม่ ภาพที่ 3.3 แสดงเมนูเครื่องมือเหล่านี้ และภาพที่ 3.4 แสดงผังบ้านที่ได้



ภาพที่ 3.3 เมนูเครื่องมือสร้างเนื้อหาที่พัฒนาขึ้นในโครงงาน

จากภาพที่ 3.3 เครื่องมือถูกจัดเป็นหมวด House Clocks และ Puzzle ใต้เมนู Tools > Escape Room แต่ละรายการสร้างหรือสร้างใหม่เฉพาะส่วนของตนเอง



ภาพที่ 3.4 ผังบ้านทั้งห้าห้องที่สร้างจากไฟล์ HousePlan.asset

จากภาพที่ 3.4 ห้องทั้งห้าเรียงต่อกันเป็นเส้นทางเดียวตามผังในไฟล์ข้อมูล ห้องที่ห้าเป็นห้องใหญ่ที่สุดเพราะมีเครื่องวัดทรงกลมดาราศาสตร์อยู่กลางห้อง

5. พัฒนาปริศนาห้าองก์ เมื่อกลไกพื้นฐานพร้อมแล้ว ปริศนาแต่ละองก์จึงมีคลาสเฉพาะของตนเองเพียงไม่กี่คลาส ดังตารางที่ 3.9

ตารางที่ 3.9 คลาสเฉพาะของปริศนาแต่ละองก์

องก์	คลาสเฉพาะขององก์
1	CompassNeedle และส่วนที่อยู่ในเครื่องมือ PuzzleBuilder
2	Sandglass HeldSandglass และ RunTicking
3	CuckooBird ClockStriker และต้นแบบ CuckooClock
4	StampedNumber StampedCombination และเครื่องมือ MissingThingsBuilder
5	StarLock Horologium Prism และต้นแบบ Sundial StarScreen StarChart MoonDome

ปริศนาทั้งห้าองก์ใช้กลไกชุดเดียวกันโดยไม่ต้องแก้โค้ดของกลไกนั้น ตัวอย่างที่ชัดที่สุดมี สามกรณี กรณีแรก ClockPartSocket ถูกใช้ 5 จุด คือสองจุดในองก์ที่ 4 สำหรับใส่ลูกตุ้ม และตุ้มถ่วงของนาฬิกาตั้งพื้น และสามจุดในองก์ที่ 5 สำหรับใส่ปริซึมทั้งสามชิ้น โดยไม่ต้องแก้คลาสนี้เพื่อรองรับปริซึม เพราะรับค่าเป็น ItemData ซึ่งเป็นไฟล์ข้อมูล ไม่ใช่ชนิดของวัตถุ กรณีที่สอง CodeLock และ CodeWheel ถูกใช้ใน 3 องก์ คือตู้โชว์ในองก์ที่ 1 ฐานของกรงนกในองก์ที่ 3 และประตูตู้เสื้อผ้าในองก์ที่ 4 กรณีที่สาม กลอนทุกชนิดเป็นผู้ทำตามส่วนต่อประสาน ILock เดียวกัน Openable จึงถามกลอนทุกตัวที่ติดอยู่ว่าล็อกอยู่หรือไม่ โดยไม่ต้องรู้ว่ากลอนนั้นต้องการกุญแจ รหัส เวลา หรือรูปแบบของดวงดาว

ในองก์ที่ 4 คำตอบของปริศนาเป็นข้อมูลที่แก้ได้ ไม่ใช่ตัวเลขในโค้ด คือ StampedNumber เก็บเลขประจำวัตถุหนึ่งชิ้นและพิมพ์ลงบนป้ายด้านหลังวัตถุ ส่วน StampedCombination ถือรายการวัตถุเหล่านั้นตามลำดับที่ปรากฏในภาพถ่าย แล้วส่งรหัสที่ได้ให้กลอนรหัส การเปลี่ยนคำตอบหรือเพิ่มความยาวของรหัสจากสี่หลักเป็นห้าหลักจึงทำได้ที่ตัวตรวจสอบของ Unity โดยเพิ่มสมาชิกในรายการและเพิ่มวงล้ออีกหนึ่งวง ไม่ต้องแก้ซอร์สโค้ด

6. ปรับโครงสร้างครั้งที่สอง หลังพัฒนาปริศนา นักศึกษารวมส่วนต่อประสานของการโต้ตอบไว้ในไฟล์เดียว (Interactions.cs) และปรับระบบการยกวัตถุขึ้นดู (InspectView.cs) ให้ทำงานผ่านส่วนต่อประสานเหล่านั้น พร้อมปรับระบบกระเป๋าให้ซ่อนของที่ใช้กับสิ่งอื่นไม่ได้ เช่น นาฬิกาทราย แล้วเขียนชุดทดสอบเพิ่มตามที่อธิบายในหัวข้อ 3.3.4

7. แก้ไขความถูกต้องของการนับเวลา งานชิ้นแรกของวันสุดท้ายคือแก้ปัญหาการนับเวลาของนาฬิกาและนาฬิกาทรายให้เท่ากันทุกเครื่อง เพราะคำตอบของปริศนาองก์ที่ 2 คือการเปรียบเทียบว่านาฬิกาเรือนใดเดินไม่ทันนาฬิกาทรายสิบวินาที วิธีแก้ประกอบด้วยสี่กลไก คือ นับเวลาด้วย Time.deltaTime ซึ่งเป็นเวลาจริงของเฟรมแทนจำนวนเฟรม สะสมเวลาในชนิดข้อมูล double แทน float คิดเฟรมสุดท้ายของการทดสอบเป็นเศษส่วนของเฟรม และให้ตัวจับเวลานับใน LateUpdate ส่วนนาฬิกาเดินใน Update กลไกทั้งสี่อยู่บนทฤษฎีในหัวข้อ 2.3 ซึ่งว่าด้วยความเป็นอิสระจากอัตราเฟรม ความแม่นยำของเลขทศนิยม และลำดับการทำงานของฟังก์ชันเหตุการณ์ รายละเอียดของปัญหาและวิธีแก้อยู่ในบทที่ 5 หัวข้อ 5.2.1

3.3.4 ขั้นที่ 4 การทดสอบการทำงานของเกม

การทดสอบมีสามแบบ

การทดสอบอัตโนมัติ นักศึกษาเขียนชุดทดสอบแบบ PlayMode ด้วย Unity Test Framework รวม 15 กรณีใน 3 ไฟล์ คือ ClockSeamTests 6 กรณี RecipeSeamTests 3 กรณี และ ActorSeamTests 6 กรณี ชุดทดสอบเขียนหลังการปรับโครงสร้างแต่ละครั้ง ไม่ได้เขียนก่อนตามแนวพัฒนาที่ขับเคลื่อนด้วยการทดสอบ (Test-Driven Development) เพราะมีจุดประสงค์เพื่อยืนยันว่ารอยต่อที่เพิ่งแยกออกมาใช้งานได้จริงและไม่ถูกทำพังในภายหลัง ตามแนวคิดการทดสอบความเหมาะสมของสถาปัตยกรรมในหัวข้อ 2.4.2 ผลการรันอยู่ในบทที่ 4

การทดสอบการเข้าถึงวัตถุ ตรวจว่าผู้เล่นเล็งถึงวัตถุที่ต้องโต้ตอบได้จริง และไม่เล็งทะลุผนังตู้ไปหยิบของข้างใน โดยจำลองการเล็งจากตำแหน่งที่ผู้เล่นยืนจริง ด้วยรัศมีการเล็งและชั้นของวัตถุที่บังเหมือนที่ผู้เล่นใช้ ไม่ตรวจด้วยสายตา วิธีนี้ใช้ปรับขนาดและตำแหน่งของกรงนก ตู้เสื้อผ้า และเครื่องวัดทรงกลมดาราศาสตร์

การทดลองเล่นกับผู้ใช้ นักศึกษาให้เพื่อนจำนวน 5 คนทดลองเล่นเกมจนจบ ในช่วงเดือนกันยายนถึงตุลาคม พ.ศ. 2569 หลังพัฒนาเกมครบทั้งห้าองก์ โดยให้เพื่อนแต่ละคนเล่นคนเดียวบนเครื่องของนักศึกษา ไม่บอกคำตอบหรือคำใบ้ใด ๆ ล่วงหน้า นักศึกษานั่งสังเกตการเล่น จดว่าผู้เล่นติดที่องก์ใดและทำอะไรก่อนติด และสอบถามความเห็นหลังเล่นจบ เพื่อสังเกตว่าผู้เล่นติดที่ใดและเพราะอะไร ผลการสังเกตอยู่ในบทที่ 4

บทที่ 4
ผลการดำเนินงาน

บทนี้รายงานผลที่ได้จากการดำเนินงานตามขั้นตอนในบทที่ 3 แบ่งเป็นสามส่วน คือ ผลการดำเนินงานซึ่งเป็นตัวเกมที่ส่งมอบ ผลการวิเคราะห์ข้อมูลซึ่งรวมผลการวัดโครงสร้างของซอฟต์แวร์และผลการทดสอบ และการวิจารณ์ผลโดยเปรียบเทียบกับวัตถุประสงค์ที่ตั้งไว้

ผลลัพธ์หลักของโครงงานคือเกมห้องหลบหนีสามมิติที่เล่นจบได้ ประกอบด้วยห้าห้องเรียงต่อกันเป็นเส้นทางเดียว แต่ละห้องมีปริศนาประจำห้อง และมีนาฬิกาในเกมรวม 46 เรือนที่สร้างจากหน้าปัดและกลไกพื้นฐานร่วมกัน ตัวเลขทุกตัวในบทนี้มาจากการวัดไฟล์และฉากในโปรเจกต์โดยตรง และระบุวิธีวัดกำกับไว้ เพื่อให้ตรวจซ้ำได้

4.1 ผลการดำเนินงาน

4.1.1 ผลลัพธ์ที่ส่งมอบ

ผลลัพธ์ของโครงงานคือเกม escape room มุมมองบุคคลที่หนึ่งที่เล่นจบได้หนึ่งรอบ ประกอบด้วยห้องห้าห้องเรียงต่อกันเป็นเส้นทางเดียว และปริศนาห้าองก์ที่ต้องแก้เรียงตามลำดับ ปริมาณของผลลัพธ์ที่วัดได้แสดงในตารางที่ 4.1

ตารางที่ 4.1 ปริมาณผลลัพธ์ของโครงงานที่วัดได้จากไฟล์ในโปรเจกต์

รายการ	จำนวน	วิธีวัด
ห้องที่เล่นได้	5	นับวัตถุลูกของ House/Props ในฉาก Main.unity
ปริศนาที่ต้องแก้	5 องก์	ตารางที่ 4.2
ประตูที่มีกลไก	5 บาน	นับวัตถุลูกของ House/Doors
ไฟล์โปรแกรมภาษา C#	78 ไฟล์	นับไฟล์ .cs ทั้งโปรเจกต์
จำนวนบรรทัดของโปรแกรม	10,604 บรรทัด	นับบรรทัดที่ไม่ว่าง · ตารางที่ 4.5
คลาสที่สืบทอดจาก MonoBehaviour	51 คลาส	ค้นข้อความ : MonoBehaviour
ส่วนต่อประสาน (interface) ที่ประกาศขึ้น	12 รายการ	ค้นข้อความ public interface

Prefab	33 ไฟล์	นับไฟล์ใน Assets/Prefabs/
Material	115 ไฟล์	นับไฟล์ .mat
ScriptableObject	12 ไฟล์	ItemData 10, HousePlan 1, ClockRecipe 1
นาฬิกาในฉาก	46 เรือน	นับอินสแตนซ์ของส่วนประกอบ ClockMovement ในฉาก (นับซ้ำเมื่อ 10 พ.ย. 2569)
ชุดทดสอบอัตโนมัติ	15 กรณี	นับ attribute [UnityTest]

ห้องทั้งห้าห้องตามลำดับที่ผู้เล่นเดินผ่านแสดงในภาพที่ 4.1 ถึง 4.5



ภาพที่ 4.1 ห้องหน้าร้าน องก์ที่หนึ่ง

จากภาพที่ 4.1 ห้องหน้าร้านเป็นห้องแรกที่ผู้เล่นพบ มีนาฬิกาแขวนผนังเจ็ดเรือน ตู้โชว์ที่ใส่กุญแจไว้ข้างใน และโต๊ะที่วางเหรียญกับหนังสือพิมพ์



ภาพที่ 4.2 ห้องช่าง องก์ที่สอง

จากภาพที่ 4.2 ห้องช่างมีนาฬิกาพกแขวนผนังสิบสองเรือน โต๊ะช่างที่หนีบกลไกนาฬิกาไว้ในปากกาจับ และนาฬิกาทรายที่วางบนผ้า



ภาพที่ 4.3 ห้องนั่งเล่น องก์ที่สาม

จากภาพที่ 4.3 ห้องนั่งเล่นมีนาฬิกานกกุ๊กกูสามเรือนเรียงกัน และโต๊ะที่วางกรงนกซึ่งมีล้อรหัสสามล้อที่ฐาน



ภาพที่ 4.4 ห้องนอน องก์ที่สี่

จากภาพที่ 4.4 ห้องนอนมีโต๊ะที่วางภาพถ่ายเก่า ตู้เสื้อผ้าที่ล็อกด้วยรหัสสี่หลัก และนาฬิกาตั้งพื้นที่ยังไม่เดิน



ภาพที่ 4.5 ห้องหลังตู้นาฬิกา องก์ที่ห้า

จากภาพที่ 4.5 ห้องหลังตู้นาฬิกาเป็นห้องใหญ่ที่สุดและอยู่ใต้โดมกระจก มีเครื่องวัดทรงกลมดาราศาสตร์อยู่กลางห้อง และจอที่จะฉายจุดแสงอยู่ที่ผนังด้านใต้

4.1.2 ผลการพัฒนาปริศนาทั้งห้าองก์

ปริศนาทั้งห้าองก์ทำงานได้ และคำตอบของแต่ละองก์ถูกยืนยันโดยอ่านค่าที่ตั้งไว้จากไฟล์ asset โดยตรง ผลแสดงในตารางที่ 4.2

ตารางที่ 4.2 คำตอบของปริศนาแต่ละองก์และหลักฐานที่ยืนยัน

องก์	ห้อง	กลไกที่ผู้เล่นต้องทำ	คำตอบ	แหล่งที่อ่านค่าได้
1	หน้าร้าน	อ่านเลขจากนาฬิกาที่ลูกตุ้มยังแกว่ง เรียงจากซ้ายไปขวา	**1479**	Scenes/Main.unity ⟶ ค่าที่ override ไว้บน CodeLock ของ DisplayCase (ดูหลังตาราง)
2	ห้องช่าง	จับเวลานาฬิกาพกเทียบนาฬิกาทราย หาเรือนที่เดินช้า แล้วตั้งเวลาตามหน้าปัดของเรือนนั้น	**4:40**	Scenes/Main.unity ⟶ TimeLock บน BenchDrawer (hour: 4, minute: 40)
3	ห้องนั่งเล่น	ดึงโซ่ใต้นาฬิกานกสามเรือนแล้วนับเสียงนก	**502**	Scenes/Main.unity ⟶ CodeLock ของกรงนก
4	ห้องนอน	หาของสี่ชิ้นในภาพถ่าย พลิกอ่านเลขหลังของ แล้วใส่ตามลำดับในภาพ	**4692**	Scenes/Main.unity ⟶ CodeLock ของตู้เสื้อผ้า
5	หลังตู้นาฬิกา	หมุนปริซึมสามชิ้นจนจุดแสงหกจุดตรงกับเครื่องหมายบนจอ	**2 · 3 · 4**	Prefabs/Puzzle/Sundial.prefab ⟶ Horologium.answer

รหัสขององก์ที่หนึ่งมีข้อสังเกตเรื่องที่เก็บค่า คือไฟล์ต้นแบบ Prefabs/Puzzle/DisplayCase.prefab บันทึกค่าไว้เป็น 4, 7, 1, 9 แต่ฉาก Main.unity เขียนทับสามตำแหน่งแรกเป็น 1, 4, 7 ส่วนตำแหน่งที่สี่คงค่า 9 ไว้ ค่าที่เกมใช้จริงจึงเป็น 1479 ซึ่งตรงกับค่า

เริ่มต้นที่เขียนไว้ใน CodeLock.cs ดังนั้นการตรวจรหัสของปริศนาต้องอ่านค่าที่ประกอบเสร็จแล้ว หลังรวมต้นแบบกับค่าที่ฉากเขียนทับ ไม่ใช่อ่านจากไฟล์ต้นแบบอย่างเดียว

คำตอบขององค์ที่สี่ยืนยันได้จากสองทางที่เป็นอิสระจากกัน และทั้งสองทางให้ผลตรงกัน ซึ่งเป็นหลักฐานที่หนักกว่าการอ่านค่าเดียว

ทางที่อ่าน	ค่าที่ได้
ค่าที่บันทึกไว้บน CodeLock ของตู้เสื้อผ้า	4, 6, 9, 2
StampedNumber.digit ของทั้งสี่ชิ้น เรียงตามลำดับในรายการ StampedCombination.asPictured คือ Teacup 4 · Candlestick 6 · Vase 9 · HandMirror 2	4, 6, 9, 2

ทางที่สองมีความหมายเพิ่มขึ้นอีกชั้น เพราะมันคือ **เลขที่พิมพ์อยู่บนป้ายหลังของที่ผู้เล่นพลิกอ่านจริง** เรียงตามลำดับที่ของวางอยู่ในภาพถ่าย การที่ค่าทั้งสองทางตรงกันหมายความว่าคำใบ้ที่ผู้เล่นเห็นกับรหัสที่กลอนต้องการเป็นสิ่งเดียวกัน ไม่ได้ตั้งค่าไว้แยกกันสองที่ ซึ่งเป็นผลของการที่ StampedCombination ประกอบรหัสขึ้นจาก StampedNumber แต่ละชิ้นแทนที่จะเก็บรหัสเป็นตัวเลขของตัวเอง

ผลที่สำคัญกว่าตัวคำตอบคือ **ปริศนาสามองก์จากห้าองก์ใช้โปรแกรมชุดเดียวกันทั้งหมด** ต่างกันเพียงค่าที่ตั้งใน Inspector และจำนวนวงล้อ ดังแสดงในตารางที่ 4.3

ตารางที่ 4.3 กลไกที่แต่ละองก์ใช้ร่วมกันและส่วนที่เขียนขึ้นเฉพาะ

กลไก	องก์ที่ใช้	คลาสที่ใช้ร่วมกัน	คลาสที่เขียนขึ้นเฉพาะองก์
รหัสตัวเลข	1, 3, 4	CodeLock + CodeWheel + Openable	องก์ 4 เพิ่ม StampedNumber, StampedCombination
ตั้งเวลาให้ตรง	2	TimeLock + ClockSetter + ClockMovement	TestRun, RunTicking, Sandglass, HeldSandglass
ใช้สิ่งของ (กุญแจ)	ทุกองก์	KeyLock + IItemReceiver + Inventory + PickupItem	—

ประกอบชิ้นส่วนเข้าช่อง	4, 5	ClockPartSocket	—
สังเกตและจัดตำแหน่ง	5	Openable + StarLock	Horologium, Prism
เปิดเองเมื่อเงื่อนไขครบ	4	Openable + SealedLock	ChimeRelease

– *หลักฐาน:* CodeLock 3 instance, CodeWheel 11 instance (4 + 3 + 4) ในฉาก · ไม่พบคลาสที่ตั้งชื่อผูกกับองก์ใดองก์หนึ่ง เช่น Puzzle1 หรือ HallPuzzle

– **ผลเชิงวิศวกรรม:** ปริศนาที่ผู้เล่นรับรู้ว่าต่างกันสามแบบ (อ่านเลขจากนาฬิกา นับเสียงนกเรียงตามภาพถ่าย) ใช้ implementation เดียวกันทั้งหมด ความต่างอยู่ที่ *คำใบ้* ไม่ใช่ที่ *กลอน* ซึ่งเป็นผลโดยตรงของการแยกหน้าที่ระหว่าง Openable กับ ILock



ภาพที่ 4.6 ผนังนาฬิกาเจ็ดเรือนขององก์ที่หนึ่ง ให้เห็นลูกตุ้มที่แกว่งและที่หยุด

จากภาพที่ 4.6 นาฬิกาที่ยังเดินมีลูกตุ้มแกว่งอยู่ ส่วนเรือนที่เสียลูกตุ้มหยุดนิ่ง ผู้เล่นใช้ความต่างนี้แยกเรือนที่ต้องอ่านเลขออกจากเรือนที่ไม่ต้องอ่าน



ภาพที่ 4.7 การยกนาฬิกาขึ้นดูและแป้นเลขที่ปรากฏเมื่อไขลาน

จากภาพที่ 4.7 ผู้เล่นยกนาฬิกาขึ้นดูด้วยปุ่ม F พลิกด้วยปุ่ม E แล้วไขลานที่ด้านหลัง แป้นเลขจะแสดงตัวเลขของเรือนนั้น ซึ่งนาฬิกาที่เสียก็แสดงตัวเลขเช่นกัน



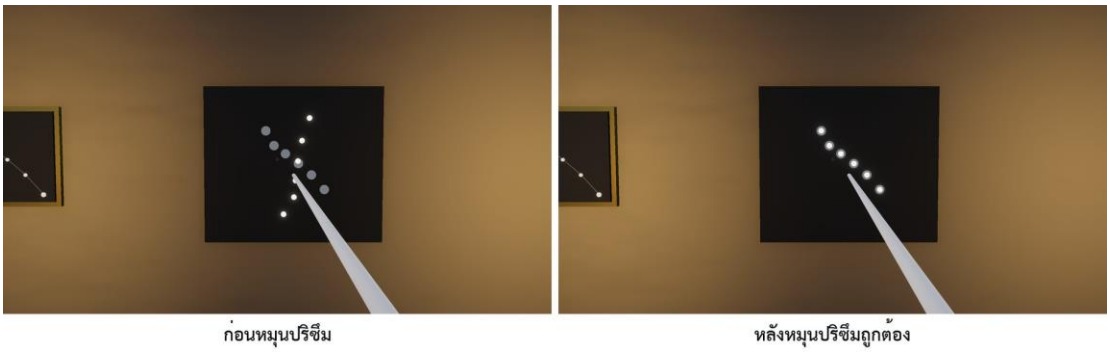
ภาพที่ 4.8 นาฬิกาพกและนาฬิกาทรายที่ชูขึ้นข้างกันขณะจับเวลาในองก์ที่สอง

จากภาพที่ 4.8 ผู้เล่นชูนาฬิกาพกคู่กับนาฬิกาทราย แล้วกด E เพื่อเริ่มทั้งสองพร้อมกัน เมื่อทรายหมดสิบวินาที เข็มวินาทีของเรือนที่เดินปกติควรอยู่ที่ตำแหน่งเลข 2



ภาพที่ 4.9 ภาพถ่ายเก่าและป้ายเลขหลังของในองก์ที่สี่

จากภาพที่ 4.9 ภาพถ่ายแสดงของสี่ชิ้นเรียงจากซ้ายไปขวา และป้ายราคาที่ด้านหลังของแต่ละชิ้นมีตัวเลขหนึ่งหลัก ซึ่งผู้เล่นต้องเรียงตามลำดับในภาพ ไม่ใช่ตามลำดับที่พบของในห้อง



ภาพที่ 4.10 จุดแสงหกจุดบนจอก่อนและหลังหมุนปริซึมถูกต้อง ในองก์ที่ห้า

จากภาพที่ 4.10 ก่อนหมุนปริซึม จุดแสงหกจุดกระจายผิดตำแหน่ง เมื่อหมุนปริซึมแต่ละชิ้นถูกต้อง จุดแสงทั้งหกจะตรงกับเครื่องหมายบนจอและค้างอยู่สองวินาที จึงถือว่าผ่าน

4.1.3 ผลการประกอบนาฬิกาทั้ง 46 เรือน

นาฬิกาในเกมมี 46 เรือน ทุกเรือนมีกลไก (ClockMovement) และหน้าปัด (ClockDial) ชุดเดียวกันจำนวน 46 อินสแตนซ์เท่ากัน ส่วนที่ทำให้นาฬิกาแต่ละเรือนมีพฤติกรรมต่างกันคือชิ้นส่วนที่ประกอบเพิ่มเข้าไป ตารางที่ 4.4 แสดงจำนวนชิ้นส่วนแต่ละชนิดที่พบในฉาก

ตารางที่ 4.4 ชิ้นส่วนที่ประกอบเป็นนาฬิกาทั้ง 46 เรือน

ชิ้นส่วน	หน้าที่	อินสแตนซ์ในฉาก	ตัวอย่างที่ใช้
ClockMovement	เก็บเวลาและส่งให้ส่วนแสดงผล	46	นาฬิกาทุกเรือน
ClockDial	หน้าปัดแบบเข็ม	46	นาฬิกาทุกเรือน
Mainspring	ลานสปริง ที่เก็บกำลัง	45	นาฬิกาเกือบทุกเรือน
ClockWinder	ก้านไขลานพร้อมแป้นเลข	12	นาฬิกาที่ต้องไขลาน
Pendulum	ลูกตุ้ม ถ้าหยุดนาฬิกาจะหยุด	11	นาฬิกาที่มีลูกตุ้ม
TestRun	ให้นาฬิกาเดินตามช่วงการทดสอบ	12	นาฬิกาพกในห้องช่างองก์ 2
WeightChain	โซ่ตุ้มถ่วง ใส่กำลังให้ลานสปริง	3	นาฬิกานกกุ๊กกูองก์ 3
ClockStriker	ตีบอกเวลาด้วยการเปิดประตูนก	3	นาฬิกานกกุ๊กกูองก์ 3
ClockSetter	ปุ่มตั้งเวลาชั่วโมงและนาที	2	กลไกบนโต๊ะช่างองก์ 2
DigitalReadout	ส่วนแสดงเวลาแบบตัวเลข	1	เครื่องจับเวลาบนโต๊ะช่างองก์ 2

วิธีนับ: นับอินสแตนซ์ของแต่ละส่วนประกอบในฉาก Main.unity เมื่อวันที่ 10 พฤศจิกายน 2569 รวมวัตถุที่ปิดการใช้งานอยู่ด้วย

จากตารางที่ 4.4 นาฬิกาที่ต่างกันเกิดจากการเลือกชิ้นส่วนมาประกอบ เช่น นาฬิกาพกต้องมี TestRun จึงเริ่มเดินเมื่อผู้เล่นเริ่มการทดสอบ นาฬิกานกกุ๊กกูมี WeightChain และ ClockStriker จึงรับกำลังจากโซ่และตีบอกเวลา ส่วนสภาพ เดินตรง เดินช้า หรือหยุดเดิน กำหนดด้วยค่า ClockCondition ที่ตั้งบนกลไก เรือนเดียวที่ใช้ส่วนแสดงเวลาแบบตัวเลขคือเครื่องจับเวลาบนโต๊ะช่าง ซึ่งกลไกเดียวกันขับทั้งหน้าปัดแบบเข็มและแบบตัวเลขพร้อมกัน นาฬิกาปลุก 12 เรือนในห้องนอนเป็นของตกแต่ง ตั้งค่าเหมือนกันทุกเรือนและไม่มีบทบาทในปริศนา

ข้อสังเกตที่ต้องระบุตามจริงคือ นาฬิกาเป็นแกนของปริศนาโดยตรงในองก์ที่ 1 ถึง 3 เท่านั้น องก์ที่ 4 นาฬิกาตั้งพื้นเป็นกลไกที่เปิดทางออกเมื่อใส่ลูกตุ้มและตุ้มถ่วงครบ ส่วนองก์ที่ 5 ใช้เครื่องวัดทรงกลมดาราศาสตร์กับปริซึม ซึ่งไม่ใช้ ClockMovement และเกี่ยวกับนาฬิกาทางธีมเท่านั้น

4.1.4 ผลของการสร้างเนื้อหาด้วยโปรแกรม

สัดส่วนของงานเขียนโปรแกรมที่วัดได้ให้ผลที่ไม่ตรงกับความคาดหมายทั่วไปของโครงงานเกม ดังตารางที่ 4.5

ตารางที่ 4.5 จำนวนบรรทัดของโปรแกรมแยกตามบทบาท

บทบาท	ไฟล์	บรรทัด	สัดส่วน	ทำงานในเกมหรือไม่
โปรแกรมส่วนที่ทำงานในเกม	63	3,481	32.8%	ทำงาน
โปรแกรมสร้างเนื้อหาสำหรับตัวแก้ไข	11	5,772	54.4%	**ไม่ทำงาน** ไม่ติดไปกับเกมที่ build
ชุดทดสอบ	3	393	3.7%	ทำงานเฉพาะเมื่อรันทดสอบ
ไฟล์ที่ Unity สร้างให้จากไฟล์ค่าอินพุต	1	958	9.0%	ทำงาน แต่ไม่ได้เขียนเอง
รวม	**78**	**10,604**	**100%**	

วิธีนับ: นับ **บรรทัดที่ไม่ว่าง** ของไฟล์ .cs ทุกไฟล์ใน Assets/ การนับทุกบรรทัดรวมบรรทัดว่างจะได้ 12,623 บรรทัด แยกเป็น 4,196 / 6,954 / 484 / 989 ตามลำดับเดียวกัน ตัวเลขในตารางนี้และทุกตัวเลขบรรทัดในรายงานใช้วิธีเดียวกันทั้งหมด คือไม่นับบรรทัดว่าง

- **ผลที่พบ:** โปรแกรมเกินครึ่งของโครงงาน (5,772 จาก 10,604 บรรทัด) เป็นเครื่องมือสร้างเนื้อหา ไม่ใช่ตัวเกม ไฟล์ที่ใหญ่ที่สุดในโครงงานคือ Assets/Scripts/Editor/PuzzleBuilder.cs 1,727 บรรทัด และ DressingBuilder.cs 966 บรรทัด ซึ่งทั้งสองไฟล์ไม่มีบรรทัดใดทำงานขณะผู้เล่นเล่น
- **ผลสืบเนื่องที่ยืนยันได้:** เรขาคณิตและพื้นผิวทุกชิ้นในเกมถูกสร้างจากโปรแกรม ไม่ได้ปั้นด้วยโปรแกรมสามมิติ — ตรวจแล้วไม่พบไฟล์ .blend, .obj หรือ .dae ในโปรเจกต์เลย ส่วนไฟล์ .fbx ที่พบหกไฟล์อยู่ในโฟลเดอร์ Assets/Character Models/ ทั้งหมดและมีวันที่ 20 มีนาคม 2562 ซึ่งเป็นชุดโมเดลที่นำเข้ามา ไม่ใช่งานของโครงงาน

– **ข้อแลกเปลี่ยนที่ตามมา:** การแก้เรขาคณิตด้วยมือใน Editor จะถูกเขียนทับเมื่อสั่งสร้างใหม่ครั้งถัดไป การแก้รูปทรงจึงต้องแก้ที่โปรแกรมสร้าง ไม่ใช่ที่วัตถุในฉาก ซึ่งเป็นข้อจำกัดของกระบวนการทำงาน ไม่ใช่ข้อบกพร่องของโปรแกรม

ผังบ้านทั้งห้าห้องที่สร้างจากโปรแกรมแสดงไว้แล้วในภาพที่ 3.4

4.2 ผลการวิเคราะห์ข้อมูล

หัวข้อนี้รายงานผลการวัดคุณสมบัติเชิงโครงสร้างของโปรแกรมที่เขียนขึ้นตามทฤษฎีการออกแบบซอฟต์แวร์แบบมอดูลาร์ในบทที่ 2 และผลการทดสอบ ผลทุกข้อเป็นการวัดจากไฟล์และระบุวิธีวัดกำกับไว้

4.2.1 ผลการแบ่งโมดูล

โปรแกรมถูกแบ่งเป็นเก้ากลุ่มตามหน้าที่ โดยหกกลุ่มมีขอบเขตบังคับระดับการคอมไพล์ด้วยไฟล์ Assembly Definition [24] และสามกลุ่มไม่มี ดังตารางที่ 4.6

ตารางที่ 4.6 โมดูลทั้งเก้ากลุ่มและขนาดของแต่ละกลุ่ม

โมดูล	โฟลเดอร์	Assembly	ไฟล์	บรรทัด	หน้าที่เดียวที่รับผิดชอบ
Items	Scripts/Items/	EscapeRoom.Items	2	37	นิยามว่าสิ่งของหนึ่งชิ้นคืออะไร และผู้เล่นถืออะไรอยู่
Interaction	Scripts/Interaction/	EscapeRoom.Interaction	11	373	สัญญากลางของการโต้ตอบและกลไกเปิด–ปิด–ล็อก
Clock	Scripts/Clock/	EscapeRoom.Clock	23	1,334	นาฬิกาทุกชนิด การเก็บเวลา การแสดงเวลา และกลไกที่ทำให้เดิน
Puzzle	Scripts/Puzzle/	EscapeRoom.Puzzle	8	384	ปริศนาเฉพาะเรื่อง คือรหัสตัวเลขและกลไกแสง

World	Scripts/World/	EscapeRoom.World	3	162	สภาพแวดล้อมที่ไม่เกี่ยวกับปริศนา และผังบ้านในรูปข้อมูล
Player	Scripts/Player/	*Assembly-CSharp*	10	830	ร่างผู้เล่น การอ่านอินพุต การเดิน การมอง การเล็ง การยกของขึ้นดู
UI	Scripts/UI/	*Assembly-CSharp*	3	216	การแสดงผลบนจอ
Game	Scripts/Game/	*Assembly-CSharp*	3	145	การเริ่มเกม การจบเกม และบรรยากาศแสง
Editor tooling	Scripts/Editor/	*Assembly-CSharp-Editor*	11	5,772	สร้างเรขาคณิต พื้นผิว และ prefab จากโปรแกรม
(ชุดทดสอบ)	Tests/PlayMode/	EscapeRoom.Tests	3	393	—

– **ผลที่พบ:** โครงงานนี้ **ไม่ใช้ namespace เลยทั้งโปรเจกต์** ทุกคลาสอยู่ใน global namespace และทุกไฟล์ .asmdef ตั้ง "rootNamespace": ""

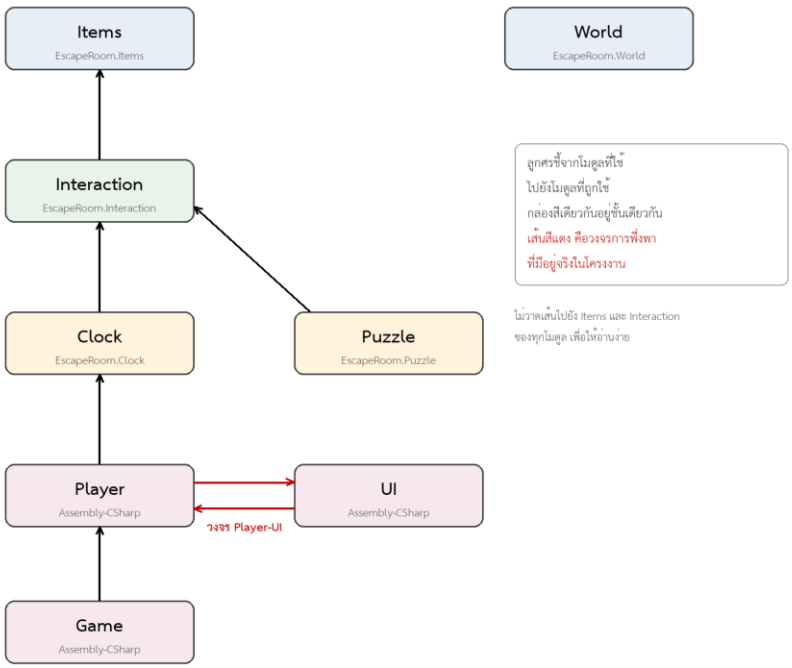
– **ผลเชิงวิศวกรรม:** การแบ่งโมดูลที่นี่ไม่ได้ใช้ namespace เป็นเครื่องมือ แต่ใช้ Assembly Definition แทน ซึ่งเข้มกว่าในแง่การบังคับ — namespace เพียงจัดกลุ่มชื่อและข้ามได้ด้วยคำสั่ง using ขณะที่ Assembly Definition **ปิดกั้นการอ้างถึงในระดับการคอมไพล์** [24] ข้อเสียที่ต้องยอมรับคือหากนำ asset จากภายนอกเข้ามาแล้วมีคลาสชื่อซ้ำโปรเจกต์จะคอมไพล์ไม่ผ่านทันที

4.2.2 ผลการวัดการพึ่งพาระหว่างโมดูล

วิธีวัด: สำหรับแต่ละโฟลเดอร์โมดูล ลบคอมเมนต์ออกจากทุกบรรทัดก่อน แล้วค้นหาชื่อชนิดข้อมูลทุกตัวที่ประกาศในโมดูลอื่น หากพบถือว่ามีการพึ่งพาจริงในโปรแกรม การตัดคอมเมนต์ออกก่อนเป็นขั้นตอนที่จำเป็น เพราะพบกรณีที่ชื่อชนิดข้อมูลของอีกโมดูลปรากฏในคอมเมนต์เท่านั้น

ตารางที่ 4.7 การพึ่งพาระหว่างโมดูลที่วัดได้จากโปรแกรม

จากโมดูล	ไปโมดูล	ตัวอย่างหลักฐาน
Items	*ไม่พึ่งโมดูลใด*	—
World	*ไม่พึ่งโมดูลใด*	—
Interaction	Items	IActor.cs:25, KeyLock.cs:11,26, PickupItem.cs:8
Clock	Items	ClockPartSocket.cs:13,43
Clock	Interaction	ChimeRelease.cs:20,35,41, ClockPartSocket.cs:10 และอีก 15 จุด
Puzzle	Interaction	CodeLock.cs:12,13,29,37 และอีก 7 จุด
Player	Interaction	InspectView.cs:57,61,72,83 และอีก 20 จุด
Player	Items	HeldSandglass.cs:13,14,27, PlayerInteractor.cs:12 และอีก 4 จุด
Player	UI	InspectView.cs:25, PlayerInteractor.cs:31,33,40
Player	Clock	HeldSandglass.cs:12
UI	Player	InventoryUI.cs:18,19,63,70
UI	Items	InventoryUI.cs:20,29,42,50 และอีก 3 จุด
UI	Interaction	InventoryUI.cs:31,56, PagePanel.cs:21
Game	Player	GameManager.cs:17,45, TitleScreen.cs:12,22
Game	Interaction	GameManager.cs:15



ภาพที่ 4.11 ทิศทางการพึ่งพาระหว่างโมดูลของโครงงาน

จากภาพที่ 4.11 การพึ่งพาไหลทางเดียวจากโมดูลชั้นบนลงสู่ชั้นล่าง คือ Game พึ่ง Player Player พึ่ง Clock Clock และ Puzzle พึ่ง Interaction และ Interaction พึ่ง Items มีเพียงวงจรระหว่าง Player กับ UI ที่ย้อนกลับ ส่วน World ไม่พึ่งและไม่ถูกโมดูลใดพึ่ง

ผลที่สำคัญสามข้อ

1. **โมดูล Clock และ Puzzle ไม่อ้างถึงโมดูล Player หรือ UI แม้แต่ครั้งเดียว** ยืนยันด้วยการค้นหาชื่อ PlayerInteractor, PlayerHud, PlayerModeController, InspectView, InputReader และ InventoryUI ในโฟลเดอร์ Clock/ และ Puzzle/ ได้ผลเป็นศูนย์

นี่คือผลที่วัดได้ของการสร้างส่วนต่อประสาน IActor ในขั้นการพัฒนาโปรแกรม — นาฬิกาและปริศนาถูกสั่งงานผ่านสัญญาสามเมธอด (Say, OfferBag, Take) ไม่ใช่ผ่านคลาสของผู้เล่น ผลคือโมดูลทั้งสองคอมไพล์และทดสอบได้โดยไม่ต้องมีผู้เล่นอยู่เลย ซึ่งเป็นสิ่งที่ชุดทดสอบในหัวข้อ 4.2.7 ใช้จริง

2. **พบการพึ่งพา Player ⟶ Clock หนึ่งจุด** คือ HeldSandglass.cs:12 ที่ประกาศ [SerializeField] private Sandglass glass;

สาเหตุคือคลาส Sandglass จงใจไม่ implement ส่วนต่อประสานใดเลย เพราะ prop นี้มี PickupItem อยู่แล้ว การใส่ IInteractable เพิ่มจะทำให้กำกวมว่าปุ่ม E ไปถึงตัวไหน (คอมเมนต์ที่

Sandglass.cs:10-11) จึงต้องเรียกเมธอดบนคลาสรูปธรรมโดยตรง เป็นเหตุผลที่สมเหตุสมผลในระดับการเล่น แต่ผลข้างเคียงทางโครงสร้างคือชั้น Player ทะลุไปรู้จักคลาสรูปธรรมในโมดูล Clock

3. **โมดูล EscapeRoom.World ไม่มีการพึ่งพาเข้าหรือออกเลย** จึงย้ายไปใช้ในโปรเจกต์อื่นได้ทันทีโดยไม่ต้องแก้อะไร เป็นโมดูลเดียวในโครงงานที่มีคุณสมบัตินี้

4.2.3 ผลการเทียบขอบเขตที่บังคับไว้กับที่โปรแกรมใช้จริง

เนื่องจากไฟล์ Assembly Definition เป็นตัวประกาศว่าโมดูลหนึ่งได้รับอนุญาตให้อ้างถึงโมดูลใดได้ จึงนำสิ่งที่ประกาศไว้มาเทียบกับสิ่งที่โปรแกรมใช้จริง ผลแสดงในตารางที่ 4.8

ตารางที่ 4.8 การเทียบขอบเขตที่ประกาศไว้กับที่โปรแกรมใช้จริง

Assembly	ประกาศให้อ้างถึงได้	โปรแกรมใช้จริง	ผล
EscapeRoom.Items	*ไม่มี*	—	ตรง
EscapeRoom.Interaction	Items	Items	ตรง
EscapeRoom.Clock	Items, Interaction	Items, Interaction	ตรง
EscapeRoom.World	*ไม่มี*	—	ตรง
EscapeRoom.Puzzle	Items, Interaction, **Clock**	Interaction เท่านั้น	**ประกาศเกินสองรายการ**
EscapeRoom.Tests	Items, Interaction, Clock, **Puzzle**	Items, Interaction, Clock	**ประกาศเกินหนึ่งรายการ**

– **ผลที่พบ:** EscapeRoom.Puzzle.asmdef ประกาศว่าอ้าง EscapeRoom.Items และ EscapeRoom.Clock ได้ แต่โปรแกรมในโฟลเดอร์ Puzzle/ ไม่ได้ใช้ชนิดข้อมูลจากสองโมดูลนั้นเลย กรณีเดียวที่พบคำว่า IClockDisplay อยู่ในคอมเมนต์ที่ ICodeInput.cs:5

– **ผลกระทบ:** ไม่ใช่ข้อบกพร่องและไม่กระทบการทำงาน แต่ทำให้ขอบเขตโมดูลหลวมกว่าที่โปรแกรมต้องการจริง หากในอนาคตมีผู้เขียนให้ Horologium ไปอ่าน ClockMovement โดยตรง คอมไพเลอร์จะไม่แจ้งเตือน เพราะการอ้างถึงเปิดไว้แล้ว การตัดรายการที่ไม่ใช้ออก

จะทำให้ไฟล์ Assembly Definition เป็นเอกสารที่บังคับใช้ได้ของการพึ่งพาที่เกิดขึ้นจริง ซึ่งเป็นเหตุผลที่เลือกใช้เครื่องมือนี้ตั้งแต่ต้น

– **เป็นข้อเสนอแนะที่แก้ได้ทันที** โดยลบสองรายการออกจากไฟล์ .asmdef และไม่ต้องแก้โปรแกรมเลย

สำหรับสาเหตุที่โมดูล Player, UI และ Game ไม่มีไฟล์ Assembly Definition มีข้อจำกัดทางเทคนิคบังคับอยู่สองข้อ

1. **พบวงจรการพึ่งพา Player ⇄ UI** (ตารางที่ 4.7) และ Unity ห้ามการอ้างถึงกันเป็นวงจรระหว่าง assembly เด็ดขาด [24] จึงใส่ไฟล์ Assembly Definition ให้สองโมดูลนี้ไม่ได้โดยไม่ตัดวงจรก่อน

2. ไฟล์ PlayerControls.cs ที่ Unity สร้างให้จากค่าอินพุตอยู่ที่ Assets/ ไม่ได้อยู่ใน Scripts/ หากสร้าง assembly สำหรับโมดูล Player คลาสในนั้นจะมองไม่เห็นไฟล์นี้เพราะอยู่ต่าง assembly

และเนื่องจากกฎของ Unity คือ predefined assembly อ้างถึง Assembly Definition ได้ แต่ Assembly Definition อ้างถึง predefined assembly ไม่ได้ [24] การวางชั้นล่างไว้ในไฟล์ Assembly Definition และปล่อยชั้นบนไว้ใน Assembly-CSharp จึงเป็นการ **บังคับทิศทางการพึ่งพาให้เป็นทางเดียวโดยอัตโนมัติ** — การเขียน GetComponent<PlayerMovement>() ลงในคลาส ClockDial จะคอมไพล์ไม่ผ่าน ผลข้างเคียงของการเลือกโครงแบบนี้คือชุดทดสอบไม่สามารถครอบสามโมดูลชั้นบนได้ ซึ่งรายงานไว้ในหัวข้อ 4.2.7

4.2.4 ผลการใช้ส่วนต่อประสานและการนำกลับมาใช้

ตารางที่ 4.9 ส่วนต่อประสานทั้งสิบสองรายการและจำนวนคลาสที่นำไปใช้

ส่วนต่อประสาน	โมดูล	จำนวน implementation	รายชื่อคลาส
IInteractable	Interaction	9	Openable, PickupItem, ClockWinder, ClockSetter, WeightChain, ClockPartSocket, TestRun, CodeWheel, Prism

ILock	Interaction	6	KeyLock, SealedLock, TimeLock, RunningLock, CodeLock, StarLock
IClockPart	Clock	3	Mainspring, Pendulum, TestRun
IItemReceiver	Interaction	2	KeyLock, ClockPartSocket
IClockDisplay	Clock	2	ClockDial, DigitalReadout
IActor	Interaction	1	PlayerInteractor (และ RecordingActor ในชุดทดสอบ)
IInspectable	Interaction	1	Inspectable
IReadable	Interaction	1	WrittenPage
IPowerStore	Clock	1	Mainspring
ICodeInput	Puzzle	1	CodeWheel
IInHandOnly	Interaction	1	TestRun (เป็น marker interface ไม่มีเมธอด)
ITimedTest	Interaction	1	TestRun
รวม		**29 ความสัมพันธ์**	จาก **22 คลาส**

วิธีนับ: นับ "ความสัมพันธ์" คือหนึ่งคลาส implement หนึ่งส่วนต่อประสาน หากหนึ่งคลาส implement สามส่วนต่อประสาน (เช่น TestRun) นับเป็นสามความสัมพันธ์ จึงได้ 29 ความสัมพันธ์จาก 22 คลาสที่ implement อย่างน้อยหนึ่งรายการ

การนำกลับมาใช้ที่วัดได้แสดงในตารางที่ 4.10 โดยเลือกเฉพาะกรณีที่ยืนยันได้จากไฟล์ ไม่ใช่กรณีที่เพียงดูเหมือนจะนำกลับมาใช้ได้

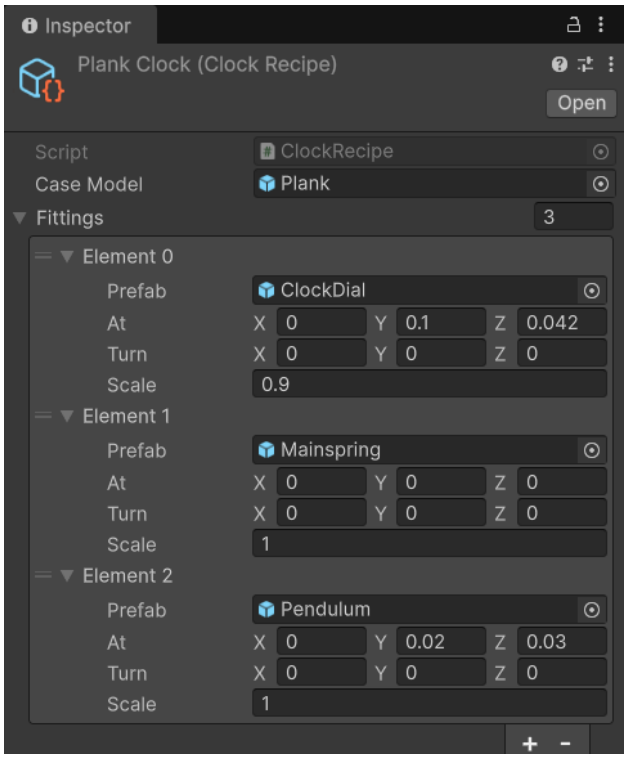
ตารางที่ 4.10 หลักฐานการนำกลับมาใช้ที่วัดได้

สิ่งที่นำกลับมาใช้	ใช้ที่ไหน	จำนวน	หลักฐาน
ClockPartSocket	ช่องใส่ลูกตุ้มและตุ้มถ่วงขององก์ 4 **และ** ช่องเสียบปริซึมขององก์ 5	5 instance	ค้น GUID ของ ClockPartSocket.cs พบใน Sundial.prefab 3 ครั้ง และใน Main.unity 2 ครั้ง รวม 5 ตรงกับที่ Unity รายงาน
CodeLock + CodeWheel	องก์ 1, 3 และ 4	CodeLock 3, CodeWheel 11	นับอินสแตนซ์ในฉาก
Prefabs/Clocks/ClockDial.prefab	หน้าปัดของนาฬิกาทุกเรือน	46 instance	นับอินสแตนซ์ของ ClockDial ในฉาก
Prefabs/Clocks/Parts/Mainspring.prefab	แหล่งพลังของนาฬิกา	45 instance	นับอินสแตนซ์ของ Mainspring ในฉาก
Prefabs/House/DoorLeaf.prefab	บานประตูภายในทั้งสี่บาน	4 บาน	นับวัตถุลูกของ House/Doors ที่ใช้ prefab นี้

กรณี ClockPartSocket เป็นหลักฐานที่หนักที่สุด เพราะเป็น **การนำกลับมาใช้ข้ามโมดูลและข้ามองก์โดยไม่แก้โปรแกรมเลย** คลาสนี้อยู่ในโมดูล Clock และเขียนขึ้นเพื่อใส่ลูกตุ้มเข้านาฬิกา โดยรับค่า ItemData กับรายการวัตถุที่จะเปิด แล้วเปิดวัตถุนั้นเมื่อใส่ของถูก (ClockPartSocket.cs:43-53) สิ่งที่ถูกเปิดคือคลาส Pendulum (ซึ่งเป็น IClockPart ในโมดูล Clock) ในกรณีหนึ่ง และคลาส Prism (ซึ่งเป็น IInteractable ในโมดูล Puzzle) ในอีกกรณี โดยที่ ClockPartSocket ไม่รู้ความต่างระหว่างสองสิ่งนั้นเลย

ผลอีกข้อหนึ่งคือ **ชนิดของนาฬิกาเป็นไฟล์ข้อมูล ไม่ใช่เมธอดในโปรแกรม** คลาส ClockRecipe เก็บตัวเรือนเปล่าหนึ่งชิ้นกับรายการสิ่งที่ประกอบเข้าไป โดยมี **รายการเดียว ไม่แยก**

ว่าอะไรคือส่วนแสดงเวลาและอะไรคือชิ้นส่วนกลไก (ClockRecipe.cs:37) และ ClockMovement แยกเองตอน Awake ด้วยการถามว่าแต่ละชิ้น implement ส่วนต่อประสานใด (ClockMovement.cs:112,116) ไฟล์ Assets/Recipes/PlankClock.asset คือหลักฐานว่าใช้งานได้จริง — เป็นนาฬิกาที่ประกอบจากไฟล์ข้อมูลเพียงไฟล์เดียว ไม่มีเมธอดในโปรแกรมรองรับเฉพาะ และมีชุดทดสอบสามกรณียืนยันว่าทำงานเหมือนนาฬิกาที่เขียนด้วยโปรแกรม



ภาพที่ 4.12 ไฟล์ PlankClock.asset ใน Inspector ให้เห็นรายการ Fittings ที่ไม่แยกส่วนแสดงเวลาจากชิ้นส่วนกลไก

จากภาพที่ 4.12 รายการ Fittings ของ PlankClock.asset ใส่หน้าปัด ลานสปริง และลูกตุ้มเรียงกันในรายการเดียว ไม่แยกว่าชิ้นใดเป็นส่วนแสดงผลหรือชิ้นส่วนกลไก

4.2.5 ผลการตรวจรูปแบบการเขียนโปรแกรมเชิงปริมาณ

ผลการนับรูปแบบการเขียนโปรแกรมที่เป็นตัวชี้วัดของการพึ่งพาแบบต่าง ๆ แสดงดังตารางที่ 4.11 ค่าที่เป็นศูนย์ในตารางนี้มีความหมายทางวิศวกรรมมากกว่าค่าที่ไม่เป็นศูนย์ เพราะเป็นรูปแบบที่สร้างการพึ่งพาแบบมองไม่เห็นหรือการพึ่งพาข้ามทั้งระบบ

ตารางที่ 4.11 ผลการนับรูปแบบการเขียนโปรแกรมในโปรแกรมส่วนที่ทำงานในเกม 63 ไฟล์

รูปแบบ	จำนวนที่พบ	ความหมายเมื่อพบ
abstract	**0**	ไม่มีคลาสฐานที่ต้องสืบทอด
virtual	**0**	ไม่มีเมธอดที่ออกแบบให้เขียนทับ
override	**0**	ไม่มีการเขียนทับเมธอดของคลาสแม่
คลาสที่สืบทอดจาก MonoBehaviour โดยตรง	**51**	ทุกคลาสอยู่ระดับเดียวกัน ไม่มีลำดับชั้นให้ไล่
Singleton (static … Instance)	**0**	ไม่มีวัตถุที่ทุกคลาสเรียกถึงได้
FindObjectOfType / FindAnyObjectByType	**0**	ไม่มีการค้นหาวัตถุด้วยชนิดตอนทำงาน
GameObject.Find	**0**	ไม่มีการค้นหาวัตถุด้วยชื่อเป็นข้อความ
SendMessage	**0**	ไม่มีการเรียกเมธอดด้วยชื่อเป็นข้อความ
DontDestroyOnLoad	**0**	ไม่มีวัตถุที่อยู่ข้ามฉาก
UnityEvent	**0**	ไม่มีการต่อเหตุการณ์ผ่าน Inspector
StartCoroutine	**0**	ไม่มีการทำงานแบบ coroutine
event Action	**5**	การแจ้งเหตุการณ์แบบ Observer (3 จุดใช้งานจริง 2 จุดประกาศในส่วนต่อประสาน)
[SerializeField]	**215** ใน 53 ไฟล์	การอ้างอิงที่ตั้งค่าผ่าน Inspector
GetComponent และตระกูล	**43**	การค้นหาผู้ร่วมงานผ่านส่วนต่อประสาน

– **ผลเชิงวิศวกรรม:** ชุดค่าศูนย์นี้ยืนยันว่าโครงงานไม่ได้ใช้รูปแบบที่สร้างการพึ่งพาแบบ global เลย ซึ่งรวมถึง Singleton ที่ทุกคลาสเรียกถึงได้ การค้นหาวัตถุด้วยชื่อหรือชนิดตอนทำงาน และการเรียกเมธอดด้วยชื่อเป็นข้อความ รูปแบบทั้งสามทำให้การพึ่งพาไม่ปรากฏในลายเซ็นของเมธอดและคอมไพเลอร์ตรวจไม่ได้ [2]

– **ผลสำคัญของการไม่มี inheritance:** การที่ abstract, virtual และ override เป็นศูนย์ทั้งหมด ขณะที่มีคลาส 51 คลาสและส่วนต่อประสาน 12 รายการ หมายความว่าความหลากหลาย (polymorphism) ในโครงงานนี้เกิดจากการ implement ส่วนต่อประสานเท่านั้น ไม่ได้เกิดจากการสืบทอด ซึ่งเป็นรูปแบบ composition over inheritance ตามที่อธิบายในบทที่ 2 [5, 9]

– **จุดที่ยังเป็นการพึ่งพาแบบแน่น:** ค่า 215 ของ [SerializeField] คือการอ้างอิงไปยัง **คลาสรูปธรรม** ที่ลากต่อใน Inspector ซึ่งมองจากตัวโปรแกรมไม่เห็นว่าอะไรต่อกับอะไร ต้องเปิด Unity ดู รายงานเป็นข้อจำกัดในหัวข้อ 4.3.4

4.2.6 ผลการประเมินความสามารถในการขยาย

ประเมินตามหลัก Open–Closed Principle [9] คือหากเพิ่มสิ่งใหม่ได้ด้วยการสร้างไฟล์ใหม่เพียงอย่างเดียว ไม่ต้องแก้ไฟล์เดิม ถือว่าผ่าน ผลการประเมินเก้าสถานการณ์แสดงในตารางที่ 4.12

ตารางที่ 4.12 ผลการประเมินความสามารถในการขยายเก้าสถานการณ์

ต้องการเพิ่ม	ต้องเขียนไฟล์ใหม่	ต้องแก้ไฟล์เดิม	ผล
วิธีแสดงเวลาแบบใหม่	1 คลาส implement IClockDisplay	ไม่ต้อง	**ผ่าน** (มีชุดทดสอบยืนยัน)
ชิ้นส่วนนาฬิกาแบบใหม่	1 คลาส implement IClockPart	ไม่ต้อง	**ผ่าน**
แหล่งพลังแบบใหม่	1 คลาส implement IPowerStore	ไม่ต้อง	**ผ่าน**
กลอนแบบใหม่	1 คลาส implement ILock	ไม่ต้อง	**ผ่าน**
วัตถุที่โต้ตอบได้แบบใหม่	1 คลาส implement IInteractable	ไม่ต้อง	**ผ่าน**
ชนิดของนาฬิกาแบบใหม่	ไม่ต้องเขียนโปรแกรมเลย สร้างไฟล์ ClockRecipe.asset	ไม่ต้อง	**ผ่าน** (มีชุดทดสอบยืนยัน)

ปริศนารหัสที่มีห้าหลัก	ไม่ต้องเขียนโปรแกรม เพิ่ม CodeWheel และเพิ่มสมาชิกในรายการ	ไม่ต้อง	**ผ่าน**
อุปกรณ์ใส่รหัสแบบใหม่ เช่น แป้นกดตัวเลข	1 คลาส implement ICodeInput	**ต้องแก้ CodeLock**	**ไม่ผ่าน**
วิธีมองวัตถุแบบใหม่ นอกจาก Hold, Turn, Zoom	—	**ต้องเพิ่มค่าใน enum และแก้ InspectView หลายจุด**	**ไม่ผ่าน**

ผลคือผ่านเจ็ดจากเก้าสถานการณ์ และสองข้อที่ไม่ผ่านมีสาเหตุต่างกัน จึงต้องแยกรายงาน

ข้อที่ไม่ผ่านข้อแรก เป็นข้อจำกัดที่ยังแก้ไม่สำเร็จ ฟิลด์ที่รับอุปกรณ์ใส่รหัสใน CodeLock ประกาศเป็นชนิด CodeWheel[] ซึ่งเป็นคลาสรูปธรรม ไม่ใช่ ICodeInput[]

```
// Assets/Scripts/Puzzle/CodeLock.cs:15-17
[SerializeField]
[Tooltip("Left to right, as the player reads them. Anything implementing
ICodeInput.")]
private CodeWheel[] wheels;          // ชนิดเป็น CodeWheel ไม่ใช่ ICodeInput
```

```
// Assets/Scripts/Puzzle/CodeLock.cs:65-70
for (int i = 0; i < wheels.Length; i++)
{
    ICodeInput input = wheels[i];     // แปลงขึ้นเป็นส่วนต่อประสานตอนใช้
    if (input.Digit != combination[i]) return;
}
```

การเขียนคลาสใหม่ที่ implement ICodeInput จึงใส่ลงในรายการนี้ไม่ได้ ต้องแก้ชนิดของฟิลด์ใน CodeLock.cs ก่อน และการเปลี่ยนเป็น ICodeInput[] ตรง ๆ ก็ทำไม่ได้ เพราะ **Unity ไม่สามารถบันทึกค่าของฟิลด์ที่เป็นชนิดส่วนต่อประสานได้** [7] ทางแก้ที่ใช้ได้มีสามทาง คือเก็บเป็น

MonoBehaviour[] แล้วแปลงชนิดตอนใช้ ใช้ attribute [SerializeReference] หรือค้นหาด้วย GetComponentsInChildren<ICodeInput>() แบบเดียวกับที่โมดูล Clock ทำ

ข้อสังเกตที่สำคัญคือ **โมดูล Clock แก้ปัญหานี้แล้วด้วยวิธีที่สาม แต่โมดูล Puzzle ยังไม่ได้แก้** จึงเป็นกรณีที่หลักการออกแบบของโครงงานยังไม่ถูกใช้อย่างทั่วถึง และคอมเมนต์ในโปรแกรมที่ระบุว่าเปลี่ยนอุปกรณ์ใส่รหัสได้โดยไม่แก้โปรแกรมนั้น **ยังไม่เป็นจริงตามสภาพของโปรแกรมปัจจุบัน** รายงานไว้ตามข้อเท็จจริง

ข้อที่ไม่ผ่านข้อที่สอง เป็นข้อแลกเปลี่ยนที่เลือกไว้โดยรู้ตัว โครงงานใช้ enum กับคำสั่ง switch อยู่สี่จุด ดังนี้

enum	จำนวนค่า	ผู้อ่านค่า	ผลเมื่อเพิ่มค่าใหม่
InspectStyle	3	InspectView (8 จุด)	ต้องแก้ InspectView
ClockCondition	5	ClockMovement.ConditionRate (:83-95)	ต้องแก้ ClockMovement
PlayerMode	5	PlayerModeController.SwitchTo (:60-74)	ต้องแก้ PlayerModeController
RoomBrightness	3	RoomLighting.LevelFor (:74-81)	ต้องแก้ RoomLighting

เกณฑ์ที่ใช้ตัดสินว่าจะใช้ enum หรือส่วนต่อประสานปรากฏสม่ำเสมอทั้งโครงงาน คือ **มิติที่รู้ค่าครบแล้วและคาดว่าจะไม่เพิ่ม ใช้ enum กับ switch ส่วนมิติที่คาดว่าจะเพิ่มเรื่อย ๆ ใช้ส่วนต่อประสาน** ซึ่งเป็นหลักที่ถูกต้องในวิศวกรรมซอฟต์แวร์ ไม่ใช่การเลี่ยงการออกแบบ มิติที่เลือกใช้ส่วนต่อประสานคือสิ่งที่แสดงเวลา ชิ้นส่วน กลอน และวัตถุที่โต้ตอบได้ ซึ่งทั้งสี่มิติมีจำนวน implementation เพิ่มขึ้นจริงระหว่างการพัฒนา (ตารางที่ 4.9) ขณะที่สี่ enum ข้างต้นมีจำนวนค่าคงเดิมตั้งแต่ถูกสร้าง

4.2.7 ผลการทดสอบด้วยชุดทดสอบอัตโนมัติ

โครงงานมีชุดทดสอบ **15 กรณีในสามไฟล์** เขียนด้วย NUnit [26] และรันผ่าน Unity Test Framework รุ่น 1.6.0 [19] ทั้งหมดเป็น **PlayMode test** คือรันในสภาพแวดล้อมการทำงานจริงของเอนจิน ใช้ attribute [UnityTest] กับเมธอดที่คืนค่า IEnumerator และสั่ง yield return null

เพื่อให้เฟรมเดินไปข้างหน้า ซึ่งจำเป็นเพราะสิ่งที่ทดสอบคือพฤติกรรมที่เกิดขึ้นข้ามเฟรม เช่น นาฬิกาเดินไปแล้วกี่นาที

ตารางที่ 4.13 ชุดทดสอบทั้ง 15 กรณีและสิ่งที่แต่ละกรณียืนยัน

#	ไฟล์	สิ่งที่กรณีนั้นยืนยัน
1	ActorSeamTests.cs	ตู้ที่ล็อกอยู่จะเรียก IActor.OfferBag และไม่เปิด
2	ActorSeamTests.cs	KeyLock คัดกุญแจผิดออก และเปิดได้เมื่อใส่กุญแจถูก
3	ActorSeamTests.cs	**กลอนสองชนิดต่างกันบน Openable เดียวทำงานร่วมกันได้โดยไม่ต้องมีคลาสใหม่** ข้ามสามโมดูล
4	ActorSeamTests.cs	SealedLock ปฏิเสธการกดทุกครั้ง เปิดได้จากภายนอกเท่านั้น และเลิกขัดขวางหลังเปิดแล้ว
5	ActorSeamTests.cs	**องก์ที่สี่ทั้งองก์ทำงานได้โดยไม่เปิดฉากและไม่มีผู้เล่น** คือ เติมพลัง นาฬิกาเดิน ตีระฆัง แล้วผนังเปิดเมื่อตีจบ ไม่ใช่เปิดทันที
6	ActorSeamTests.cs	ClockPartSocket ซึ่งอยู่ในโมดูลที่ไม่รู้ว่ามีผู้เล่น ถูกสั่งงานผ่านสัญญาเดียวกันได้
7	ClockSeamTests.cs	**คลาสแสดงเวลาที่เขียนขึ้นภายหลัง ยังได้รับเวลาจาก ClockMovement**
8	ClockSeamTests.cs	ส่วนแสดงเวลาสองตัวบนกลไกเดียวแสดงเวลาตรงกัน
9	ClockSeamTests.cs	กลไกที่ไม่มีชิ้นส่วนใดเลยเดินตลอดไป คือการไม่มีชิ้นส่วนไม่สามารถยับยั้งได้
10	ClockSeamTests.cs	แหล่งพลังที่ว่างเปล่าทำให้นาฬิกาหยุด
11	ClockSeamTests.cs	การเติมพลังทำให้นาฬิกาเดิน โดยกลไกไม่รู้ว่าใครเติม
12	ClockSeamTests.cs	**ชิ้นส่วนที่ยังปิดการทำงานอยู่ยังยับยั้งได้** ซึ่งเป็นรากของกลไกองก์ที่สี่
13	RecipeSeamTests.cs	นาฬิกาที่ประกอบจากไฟล์ข้อมูลบอกเวลา 3:25 และไม่เดินตามที่ระบุไว้ในไฟล์

14	RecipeSeamTests.cs	ชิ้นส่วนที่ระบุในไฟล์ข้อมูลยับยั้งได้จริง
15	RecipeSeamTests.cs	ส่วนแสดงเวลาที่ระบุในไฟล์ข้อมูลถูกขับเคลื่อนจริง ตรวจด้วยการวัดมุมของเข็ม

ลักษณะเฉพาะของชุดทดสอบนี้ที่ควรระบุคือ **เป็นการทดสอบว่าคุณสมบัติของโครงสร้างเป็นจริงหรือไม่ ไม่ใช่การทดสอบว่าฟังก์ชันคืนค่าถูกต้องหรือไม่** ซึ่งตรงกับนิยามของ architecture fitness function ในบทที่ 2 [20] ตัวอย่างที่ชัดที่สุดคือกรณีที่ 7

```
// Assets/Tests/PlayMode/ClockSeamTests.cs:41-67
// คลาสนี้ประกาศอยู่ในไฟล์ชุดทดสอบ จึงไม่มีอยู่ตอน ClockMovement ถูกคอมไพล์
private class CountingDisplay : MonoBehaviour, IClockDisplay
{
    public int Shown; public float Last;
    public void Show(float minutes) { Shown++; Last = minutes; }
}

[UnityTest]
public IEnumerator A_display_written_after_the_movement_still_gets_the_time()
{
    GameObject go = Clock(out ClockMovement movement);
    var display = go.AddComponent<CountingDisplay>();
    Set(movement, "startHour", 4); Set(movement, "startMinute", 40);
    go.SetActive(true);
    yield return null;
    Assert.Greater(display.Shown, 0, "the movement never called Show");
    Assert.AreEqual(4 * 60 + 40, display.Last, 1f, "it was handed the wrong time");
}
```

กรณีนี้จะล้มทันทีหากมีผู้แก้ ClockMovement ให้ถือฟิลด์ชี้ไปที่หน้าปัดโดยตรงแทนการค้นหาด้วยส่วนต่อประสาน และกรณีที่ 5 จะล้มหากมีผู้ทำให้ผนังลับเปิดได้ด้วยการกดปุ่ม E ชุดทดสอบจึงทำหน้าที่ **ปิดกั้นการถอยกลับของโครงสร้าง** ไม่ใช่เพียงตรวจความถูกต้องของค่า

หลักฐานอีกชิ้นหนึ่งคือคลาสที่ทำหน้าที่แทนผู้เล่นในชุดทดสอบ ซึ่งเป็นคลาส C# ธรรมดาสิบบรรทัด ไม่ใช่ MonoBehaviour และไม่อยู่ในฉากใด

```
// Assets/Tests/PlayMode/ActorSeamTests.cs:18-27
private class RecordingActor : IActor
{
    public readonly List<string> Said = new List<string>();
    public readonly List<IItemReceiver> BagOfferedTo = new List<IItemReceiver>();
    public readonly List<ItemData> Taken = new List<ItemData>();
    public void Say(string message) => Said.Add(message);
    public void OfferBag(IItemReceiver to) => BagOfferedTo.Add(to);
    public void Take(ItemData item) => Taken.Add(item);
}
```

คลาสนี้เปิดกลอน ใส่ชิ้นส่วนนาฬิกา และรับสิ่งของได้ครบทุกอย่างที่ผู้เล่นทำได้ในส่วนที่เกี่ยวกับกลอนและนาฬิกา ซึ่งเป็นการยืนยันเชิงปฏิบัติว่าสัญญา IActor สามเมธอดเพียงพอต่อสิ่งที่ prop ต้องการจากผู้ทำงานกับมัน



ภาพที่ 4.13 ผลการรันชุดทดสอบใน Test Runner แสดงทั้ง 15 กรณีและเวลาที่ใช้

ผลการรันชุดทดสอบแบบ PlayMode ด้วย Unity Test Runner ผ่านครบทั้ง 15 กรณี ไม่มีกรณีที่ไม่ผ่านหรือถูกข้าม ใช้เวลารวม 2.222 วินาที ดังภาพที่ 4.13

ตารางที่ 4.14 ความครอบคลุมของชุดทดสอบแยกตามโมดูล

โมดูล	มีชุดทดสอบครอบ	เหตุผลเมื่อไม่มี
Items	ครอบ (ทางอ้อม ผ่าน ItemData ในสี่กรณี)	—
Interaction	ครอบ (Openable, KeyLock, SealedLock, IActor, ILock, IItemReceiver)	—

Clock	ครอบ (ClockMovement, Mainspring, Pendulum, ClockDial, ClockPartSocket, ChimeRelease, RunningLock)	—
Puzzle	**ไม่ครอบ**	ไม่มีข้อจำกัดทางเทคนิค การอ้างถึงเปิดไว้แล้วในไฟล์ .asmdef แต่ยังไม่ได้เขียน
World	ไม่ครอบ	ไม่มีข้อจำกัดทางเทคนิค
Player	**ไม่ครอบ**	**ใส่ไม่ได้** — อยู่ใน Assembly-CSharp ซึ่ง .asmdef ของชุดทดสอบอ้างถึงไม่ได้ [24]
UI	**ไม่ครอบ**	เหตุผลเดียวกัน
Game	**ไม่ครอบ**	เหตุผลเดียวกัน
Editor tooling	ไม่ครอบ	—

หลักฐาน: ชนิดข้อมูลทั้งหมดที่ชุดทดสอบใช้คือ Openable, KeyLock, SealedLock, ClockMovement, Mainspring, Pendulum, ClockDial, ClockPartSocket, ChimeRelease, RunningLock, IPowerStore, IClockDisplay, IActor, IItemReceiver, ItemData และ ClockCondition — ไม่มีชนิดข้อมูลจากโมดูล Puzzle เลยสักรายการ

ผลนี้มีประเด็นที่ลึกกว่าการเขียนชุดทดสอบไม่ครบ คือ **การที่สามโมดูลชั้นบนไม่มีชุดทดสอบเป็นผลโดยตรงจากการเลือกโครง assembly ในขั้นที่ 4** ไม่ใช่การละเลย เพราะกฎของ Unity ห้าม Assembly Definition อ้างถึง predefined assembly [24] การจะทดสอบสามโมดูลนี้ต้องตัดวงจร Player ⇄ UI และย้ายไฟล์ PlayerControls.cs ก่อน ซึ่งเป็นงานที่ใหญ่กว่าการเขียนชุดทดสอบเอง

ส่วนโมดูล Puzzle **ไม่มีข้อจำกัดแบบนั้น** และเป็นช่องว่างที่เติมได้ทันที ซึ่งมีน้ำหนักเพิ่มขึ้นเมื่อพิจารณาร่วมกับที่เก็บค่ารหัสขององค์ที่หนึ่งในหัวข้อ 4.1.2 ชุดทดสอบที่ตรวจว่า CodeLock บนตู้โชว์ถือรหัสตรงกับเลขบนนาฬิกาที่ยังเดิน จะจับความไม่ตรงกันของค่าได้ทันที

4.2.8 ผลการทดลองเล่นกับผู้ใช้

นักศึกษาให้เพื่อนจำนวน 5 คนทดลองเล่นเกมจนจบตามที่อธิบายไว้ในหัวข้อ 3.3.4 ผลการสังเกตสรุปในตารางที่ 4.15

ตารางที่ 4.15 ผลการสังเกตจากการทดลองเล่นของเพื่อน 5 คน

องก์	สิ่งที่สังเกตได้	ความเห็นต่อการออกแบบ
1	ผู้เล่นทั้งห้าคนเข้าใจปริศนานี้ง่ายที่สุดในทั้งห้าองก์	การให้ผู้เล่นอนุมานจากเนื้อเรื่อง คือ คู่มือที่บอกให้ไขลานทุกสัปดาห์ ทำให้ผู้เล่นเข้าใจเหตุผลที่ต้องไขนาฬิกาแต่ละเรือน
3	ผู้เล่นทั้งห้าคนติดที่องก์นี้	เป็นความตั้งใจของการออกแบบ เพราะมีของหลอกหลายอย่างในห้อง รวมถึงรหัสจริงที่มีเลข 0 ปนอยู่ ผู้เล่นที่ผ่านสองห้องแรกมาโดยต้องหาคำใบ้รอบตัวจึงมักไปหาคำใบ้ต่อ
3	หลังเล่นจบ ผู้เล่นทั้งห้าคนบอกตรงกันว่าน่าจะมาดูที่กรงนกก่อน	กรงนกเป็นที่วางกุญแจและมีล้อรหัสสามล้อ จึงชี้ว่าต้องหาตัวเลขสามหลักมาใส่ ผู้เล่นจะรู้ว่าต้องหาอะไรได้ทันทีหากสังเกตสิ่งที่ต้องไปตอบก่อน
2	ผู้เล่นค่อนข้างใช้เวลานานในองค์นี้ และพยายามหาอันที่ตรง 10 วินาทีที่สุดแบบเป๊ะๆ หลังจับเวลาไปหลายเรือนก็เกิดข้อสงสัยว่า ต้องหาเรือนที่อาจจะเดินผิดปกติ หรือเดินช้า	การทำองค์นี้ต้องคำนึงถึงผู้เล่นเป็นอย่างมาก เพราะเวลาที่ผู้เล่นรอ 10 วินาที คือเวลาที่ผู้เล่นรอจริง เพราะฉะนั้นห้องนี้หากมีข้อผิดพลาด หรือนาฬิกาที่ควรจะเดินเหมือนกัน ต่างกันนิดเดียว อาจทำให้ผู้เล่นสับสนได้

4	ผู้เล่นไขปริศนาได้เร็ว สามารถนำลูกตุ้มและตุ้มถ่วง มาใส่นาฬิกาให้นาฬิกาทำงานได้ ผู้เล่นตกใจเล็กน้อยหลังนาฬิกาทำงานและเกิดเสียง	ตรงนี้ปริศนาไม่ได้ยากมาก แต่เน้นออกแบบนาฬิกาที่จะยังเดินไม่ได้หากเงื่อนไขไม่ครบ คือการนำลูกตุ้มและตุ้มถ่วงนาฬิกามาใส่
5	ผู้เล่นรู้สึกตื่นเต้นเหมือนกำลังจะได้พบกับเนื้อเรื่องหลักแล้ว และคิดว่าจะเกิดอะไรขึ้นหลังจากนี้ เมื่อเกมจบ ผู้เล่นอยากเล่นต่อ	ห้องนี้ออกแบบให้เป็นห้องสุดท้าย โดยจะวางปริซึมสำหรับทำเงื่อนไขกลไกให้สำเร็จไว้ตั้งแต่ห้องแรก

จากตารางที่ 4.15 องก์ที่ 3 เป็นจุดที่ผู้เล่นติดมากที่สุดตามที่ออกแบบไว้ และคำบอกเล่าของผู้เล่นหลังเล่นจบยืนยันว่าคำใบ้อยู่ที่กรงนกจริง นักศึกษาจึงถือว่าการออกแบบองก์นี้บรรลุเจตนา คือทำให้ผู้เล่นเรียนรู้ว่าบางครั้งคำตอบที่ง่ายที่สุดอยู่ที่สิ่งที่ต้องไปตอบ

ปริศนาองก์ที่หนึ่งใช้เหรียญบนโต๊ะบอกว่าให้เริ่มนับจากผนังด้านใด เข็มบนเหรียญชี้ทิศในพิกัดของโลก ไม่ใช่ทิศตามหน้าเหรียญ (Assets/Scripts/World/CompassNeedle.cs) หยิบเหรียญเดินไปที่ใดหรือหมุนตัวอย่างไรเข็มก็ยังชี้ผนังเดิม ทิศที่ตั้งไว้ในฉากคือ +x และเหรียญวางห่างผนังด้านนั้น 1.39 เมตร ซึ่งเป็นผนังที่ใกล้ที่สุด

ข้อจำกัดของผลการทดลองเล่น

– ผู้ทดลองเล่นห้าคนเป็นกลุ่มตัวอย่างขนาดเล็กและไม่ได้สุ่ม ผลจึงใช้ชี้แนวโน้มของการออกแบบ ไม่ใช่ใช้สรุปกับผู้เล่นทั่วไป

– ไม่มีแบบสอบถามและไม่ได้บันทึกเวลาที่ใช้แก้แต่ละองก์ ผลที่รายงานจึงเป็นการสังเกตและการสอบถามหลังเล่นจบ ไม่ใช่ตัวเลข

– ปริซึมสามชิ้นถูกเก็บในสามห้องต่างกันแต่ใช้พร้อมกันในห้องสุดท้าย หากผู้เล่นพลาดเก็บชิ้นใดชิ้นหนึ่งจะติดที่ห้องสุดท้ายโดยไม่มีคำใบ้ว่าต้องย้อนกลับไปห้องไหน จากการทดลองเล่น มีผู้เล่น 2 ใน 5 คนที่พลาดเก็บปริซึม จึงเป็นหลักฐานว่าจุดเสี่ยงนี้เกิดขึ้นจริง

4.2.9 ผลการตรวจสอบสถานะการคอมไพล์และข้อผิดพลาดที่พบ

ตรวจสอบสถานะของโปรเจกต์จากหน้าต่าง Console ของ Unity Editor เมื่อวันที่ 6 พฤศจิกายน 2569 ผลแสดงในตารางที่ 4.16

ตารางที่ 4.16 สถานะของโปรเจกต์และข้อความที่พบใน Console

รายการ	ผล
ข้อผิดพลาดการคอมไพล์	**ไม่พบ**
ฉากที่เปิดอยู่	Assets/Scenes/Main.unity
รุ่นของ Unity Editor	6000.3.5f2
ข้อยกเว้นขณะทำงาน (exception)	**พบ 1 รายการ** — NullReferenceException
คำเตือน	พบ 1 ข้อความ ซ้ำหลายครั้ง — เรื่องความละเอียดของ shadow map

ข้อผิดพลาดที่หนึ่ง เป็นข้อบกพร่องที่ยังค้างอยู่ในโปรแกรม

```
NullReferenceException: Object reference not set to an instance of an object
  DigitalReadout.SetDigit (System.Int32 place, System.Int32 value)  (at
Assets/Scripts/Clock/DigitalReadout.cs:67)
  DigitalReadout.Show (System.Single minutes)                  (at
Assets/Scripts/Clock/DigitalReadout.cs:53)
  ClockMovement.ShowTime ()                                    (at
Assets/Scripts/Clock/ClockMovement.cs:158)
  ClockMovement.Awake ()                                       (at
Assets/Scripts/Clock/ClockMovement.cs:128)
```

สาเหตุที่วิเคราะห์ได้จากโปรแกรม มีดังนี้

1. ClockMovement.Awake() ค้นหาส่วนแสดงเวลาทุกตัวที่ประกอบอยู่ด้วย GetComponentsInChildren<IClockDisplay>(true) แล้วเรียก ShowTime() ทันทีในบรรทัดสุดท้ายของ Awake (ClockMovement.cs:128) เพื่อให้นาฬิกาแสดงเวลาตั้งต้นทันทีที่ฉากโหลด

2. DigitalReadout เตรียมอาเรย์ _faces ที่เก็บ material ของแต่ละหลักไว้ใน Awake() ของตัวเอง (DigitalReadout.cs:36-42)

3. ทั้งสองคลาสอยู่บน GameObject **ต่างกัน** — ClockMovement อยู่บน BenchMovement และ DigitalReadout อยู่บน TimingMachine ซึ่งเป็นวัตถุลูก (ตรวจจากไฟล์ Main.unity)

4. **Unity ไม่รับประกันลำดับการเรียก Awake ระหว่าง component ที่อยู่บน GameObject ต่างกัน** [17] และตรวจแล้วว่าโปรเจกต์นี้ **ไม่ได้ตั้งค่า Script Execution Order ไว้เลย**

5. ดังนั้นหาก ClockMovement.Awake() ทำงานก่อน DigitalReadout.Awake() อาเรย์ _faces จะยังเป็น null ขณะที่ SetDigit เข้าถึง _faces[place] ที่บรรทัด 67 จึงเกิดข้อยกเว้น

ความสำคัญของข้อบกพร่องนี้ คือเป็น **ปัญหาชนิดเดียวกันกับที่โครงงานแก้ไปแล้วหนึ่งจุด** ในขั้นการพัฒนาโปรแกรม ปัญหาลำดับการทำงานระหว่าง TestRun กับ ClockMovement ถูกแก้โดยย้ายการนับไปไว้ใน LateUpdate ซึ่งรับประกันว่าทำงานหลัง Update ทุกตัว แต่ลำดับของ Awake ยังไม่ได้รับการจัดการด้วยวิธีใด จึงเป็นข้อบกพร่องประเภท **ลำดับการทำงานของวงจรชีวิตที่ไม่ถูกกำหนด** ที่เหลืออยู่

วัตถุที่ได้รับผลกระทบคือเครื่องจับเวลาบนโต๊ะช่าง ซึ่งเป็นวัตถุชิ้นเดียวในเกมที่มีส่วนแสดงเวลาสองชนิดบนกลไกเดียว และเป็นวัตถุที่ตั้งใจวางไว้เพื่อแสดงคุณสมบัติของส่วนต่อประสาน IClockDisplay โดยตรง

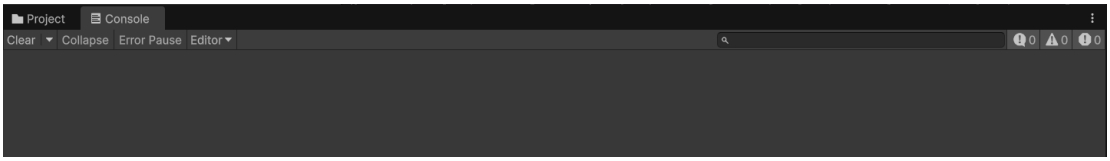
ข้อจำกัดของการรายงานข้อนี้: ข้อความใน Console ยืนยันว่าข้อผิดพลาดเกิดขึ้นจริง และกลไกที่ทำให้เกิดยืนยันได้จากตัวโปรแกรม แต่ **ยังยืนยันไม่ได้ว่าเกิดขึ้นทุกครั้งที่เปิดเกมหรือไม่** เพราะลำดับการเรียก Awake ที่ไม่ถูกกำหนดนั้นในทางปฏิบัติอาจคงที่บนเครื่องเดียวและเปลี่ยนเมื่อย้ายเครื่อง การยืนยันต้องกด Play ตรวจซ้ำ

วิธีแก้ที่เสนอ มีสองทาง ทางแรกคือให้ DigitalReadout เตรียม _faces แบบ lazy คือตรวจและเตรียมในครั้งแรกที่ Show ถูกเรียก ซึ่งแก้ที่ต้นเหตุและไม่ขึ้นกับลำดับ ทางที่สองคือตั้ง Script Execution Order ให้ DigitalReadout ทำงานก่อน ClockMovement ซึ่งแก้ได้แต่ผูกการทำงานของสองคลาสไว้กับการตั้งค่าในโปรเจกต์ที่มองจากโปรแกรมไม่เห็น จึงเสนอทางแรก

คำเตือนที่สอง เป็นผลที่ประเมินแล้วและยอมรับไว้ ไม่ใช่ข้อบกพร่อง

```
Reduced additional punctual light shadows resolution by 2 to make 24 shadow maps fit in the
4096x4096 shadow atlas. To avoid this, increase shadow atlas size, decrease big shadow
resolutions, or reduce the number of shadow maps active in the same frame
```

แหล่งกำเนิดแสงแบบจุดหนึ่งดวงต้องใช้ shadow map หกใบ ค่า 24 ใบจึงหมายถึงแสงแบบจุดที่สร้างเงาสี่ดวง ซึ่งตรงกับจำนวนที่กำหนดไว้ในโครงงาน URP แก้ปัญหาด้วยการลดความละเอียดลงครึ่งหนึ่งเองโดยไม่หยุดทำงาน และ 4096×4096 เป็นขนาด atlas ที่ใหญ่ที่สุดที่ URP รองรับ [21] คำเตือนนี้จึงทำให้หายไปไม่ได้นอกจากลดจำนวนแสงที่สร้างเงาลงเหลือสองดวง ซึ่งจะทำให้รูปทรงของห้องอ่านไม่ออก เป็น**ข้อแลกเปลี่ยนที่เลือกไว้** ไม่ใช่ปัญหาที่ยังไม่ได้แก้



ภาพที่ 4.14 หน้าต่าง Console ของ Unity แสดงสถานะการคอมไพล์และข้อความที่พบ

จากภาพที่ 4.14 หน้าต่าง Console แสดงสถานะการคอมไพล์และข้อความที่พบ ณ เวลาที่ตรวจ การตรวจซ้ำเมื่อวันที่ 10 พฤศจิกายน 2569 พบว่าซอร์สโค้ดของ DigitalReadout และ ClockMovement ยังเป็นรูปแบบเดิม จึงยังมีความเสี่ยงที่ข้อผิดพลาดนี้จะเกิดขึ้นได้ ส่วน Console ขณะตรวจไม่พบข้อความ NullReferenceException ซึ่งสอดคล้องกับที่ลำดับการเรียก Awake บนเครื่องหนึ่งอาจคงที่และเปลี่ยนเมื่อย้ายเครื่อง ข้อบกพร่องนี้จึงยังไม่ถือว่าแก้ไขแล้ว และเสนอวิธีแก้ไว้ในหัวข้อ 5.2.3

4.3 วิจารณ์ข้อมูลโดยเปรียบเทียบผลที่ได้รับกับวัตถุประสงค์และจุดมุ่งหมายของการปฏิบัติงานหรือการจัดทำโครงงาน

4.3.1 ผลเทียบกับวัตถุประสงค์ของโครงงาน

ผลการดำเนินงานเทียบกับวัตถุประสงค์ทั้งสี่ข้อของโครงงานตามหัวข้อ 1.8 เป็นดังตารางที่ 4.17

ตารางที่ 4.17 ผลการดำเนินงานเทียบกับวัตถุประสงค์ของโครงงาน

วัตถุประสงค์	ผลที่ได้	หลักฐาน	ผล
1. ศึกษาและประยุกต์ใช้หลักการออกแบบซอฟต์แวร์แบบมอดูลาร์ในการพัฒนาเกม	แบ่งโค้ดเป็นเก้ากลุ่มตามหน้าที่ หกกลุ่มมีขอบเขตที่คอมไพเลอร์บังคับ โมดูล Clock และ Puzzle ไม่อ้างถึง Player หรือ UI เลย และไม่มีการสืบทอดเลย	ตารางที่ 4.6 ตารางที่ 4.7 ตารางที่ 4.9 ตารางที่ 4.11	บรรลุ ยกเว้นวงจร Player กับ UI ที่ยังเหลืออยู่
2. ออกแบบและพัฒนาคอมโพเนนต์ที่นำกลับมาใช้ซ้ำและประกอบเป็นวัตถุที่มีพฤติกรรมต่างกันได้	นาฬิกา 46 เรือนใช้หน้าปัดและกลไกชุดเดียวกัน ต่างกันที่ชิ้นส่วนที่ประกอบ และ ClockPartSocket ใช้ซ้ำข้ามองก์ 4 และ 5 โดยไม่แก้โปรแกรม	ตารางที่ 4.4 ตารางที่ 4.10	บรรลุ
3. พัฒนาเกมห้องหลบหนีสามมิติที่ประกอบด้วยปริศนาทั้งห้าองก์ตามที่ออกแบบไว้	เกมเล่นจบได้ ห้าห้องเส้นทางเดียว ปริศนาครบห้าองก์และตรวจคำตอบแล้ว	ตารางที่ 4.1 ตารางที่ 4.2	บรรลุ
4. ทดสอบการทำงานของระบบและตรวจสอบความถูกต้องของโครงสร้างที่พัฒนาขึ้น	ชุดทดสอบ PlayMode 15 กรณี การวัดโครงสร้างจากซอร์สโค้ด และการทดลองเล่นกับผู้ใช้ 5 คน	ตารางที่ 4.15 ตารางที่ 4.14	บรรลุบางส่วน ชุดทดสอบครอบคลุมเฉพาะสามโมดูลชั้นล่าง

ผลสำคัญที่ตรงกับแกนของโครงงานคือ วัตถุใหม่เกิดจากการประกอบคอมโพเนนต์ที่มีอยู่ได้จริง แต่มีขอบเขตที่ต้องระบุให้ถูกต้อง สำหรับนาฬิกา การสร้างชนิดใหม่ทำได้ด้วยไฟล์ข้อมูลไฟล์เดียว โดยไม่เขียนโปรแกรมเพิ่ม และมีชุดทดสอบสามกรณียืนยัน ส่วนวัตถุปริศนาชนิดที่ยังไม่มีอยู่ ต้องเพิ่มคอมโพเนนต์ใหม่หนึ่งไฟล์ โดยไม่ต้องแก้ไฟล์เดิม ตัวอย่างคือ TestRun Sandglass StampedNumber และ Horologium

4.3.2 ผลเทียบกับขอบเขตที่กำหนดไว้

ตารางที่ 4.18 ผลเทียบกับขอบเขตที่กำหนดไว้ในหัวข้อ 3.1

ขอบเขตที่กำหนดไว้	ผล	หลักฐาน
เดินด้วย WASD ไม่มีการกระโดด ย่อ หรือวิ่ง	**ทำได้**	PlayerMovement.cs มีเพียงการเดินและแรงโน้มถ่วง
ปุ่ม F ดูวัตถุที่ชี้อยู่	**ทำได้**	Inspectable รองรับสามรูปแบบคือ Hold, Turn และ Zoom
ปุ่ม E โต้ตอบ เก็บของ และเปิดกระเป๋า	**ทำได้**	PlayerInteractor ส่งปุ่ม E ให้ IInteractable ที่พบ
ปุ่ม Q ออกจากมุมมองย่อย	**ทำได้**	PlayerModeController สลับชุดการควบคุมสามชุด
ข้อความ "ไม่มีอะไรเกิดขึ้น" เมื่อเลือกของผิด	**ทำได้**	InventoryUI.cs:134 ทำงานเมื่อ IItemReceiver.Give คืนค่าเท็จ
ทำเกมทั้งเกม ไม่ใช่เพิ่มด่านในเกมที่มีอยู่	**ทำได้**	ห้าห้อง ห้าองก์ สร้างขึ้นทั้งหมดในโครงงาน
ออกแบบปริศนาเอง เพื่อให้เข้าเกณฑ์ Capstone Project	**ทำได้**	ปริศนาทั้งห้าองก์และการจัดวางฉากเป็นงานออกแบบของนักศึกษา
ระยะเวลาหนึ่งสัปดาห์สำหรับออกแบบธีมและปริศนาเพื่อนำเสนอแนวทางของเกม	**ทำได้**	ใช้เวลา 1 สัปดาห์ออกแบบและนำเสนอแนวทางของเกม ส่วนการพัฒนาตัวเกมทั้งหมดใช้เวลา 4 เดือนของการฝึกสหกิจศึกษา (หัวข้อ 3.1)

สิ่งที่อยู่นอกขอบเขตและไม่ได้ทำ ซึ่งระบุไว้แล้วในหัวข้อ 3.1 ได้แก่ การบันทึกและโหลดเกม ระบบคำใบ้ เสียงแบบสามมิติ และการรองรับหลายภาษา ทั้งสี่รายการยืนยันแล้วว่าไม่มีอยู่ในโปรเจกต์ โดยไม่พบการเรียก PlayerPrefs หรือ JsonUtility หรือการเขียนไฟล์ใด ๆ และข้อความทั้งหมดในเกมเป็นภาษาไทยที่ฝังอยู่ในโปรแกรมและในค่าที่ตั้งผ่าน Inspector

4.3.3 ผลการประเมินคุณภาพของโครงสร้างตามเกณฑ์

ประเมินตามเกณฑ์คุณภาพซอฟต์แวร์ที่อ้างในบทที่ 2 โดยเกณฑ์การพึ่งพาและความเกาะเกี่ยวใช้นิยามของ Stevens, Myers และ Constantine [3] เกณฑ์การแยกส่วนใช้นิยามของ Parnas [2] และเกณฑ์การบำรุงรักษาและความสามารถในการทดสอบอ้างตาม ISO/IEC 25010 [4] ทุกช่องในตารางที่ 4.19 มีทั้งหลักฐานที่สนับสนุนและข้อจำกัดที่พบ เพื่อไม่ให้การประเมินเอนไปข้างใดข้างหนึ่ง

ตารางที่ 4.19 ผลการประเมินคุณภาพของโครงสร้างแปดเกณฑ์

เกณฑ์	ผลการประเมิน	หลักฐานที่สนับสนุน	ข้อจำกัดที่พบ
การแยกหน้าที่ (Separation of Concerns)	**ดี**	สถานะแยกจากการเคลื่อนไหว (Openable + SwingMotion) · เวลาแยกจากการแสดงเวลา (ClockMovement + IClockDisplay) · ตรรกะแยกจากเสียง (TestRun + RunTicking)	InspectView รวมแปดความรับผิดชอบไว้ในคลาสเดียว
ความเป็นโมดูล (Modularity)	**ดี–ดีมาก**	หกโมดูลมีขอบเขตบังคับระดับการคอมไพล์ · โมดูล Clock และ Puzzle อ้างถึง Player และ UI เป็นศูนย์ · โมดูล World แยกสมบูรณ์	สามโมดูลชั้นบนไม่มีขอบเขตบังคับเพราะมีวงจร · Puzzle.asmdef ประกาศเกินสองรายการ

การพึ่งพา (Coupling)	**ต่ำในชั้นล่าง · ปานกลางถึงสูงในชั้นบน**	สี่จุดเชื่อมใช้การค้นหาผ่านส่วนต่อประสาน · สัญญา IActor มีเพียงสามเมธอด · Singleton และการค้นหาวัตถุตอนทำงานเป็นศูนย์	การอ้างอิงผ่าน Inspector 215 จุดชี้ไปยังคลาสรูปธรรม · CodeLock ↔ CodeWheel และ Horologium ↔ Prism ผูกกันสองทาง · วงจร Player ⇄ UI
ความเกาะเกี่ยว (Cohesion)	**สูงเกือบทั้งหมด**	โปรแกรมส่วนที่ทำงานในเกม 63 ไฟล์ เฉลี่ย 55 บรรทัดต่อไฟล์ · ClockDial 32 บรรทัด ทำเรื่องเดียว · Pendulum 61 บรรทัด	InspectView 317 บรรทัด ใหญ่กว่าคลาสอื่นเกือบเท่าตัว
การนำกลับมาใช้ (Reusability)	**ดีมาก มีหลักฐานข้ามองก์**	ClockPartSocket ใช้ทั้งองก์ 4 และ 5 โดยไม่แก้โปรแกรม · ClockDial.prefab หนึ่งไฟล์ใช้ 46 เรือน · ClockRecipe สร้างนาฬิกาใหม่ได้โดยไม่เขียนโปรแกรม	นำกลับมาใช้ได้เฉพาะจุดเชื่อมที่ใช้การค้นหาผ่านส่วนต่อประสาน · ICodeInput ยังนำกลับมาใช้จริงไม่ได้
ความสามารถในการขยาย (Extensibility)	**ดีในมิติที่ออกแบบไว้**	ผ่านเจ็ดจากเก้าสถานการณ์ (ตารางที่ 4.12)	สองสถานการณ์ต้องแก้ไฟล์เดิม — ICodeInput และ enum สี่จุด

ความสามารถในการทดสอบ (Testability)	**ดีเฉพาะชั้นล่าง**	15 กรณีรันได้โดยไม่เปิดฉากและไม่ต้องมีผู้เล่น · RecordingActor สิบบรรทัดทำงานแทนผู้เล่นได้	ไม่มีชุดทดสอบครอบโมดูล Puzzle, Player, UI และ Game
ความสามารถในการบำรุงรักษา (Maintainability)	**ปานกลาง**	ไม่มีลำดับชั้นการสืบทอดให้ไล่ · คอมเมนต์อธิบายเหตุผล ไม่ใช่สิ่งที่โปรแกรมทำ	**ไม่มีระบบควบคุมรุ่นของซอร์สโค้ดเลย** · การพึ่งพา 215 จุดมองจากโปรแกรมไม่เห็น · นโยบายไม่ตรวจค่าว่างทำให้การลืมต่อสายหนึ่งเส้นกลายเป็นข้อผิดพลาดขณะเล่น

4.3.4 ข้อจำกัดและความเสี่ยงที่พบ

ตารางที่ 4.20 ข้อจำกัดที่พบ เรียงตามผลกระทบ

#	ข้อจำกัด	ผลกระทบ
1	**ไม่มีระบบควบคุมรุ่นของซอร์สโค้ด** ตรวจแล้วไม่พบโฟลเดอร์ .git	การต่อสายใน Inspector ผิดหรือลบวัตถุพลาดย้อนกลับไม่ได้ และไม่มีประวัติการพัฒนาเป็นหลักฐาน
2	**การอ้างอิงผ่าน Inspector 215 จุดมองจากโปรแกรมไม่เห็น**	ต้องเปิด Unity จึงรู้ว่าอะไรต่อกับอะไร เมื่อรวมกับนโยบายไม่ตรวจค่าว่าง การลืมต่อหนึ่งเส้นทำให้เกมหยุดทำงานขณะเล่น

3	**NullReferenceException ที่ DigitalReadout.cs:67** จากลำดับการเรียก Awake ที่ไม่ถูกกำหนด (หัวข้อ 4.2.9)	เครื่องจับเวลาบนโต๊ะช่างอาจแสดงเวลาไม่ได้ และเป็นข้อบกพร่องที่ยังค้าง
4	**InspectView 317 บรรทัด รับผิดชอบแปดอย่าง**	ความเกาะเกี่ยวต่ำที่สุดในโครงงาน และเป็นจุดเดียวที่ขัดกับหลักหนึ่งคลาสหนึ่งหน้าที่ที่โครงงานตั้งไว้เอง
5	**ICodeInput ยังนำกลับมาใช้จริงไม่ได้**	คอมเมนต์ในโปรแกรมระบุคุณสมบัติที่โปรแกรมปัจจุบันยังไม่มี
6	**ไม่มีชุดทดสอบครอบโมดูล Puzzle, Player, UI และ Game**	โมดูล Puzzle เติมได้ทันที สามโมดูลหลังติดข้อจำกัดของโครง assembly
7	**ไม่มีการจัดการสถานะความก้าวหน้าแบบรวมศูนย์**	ไม่มีการบันทึกและโหลดเกม ไม่มีระบบคำใบ้ และไม่มีคลาสใดตอบได้ว่าผู้เล่นอยู่องก์ที่เท่าไร
8	**วงจรการพึ่งพา Player ⇄ UI**	ใส่ขอบเขตบังคับให้สามโมดูลชั้นบนไม่ได้ และทำให้ทดสอบไม่ได้ด้วย

บทที่ 5
บทสรุปและข้อเสนอแนะ

5.1 สรุปผลการดำเนินโครงงาน

โครงงานนี้มีวัตถุประสงค์เพื่อฝึกฝนและศึกษากระบวนการพัฒนาเกมสามมิติ โดยพัฒนาเกมห้องหลบหนีมุมมองบุคคลที่หนึ่งด้วยโปรแกรม Unity ตามโจทย์ที่ได้รับมอบหมายจากพนักงานที่ปรึกษา ซึ่งกำหนดรูปแบบการควบคุมและเงื่อนไขการเล่นไว้ให้แล้ว ส่วนการออกแบบปริศนาและการจัดวางฉากเป็นงานที่นักศึกษาออกแบบขึ้นเอง นักศึกษาได้ดำเนินงานครบทั้งสี่ขั้นของกระบวนการ คือ การออกแบบแนวคิดและเนื้อหา การออกแบบระบบเกม การพัฒนาโปรแกรม และการทดสอบการทำงานของเกม

ในการพัฒนา นักศึกษากำหนดแนวทางการออกแบบซอฟต์แวร์แบบมอดูลาร์ตั้งแต่เริ่มโครงงาน โดยให้วัตถุแต่ละชิ้นเกิดจากการประกอบคอมโพเนนต์ย่อยที่รับผิดชอบหน้าที่เฉพาะด้าน แทนการสร้างคลาสแยกสำหรับวัตถุแต่ละประเภท และเลือกนาฬิกาเป็นองค์ประกอบหลักของปริศนา เพราะนาฬิกามีหน้าปัดและกลไกร่วมกัน แต่แสดงพฤติกรรมต่างกันได้

ผลการดำเนินงานคือเกมที่เล่นจบได้ ประกอบด้วยห้าห้องเรียงต่อกันเป็นเส้นทางเดียว แต่ละห้องมีปริศนาประจำห้อง และมีนาฬิกาในเกมรวม 46 เรือนที่สร้างจากหน้าปัดและกลไกพื้นฐานร่วมกัน ผลการวัดโครงสร้างของซอฟต์แวร์ที่สนับสนุนแนวทางนี้ ได้แก่ การแบ่งโปรแกรมเป็นเก้าโมดูลตามหน้าที่ โดยหกโมดูลมีขอบเขตที่คอมไพเลอร์บังคับด้วยไฟล์ Assembly Definition [24] ไม่มีการสืบทอดคลาสเลย คือคำสั่ง abstract virtual และ override เป็นศูนย์ทั้งโครงงาน ขณะที่มีคลาส 51 คลาสและส่วนต่อประสาน 12 รายการ โมดูลที่ทำหน้าที่เรื่องนาฬิกาและปริศนาไม่อ้างถึงคลาสของผู้เล่นแม้แต่ครั้งเดียว จึงคอมไพล์และทดสอบได้โดยไม่ต้องมีผู้เล่นอยู่ในระบบ และการประเมินตามหลัก Open–Closed Principle [9] ผ่านเจ็ดจากเก้าสถานการณ์ ขนาดของงานคือโปรแกรมภาษา C# 78 ไฟล์ รวม 10,604 บรรทัด (นับบรรทัดที่ไม่ว่าง) ซึ่งกว่าครึ่งเป็นเครื่องมือสร้างฉากและทรัพยากรสำหรับตัวแก้ไข

การเพิ่มสิ่งใหม่ด้วยการประกอบคอมโพเนนต์เดิมได้จริงภายในขอบเขตที่ระบุไว้ในบทที่ 4 คือ นาฬิกาชนิดใหม่สร้างได้จากไฟล์ข้อมูลไฟล์เดียวโดยไม่เขียนโปรแกรมเพิ่ม ส่วนวัตถุปริศนาชนิดที่ยังไม่มี ต้องเพิ่มคอมโพเนนต์ใหม่หนึ่งไฟล์ โดยไม่ต้องแก้ไฟล์เดิม ผลที่ยังไม่สมบูรณ์ ได้แก่ วงจรการพึ่งพาระหว่างโมดูล Player กับ UI ชุดทดสอบที่ครอบคลุมเฉพาะโมดูลชั้นล่าง และข้อบกพร่องเรื่องลำดับการเรียก Awake ที่ยังค้างอยู่ ซึ่งรายงานไว้ในหัวข้อ 5.2

ความรู้และทักษะที่ได้รับ จากการปฏิบัติงานนักศึกษาได้เรียนรู้กระบวนการพัฒนาเกมสามมิติอย่างครบวงจร ตั้งแต่การคิดเนื้อเรื่องและออกแบบปริศนาให้ผู้เล่นอนุมานหรือทดลองเอง ไปจนถึงการทดสอบกับผู้เล่นจริงและเห็นว่าผู้เล่นคิดต่างจากที่ผู้ออกแบบคาดไว้ ได้ฝึกการออกแบบซอฟต์แวร์แบบมอดูลาร์ในงานจริง ทั้งการประกอบแทนการสืบทอด การใช้ส่วนต่อประสานเป็นรอยต่อระหว่างโมดูล และการใช้ Assembly Definition บังคับขอบเขต ได้ใช้เครื่องมือของ Unity อย่างกว้างขวาง ได้แก่ Prefab ScriptableObject Input System Test Framework และการเขียนเครื่องมือสำหรับตัวแก้ไข รวมถึงได้เรียนรู้ปัญหาเฉพาะของการพัฒนาเกม เช่น การนับเวลาให้เป็นอิสระจากความเร็วของเครื่อง และความแม่นยำของเลขทศนิยม

5.2 ปัญหาและแนวทางการแก้ไข

หัวข้อนี้รายงานปัญหาทางวิศวกรรมสามเรื่องที่พบระหว่างและหลังการพัฒนา โดยสองเรื่องแรกแก้ไขแล้ว และเรื่องที่สามยังค้างอยู่

5.2.1 การนับเวลาของนาฬิกาผูกกับความเร็วของเครื่องที่ใช้เล่น

อาการของปัญหา ระหว่างการพัฒนาพบว่าการนับเวลาของโปรแกรมชุดแรกอ้างกับรอบการทำงานของหน่วยประมวลผล เมื่อนำไปเล่นบนเครื่องที่ต่างกัน เครื่องที่เร็วทำให้นาฬิกาในเกมเดินเร็ว และเครื่องที่ช้าทำให้นาฬิกาเดินช้า ทั้งที่นาฬิกาทรายในเกมควรจับเวลาได้สิบวินาทีเท่ากันทุกเครื่อง

ความสำคัญของปัญหานี้ต่อตัวเกม ปัญหานี้ทำให้ปริศนาองก์ที่สองใช้เล่นไม่ได้ เพราะกลไกของปริศนาคือการจับเวลานาฬิกาพกสิบสองเรือนเทียบกับนาฬิกาทรายสิบวินาที แล้วหาเรือนที่เดินช้ากว่าเรือนอื่น หากทั้งนาฬิกาพกและนาฬิกาทรายเดินตามความเร็วของเครื่อง การเปรียบเทียบจะไม่ให้ผลที่คงที่ และคำตอบของปริศนาจะเปลี่ยนไปตามเครื่องที่ใช้เล่น

สาเหตุเชิงหลักการ การนับเวลาโดยอ้างกับรอบการทำงานของเครื่องทำให้ปริมาณที่เพิ่มต่อหนึ่งเฟรมเป็นค่าคงที่ ขณะที่จำนวนเฟรมต่อวินาทีเปลี่ยนไปตามเครื่อง ผลคือเวลาที่นับได้ต่อหนึ่งวินาทีจริงแปรผันตามอัตราเฟรม การแก้ที่ถูกต้องคือให้ปริมาณที่เพิ่มต่อหนึ่งเฟรมเป็นสัดส่วนกับเวลาจริงที่เฟรมนั้นใช้ไป ซึ่งใน Unity คือค่า Time.deltaTime [14]

แนวทางการแก้ไขที่ใช้ แก้ด้วยกลไกสี่อย่างที่แยกจากกัน และแต่ละอย่างแก้ปัญหาต่างข้อ ดังตารางที่ 5.1

ตารางที่ 5.1 กลไกสี่อย่างที่ใช้แก้ปัญหาความถูกต้องของการนับเวลา

#	กลไก	หลักฐานในโปรแกรม	แก้ปัญหาข้อใด
1	การนับเวลาทุกจุดใช้ Time.deltaTime ซึ่งเป็นเวลาจริงของเฟรมนั้น	ClockMovement.cs, Sandglass.cs, TestRun.cs · ค้นทั้งโปรเจกต์ไม่พบการใช้ Time.frameCount เลย	อัตราเฟรมต่างกันแต่เวลาที่นับได้เท่ากัน
2	สะสมเวลาในชนิดข้อมูล double ไม่ใช่ float	ClockMovement.cs พร้อมคำอธิบายในโปรแกรม	ความคลาดเคลื่อนสะสมจากการบวกเลขทศนิยมที่มีขนาดต่างกันมาก
3	เฟรมสุดท้ายของการจับเวลาคิดเป็นเศษส่วนของเฟรม	TestRun.cs ใช้ Mathf.Clamp01((Length - _ran) / Time.deltaTime)	เข็มหยุดตรงขีดเดิมเสมอ ไม่เลยขีดไปตามความยาวของเฟรมที่คร่อมเส้นชัย
4	ตัวจับเวลานับใน LateUpdate ส่วนนาฬิกาเดินใน Update	TestRun.cs พร้อมคำอธิบายในโปรแกรม	ลำดับการทำงานของ Update ระหว่างคอมโพเนนต์ที่ Unity ไม่กำหนดไว้

กลไกที่สองเป็นข้อที่ลึกที่สุดและไม่ใช่เรื่องของอัตราเฟรมโดยตรง แต่เป็นเรื่องความแม่นยำของเลขทศนิยม คำอธิบายที่บันทึกไว้ในโปรแกรมระบุสาเหตุไว้ชัดเจน คือนาฬิกานับเวลาด้วยการบวกค่าเล็กมากของหนึ่งเฟรมเข้ากับยอดสะสมที่โตขึ้นเรื่อย ๆ และเมื่อสองจำนวนนั้นมีขนาดต่างกันมาก การบวกในชนิด float จะปัดเศษของตัวที่เล็กกว่าทิ้ง

ตัวอย่างที่บันทึกไว้คือ ที่เวลา 4:40 ยอดสะสมมีค่า 280 นาที ซึ่งในชนิด float ระยะห่างระหว่างค่าที่แทนได้สองค่าที่ติดกันคือประมาณ 0.00003 นาที ขณะที่นาฬิกาที่เดินช้าเพิ่มค่าเพียง 0.00017 นาทีต่อเฟรม ผลคือราวหนึ่งในห้าของทุกก้าวถูกปัดทิ้ง และปัดทิ้งไม่เท่ากันในนาฬิกาแต่ละเรือน เพราะขนาดของระยะห่างนั้นขึ้นกับเวลาที่นาฬิกาเรือนนั้นกำลังแสดง ปรากฏการณ์นี้ตรงกับที่อธิบายไว้ในทฤษฎีความแม่นยำของเลขทศนิยม [16] และเป็นผลโดยตรงของการที่มาตรฐาน IEEE 754 กำหนดให้ระยะห่างระหว่างค่าที่แทนได้แปรตามขนาดของเลข [15]

เมื่อเปลี่ยนไปสะสมในชนิด double ระยะห่างดังกล่าวลดลงเหลือราว 6×10^{-14} นาที ซึ่งเล็กกว่าค่าที่เพิ่มต่อเฟรมหลายอันดับขนาด ปัญหาจึงหมดไป ทั้งนี้ส่วนที่เปิดให้ภายนอกเรียกยังใช้ float อยู่ เพราะการแปลงชนิดหนึ่งครั้งที่ขอบของคลาสทำให้คลาดเคลื่อนเพียง 0.01 องศาและเกิดเพียงครั้งเดียว สิ่งที่ทำในชนิด float ไม่ได้คือการสะสม ไม่ใช่การส่งค่าออก

กลไกที่สี่เป็นปัญหาเรื่องลำดับการทำงาน ซึ่งเดิมทั้งตัวจับเวลาและตัวนาฬิกานับอยู่ใน Update ทั้งคู่ และ Unity ไม่กำหนดลำดับการเรียก Update ระหว่างคอมโพเนนต์ [17] ผลคือตัวใดทำงานก่อนจะเป็นผู้ตัดสินว่าเฟรมสุดท้ายของการจับเวลาถูกนับหรือถูกข้าม การย้ายตัวจับเวลาไปไว้ใน LateUpdate ซึ่งรับประกันว่าทำงานหลัง Update ทุกตัวเสมอ แก้ปัญหานี้ได้โดยไม่ต้องตั้งค่า Script Execution Order ในโปรเจกต์

ผลหลังแก้ไข คำอธิบายที่บันทึกไว้ในโปรแกรมระบุผลการวัดไว้ว่า นาฬิกาสิบสองเรือนที่เหมือนกันทุกเรือนและถูกจับเวลาด้วยเงื่อนไขเดียวกัน เดิมเข็มหยุดกระจายอยู่ระหว่าง 59.3 ถึง 61.3 องศา และหลังแก้ไขทุกเรือนหยุดที่ 60.00 องศา ส่วนเรือนที่ตั้งให้เดินช้าหยุดที่ 60.00 องศา เมื่อผ่านไป 25.01 วินาที

5.2.2 การพึ่งพาระหว่างโมดูลปริศนากับคลาสของผู้เล่น

อาการของปัญหา วัตถุที่ผู้เล่นโต้ตอบได้ทุกชิ้นในเกม คือ นาฬิกา ลิ้นชัก และวงล้อรหัส เดิมรับคลาส PlayerInteractor ซึ่งเป็นคลาสของผู้เล่นทั้งคลาสเข้ามาเป็นพารามิเตอร์ ผลคือโมดูลที่ทำหน้าที่เรื่องนาฬิกาและปริศนาถูกคอมไพล์ผูกกับคลาสของผู้เล่นของเกมนี้โดยตรง ทั้งที่สิ่งที่วัตถุเหล่านั้นต้องการจากผู้เล่นมีเพียงสามอย่าง

ผลเสียของการพึ่งพาแบบนั้น มีสามข้อ ข้อแรกคือโมดูลปริศนาและนาฬิกาจะนำไปใช้ที่อื่นไม่ได้ เพราะต้องลากคลาสผู้เล่นของเกมนี้ไปด้วย ข้อที่สองคือการแก้คลาสผู้เล่นจะกระทบการคอมไพล์ของทุกโมดูลที่อ้างถึงมัน และข้อที่สามคือเขียนชุดทดสอบไม่ได้ เพราะการทดสอบกลอนประตูหนึ่งบานจะต้องสร้างผู้เล่นทั้งตัวขึ้นมาในฉากก่อน

แนวทางการแก้ไขที่ใช้ สร้างส่วนต่อประสานชื่อ IActor ที่มีเพียงสามเมธอด แล้วเปลี่ยนให้วัตถุทุกชิ้นรับส่วนต่อประสานนี้แทนคลาสของผู้เล่น

```
// Assets/Scripts/Interaction/IActor.cs
public interface IActor
{
    void Say(string message);              // บอกข้อความกับผู้ที่กำลังทำงานกับวัตถุนี้
    void OfferBag(IItemReceiver to);       // เสนอกระเป๋าให้เลือกของมาใช้กับ to
    void Take(ItemData item);              // ส่งของให้ผู้นั้นถือไป
```

```
}
```

จุดที่สำคัญคือส่วนต่อประสานนี้ถูกวางไว้ในโมดูล Interaction ไม่ใช่โมดูล Player เพราะโมดูลนาฬิกาและปริศนาอ้างถึงโมดูล Interaction อยู่แล้ว การวางไว้ที่นี่จึงไม่สร้างการพึ่งพาเส้นใหม่ ขณะที่หากวางไว้กับผู้เล่น ปัญหาเดิมจะกลับมาในชื่ออื่น ส่วนคลาส PlayerInteractor ก็ implement ส่วนต่อประสานนี้ด้วยการส่งต่อสามบรรทัด

```
// Assets/Scripts/Player/PlayerInteractor.cs
public void Say(string message) => hud.ShowMessage(message);
public void OfferBag(IItemReceiver to) => inventoryUI.Open(to);
public void Take(ItemData item) => Inventory.Add(item);
```

หลังจากนั้นได้เพิ่มไฟล์ Assembly Definition ห้าไฟล์ เพื่อบังคับไม่ให้การพึ่งพาแบบเดิมกลับมาได้อีกในระดับการคอมไพล์ [24] กล่าวคือการเขียนโปรแกรมในโมดูลนาฬิกาให้อ้างถึงคลาสของผู้เล่นจะคอมไพล์ไม่ผ่านทันที ไม่ใช่เพียงผิดธรรมเนียมที่ตกลงกันไว้

ผลหลังแก้ไข วัดได้สามข้อ ดังตารางที่ 5.2

ตารางที่ 5.2 ผลที่วัดได้หลังเปลี่ยนมาใช้ส่วนต่อประสาน IActor

ผลที่วัดได้	ค่า	วิธีวัด
ไฟล์ที่รับ IActor เป็นพารามิเตอร์	17 จุด ใน 18 ไฟล์	ค้นข้อความ IActor actor ใน โฟลเดอร์ Scripts/
การอ้างถึง PlayerInteractor ใน โฟลเดอร์ Clock/ และ Puzzle/	**0 ครั้ง**	ค้นข้อความในสองโฟลเดอร์
จำนวนบรรทัดที่ PlayerInteractor ต้องเขียนเพื่อรองรับสัญญานี้	**3 บรรทัด**	PlayerInteractor.cs

ผลที่เป็นรูปธรรมที่สุดคือ ชุดทดสอบหกกรณีในไฟล์ ActorSeamTests.cs ใช้คลาส C# ธรรมดาสิบบรรทัดที่ไม่ใช่ MonoBehaviour และไม่อยู่ในฉากใด ทำงานแทนผู้เล่นได้ครบ คือเปิดกลอนประตู ใส่ชิ้นส่วนนาฬิกา และรับสิ่งของ ซึ่งเป็นสิ่งที่ทำไม่ได้เลยก่อนการแก้ไข จึงเป็นหลักฐานว่าการกลับทิศทางการพึ่งพาเกิดขึ้นจริง ไม่ใช่เพียงการเปลี่ยนชื่อชนิดข้อมูล

5.2.3 ลำดับการเรียก Awake ที่ไม่ถูกกำหนด ปัญหาที่ยังค้างอยู่

อาการของปัญหา การตรวจ Console ของ Unity เมื่อวันที่ 6 พฤศจิกายน 2569 พบข้อยกเว้น NullReferenceException หนึ่งรายการที่ DigitalReadout.SetDigit (รายละเอียดอยู่ในหัวข้อ 4.2.9)

สาเหตุ ClockMovement.Awake() ค้นหาส่วนแสดงเวลาทุกตัวที่ประกอบอยู่ แล้วเรียกให้แสดงเวลาตั้งต้นทันทีในบรรทัดสุดท้ายของ Awake ขณะที่ DigitalReadout เตรียมอาเรย์ที่เก็บ material ของแต่ละหลักไว้ใน Awake() ของตัวเอง สองคลาสนี้อยู่บน GameObject ต่างกัน และ Unity ไม่รับประกันลำดับการเรียก Awake ระหว่างคอมโพเนนต์ที่อยู่บน GameObject ต่างกัน [17] โครงงานนี้ไม่ได้ตั้งค่า Script Execution Order ไว้ ดังนั้นหาก ClockMovement.Awake() ทำงานก่อน อาเรย์จะยังว่างขณะถูกเข้าถึง จึงเกิดข้อยกเว้น

ปัญหานี้เป็นปัญหาชนิดเดียวกับที่โครงงานแก้ไปแล้วหนึ่งจุดในหัวข้อ 5.2.1 กลไกที่สี่ คือ ลำดับการทำงานระหว่างคอมโพเนนต์ที่เอนจินไม่กำหนดไว้ ต่างกันที่จุดนั้นเป็นลำดับของ Update และแก้ได้ด้วยการย้ายไป LateUpdate ขณะที่จุดนี้เป็นลำดับของ Awake ซึ่งไม่มีเมธอดที่รับประกันว่าทำงานหลังสุดให้ย้ายไป จึงต้องแก้ด้วยวิธีอื่น วัตถุที่ได้รับผลกระทบคือเครื่องจับเวลาบนโต๊ะช่าง ซึ่งเป็นวัตถุชิ้นเดียวในเกมที่มีส่วนแสดงเวลาสองชนิดบนกลไกเดียว

แนวทางการแก้ไขที่เสนอ ให้ DigitalReadout เตรียมอาเรย์แบบ lazy คือตรวจและเตรียมในครั้งแรกที่ถูกสั่งให้แสดงเวลา ซึ่งแก้ที่ต้นเหตุและไม่ขึ้นกับลำดับการเรียกเลย ส่วนทางเลือกที่สองคือการตั้งค่า Script Execution Order ให้ DigitalReadout ทำงานก่อน ClockMovement นั้นแก้อาการได้ แต่ผูกการทำงานของสองคลาสไว้กับค่าตั้งในโปรเจกต์ที่มองจากโปรแกรมไม่เห็น ซึ่งขัดกับแนวทางที่โครงงานนี้ใช้มาตลอด จึงเสนอทางแรก

5.3 ข้อเสนอแนะจากการดำเนินงาน

จากการดำเนินโครงงานสหกิจศึกษา มีข้อเสนอแนะดังนี้

1) ควรใช้ระบบควบคุมรุ่นของซอร์สโค้ดตั้งแต่วันแรกของโครงงาน โครงงานนี้ไม่มีระบบควบคุมรุ่น ซึ่งเป็นข้อจำกัดที่ส่งผลกว้างที่สุด โครงงานมีการอ้างอิงที่ตั้งค่าผ่าน Inspector ถึง 215 จุด และใช้นโยบายไม่ตรวจค่าว่างโดยเจตนา การต่อสายผิดหรือลบวัตถุพลาดหนึ่งครั้งจึงทำให้เกมหยุดทำงานโดยไม่มีทางย้อนกลับ และการไม่มีประวัติการแก้ไขทำให้ยืนยันรูปแบบของโปรแกรมก่อนการแก้ไขไม่ได้

2) ควรเขียนชุดทดสอบครอบโมดูล Puzzle โมดูลนี้เป็นโมดูลเดียวที่ไม่มีข้อจำกัดทางเทคนิคขัดขวาง คือไฟล์ Assembly Definition ของชุดทดสอบประกาศให้อ้างถึงโมดูลนี้ได้อยู่แล้ว แต่ยังไม่มีกรณีทดสอบใดใช้ชนิดข้อมูลจากโมดูลนี้ กรณีที่ควรเพิ่มคือตรวจว่า CodeLock ปลดเมื่อ

อุปกรณ์ใส่รหัสทุกตัวตรงกับรหัสที่ตั้งไว้ ตรวจว่า StampedCombination ประกอบรหัสจาก StampedNumber ได้ถูกลำดับ และตรวจว่า Horologium คืนรูปแบบเดิมเมื่อปริซึมทุกชิ้นอยู่ที่คำตอบ

3) ควรแก้ส่วนต่อประสาน ICodeInput ให้นำกลับมาใช้ได้จริง ปัจจุบันฟิลด์ที่รับอุปกรณ์ใส่รหัสใน CodeLock ประกาศเป็นชนิด CodeWheel[] ซึ่งเป็นคลาสรูปธรรม ทำให้คลาสใหม่ที่ implement ICodeInput ใส่ลงไปไม่ได้โดยไม่แก้โปรแกรม วิธีแก้มีอยู่ในโครงงานนี้แล้ว คือใช้การค้นหาด้วย GetComponentsInChildren<ICodeInput>() แบบเดียวกับที่โมดูล Clock ใช้กับ IClockDisplay ซึ่งมีชุดทดสอบยืนยันว่าใช้ได้

4) ควรแก้ข้อบกพร่องเรื่องลำดับการเรียก Awake ตามที่เสนอในหัวข้อ 5.2.3 เพราะเป็นข้อบกพร่องที่ยังค้างอยู่ และกระทบวัตถุที่ตั้งใจวางไว้เพื่อแสดงคุณสมบัติของโครงสร้าง

5) ควรตัดรายการอ้างอิงที่ไม่ได้ใช้ออกจากไฟล์ Assembly Definition EscapeRoom.Puzzle.asmdef ประกาศให้อ้างถึงสองโมดูลที่โปรแกรมไม่ได้ใช้ และ EscapeRoom.Tests.asmdef ประกาศเกินอีกหนึ่งรายการ ทำให้ขอบเขตโมดูลหลวมกว่าที่โปรแกรมต้องการจริง การตัดออกใช้เวลาไม่กี่นาทีและไม่ต้องแก้โปรแกรม

6) ควรแยกความรับผิดชอบของคลาส InspectView และ Horologium InspectView มี 317 บรรทัดและรับผิดชอบงานแปดอย่าง เป็นคลาสที่มีความเกาะเกี่ยวต่ำที่สุดในโครงงาน ส่วนที่แยกออกได้สะอาดที่สุดคือการคำนวณตำแหน่งและระยะของวัตถุที่ยกขึ้นมา และส่วนที่จัดการนาฬิกาทราย ส่วน Horologium ขยับ transform ของจุดแสงด้วยตัวเอง ควรแยกส่วนแสดงผลออกให้เป็นคอมโพเนนต์ต่างหากแบบเดียวกับที่ระบบนาฬิกาทำ

7) ควรตัดวงจรการพึ่งพาระหว่างโมดูล Player และ UI วงจรนี้ทำให้ใส่ขอบเขตบังคับให้สามโมดูลชั้นบนไม่ได้ และทำให้เขียนชุดทดสอบครอบสามโมดูลนั้นไม่ได้ เพราะกฎของ Unity ห้าม Assembly Definition อ้างถึง predefined assembly [24] ควรให้โมดูล UI ไม่รู้จักโมดูล Player แล้วให้ Player ป้อนข้อมูลผ่านส่วนต่อประสาน ซึ่งเป็นวิธีเดียวกับที่ใช้แก้ปัญหาในหัวข้อ 5.2.2 ข้อนี้เป็นงานที่ใหญ่ที่สุดและควรทำหลังข้ออื่น

8) ควรเพิ่มการจัดการสถานะความก้าวหน้าของผู้เล่น ปัจจุบันความก้าวหน้ากระจายเป็นตัวแปรอยู่ในกลอนแต่ละตัว ไม่มีคลาสใดตอบได้ว่าผู้เล่นอยู่องก์ที่เท่าไร ผลคือเกมไม่มีการบันทึกและโหลด และทำระบบคำใบ้ไม่ได้ ซึ่งมีผลต่อการเล่นจริง เพราะปริซึมสามชิ้นเก็บได้ในสามห้องต่างกันแต่ใช้พร้อมกันในห้องสุดท้าย หากผู้เล่นพลาดเก็บชิ้นใดชิ้นหนึ่งจะติดที่ห้องสุดท้ายโดยไม่มีคำใบ้ว่าต้องย้อนกลับไปห้องไหน

9) ควรทดสอบกับผู้ใช้อย่างเป็นระบบและบันทึกผลไว้ การทดลองเล่นกับเพื่อนห้าคนให้ข้อสังเกตที่มีค่า เช่น ผู้เล่นติดที่องก์ที่ 3 ตามที่ออกแบบไว้ แต่ไม่ได้บันทึกเวลาที่ใช้แก้แต่ละองก์และ

จุดที่ติด หากบันทึกไว้จะระบุได้ว่าองก์ใดยากเกินไปหรือคำใบ้ใดไม่ชัด และควรทดสอบบนเครื่องที่มีความเร็วต่างกันอย่างน้อยสองเครื่อง เพื่อยืนยันว่าปัญหาการนับเวลาในหัวข้อ 5.2.1 แก้ได้จริงในสภาพแวดล้อมของผู้เล่น

10) ควรต่อยอดเนื้อเรื่องและเพิ่มองก์ใหม่ด้วยการประกอบคอมโพเนนต์เดิม เกมนี้จบด้วยฉากปิดท้ายของเดโมและยังไม่เฉลยว่าช่างนาฬิกาหายไปไหน การเพิ่มห้องหรือปริศนาใหม่ควรเริ่มจากการใช้ชิ้นส่วนที่มีอยู่ เช่น ลานสปริง ลูกตุ้ม ช่องใส่ชิ้นส่วน และกลอนชนิดต่าง ๆ แล้วเพิ่มคอมโพเนนต์ใหม่เฉพาะเมื่อไม่มีชิ้นส่วนใดตอบโจทย์

ลำดับความสำคัญของข้อเสนอแนะทั้งสิบข้อ โดยพิจารณาจากผลกระทบเทียบกับความยากในการทำ แสดงดังตารางที่ 5.3

ตารางที่ 5.3 ลำดับความสำคัญของข้อเสนอแนะ

ลำดับ	ข้อเสนอแนะ	ผลกระทบ	ความยาก
1	ข้อ 1 ใช้ระบบควบคุมรุ่น	สูงที่สุด ป้องกันความเสียหายที่ย้อนไม่ได้	ต่ำ
2	ข้อ 5 ตัดรายการอ้างอิงที่ไม่ใช้ออกจาก Assembly Definition	ปานกลาง	ต่ำที่สุด ไม่ต้องแก้โปรแกรม
3	ข้อ 4 แก้ข้อบกพร่องเรื่องลำดับ Awake	สูง เป็นข้อบกพร่องที่ยังค้าง	ต่ำ แก้หนึ่งคลาส
4	ข้อ 2 เขียนชุดทดสอบครอบโมดูล Puzzle	สูง จับข้อผิดพลาดของค่าที่ตั้งผิด	ต่ำ ไม่มีข้อจำกัดขัดขวาง
5	ข้อ 3 แก้ ICodeInput ให้นำกลับมาใช้ได้จริง	ปานกลาง ทำให้หลักการออกแบบทั่วถึง	ต่ำ มีแบบอย่างในโครงงานแล้ว
6	ข้อ 9 ทดสอบกับผู้ใช้และบนเครื่องที่ต่างกัน	สูง ยืนยันว่าแก้ปัญหาการนับเวลาสำเร็จจริง	ปานกลาง
7	ข้อ 6 แยกความรับผิดชอบของ InspectView และ Horologium	ปานกลาง	ปานกลาง

8	ข้อ 10 ต่อยอดเนื้อเรื่องและเพิ่มองก์ใหม่	ปานกลาง ใช้ประโยชน์จากโครงสร้างที่ทำไว้	ปานกลาง
9	ข้อ 8 เพิ่มการจัดการสถานะความก้าวหน้า	ปานกลาง เปิดทางให้คุณสมบัติใหม่	สูง
10	ข้อ 7 ตัดวงจรการพึ่งพา Player และ UI	สูงต่อโครงสร้าง	สูงที่สุด

เอกสารอ้างอิง

1. Fowler, M., 2018, **Refactoring: Improving the Design of Existing Code**, 2nd ed., Addison-Wesley, Boston, MA, USA [Online], Available : https://martinfowler.com/books/refactoring.html [2026, November 10].

2. Parnas, D. L., 1972, "On the Criteria to Be Used in Decomposing Systems into Modules", **Communications of the ACM**, Vol. 15, No. 12, pp. 1053-1058 [Online], Available : https://doi.org/10.1145/361598.361623 [2026, November 10].

3. Stevens, W. P., Myers, G. J. and Constantine, L. L., 1974, "Structured Design", **IBM Systems Journal**, Vol. 13, No. 2, pp. 115-139 [Online], Available : https://doi.org/10.1147/sj.132.0115 [2026, November 10].

4. International Organization for Standardization, 2011, **Systems and Software Engineering - Systems and Software Quality Requirements and Evaluation (SQuaRE) - System and Software Quality Models**, ISO/IEC 25010:2011, Geneva, Switzerland [Online], Available : https://www.iso.org/standard/35733.html [2026, November 10].

5. Nystrom, R., 2014, **Game Programming Patterns**, Genever Benning [Online], Available : https://gameprogrammingpatterns.com [2026, November 10].

6. Gregory, J., 2018, **Game Engine Architecture**, 3rd ed., CRC Press, Boca Raton, FL, USA [Online], Available : https://www.taylorfrancis.com/books/mono/10.1201/9781315267845/game-engine-architecture-jason-gregory [2026, November 10].

7. Unity Technologies, n.d., **MonoBehaviour**, Unity Scripting API Reference [Online], Available : https://docs.unity.com/en-us/engine/current/script-reference/unityengine/monobehaviour [2026, November 10].

8. Gamma, E., Helm, R., Johnson, R. and Vlissides, J., 1994, **Design Patterns: Elements of Reusable Object-Oriented Software**, Addison-Wesley, Reading, MA, USA [Online], Available : https://www.pearson.com/us/higher-education/program/Gamma-Design-Patterns-Elements-of-Reusable-Object-Oriented-Software/PGM14333.html [2026, November 10].

9. Martin, R. C., 2002, **Agile Software Development: Principles, Patterns, and Practices**, Prentice Hall, Upper Saddle River, NJ, USA [Online], Available : https://www.pearson.com/us/higher-education/program/Martin-Agile-Software-Development-Principles-Patterns-and-Practices/PGM272869.html [2026, November 10].

10. Martin, R. C., 2017, **Clean Architecture: A Craftsman's Guide to Software Structure and Design**, Prentice Hall, Boston, MA, USA [Online], Available : https://www.pearson.com/en-us/subject-catalog/p/clean-architecture/P200000009528 [2026, November 10].

11. Meyer, B., 1997, **Object-Oriented Software Construction**, 2nd ed., Prentice Hall, Upper Saddle River, NJ, USA [Online], Available : https://bertrandmeyer.com/OOSC2/ [2026, November 10].

12. Dijkstra, E. W., 1982, "On the Role of Scientific Thought", in **Selected Writings on Computing: A Personal Perspective**, Springer-Verlag, New York, NY, USA, pp. 60-66 [Online], Available : https://link.springer.com/doi/10.1007/978-1-4612-5695-3_12 [2026, November 10].

13. ECMA International, 2022, **C# Language Specification**, Standard ECMA-334, 6th ed., Geneva, Switzerland [Online], Available : https://ecma-international.org/publications-and-standards/standards/ecma-334/ [2026, November 10].

14. Unity Technologies, n.d., **deltaTime**, Unity Scripting API Reference [Online], Available : https://docs.unity.com/en-us/engine/current/script-reference/unityengine/time/deltatime [2026, November 10].

15. Institute of Electrical and Electronics Engineers, 2019, **IEEE Standard for Floating-Point Arithmetic**, IEEE Std 754-2019 [Online], Available : https://standards.ieee.org/ieee/754/6210/ [2026, November 10].

16. Goldberg, D., 1991, "What Every Computer Scientist Should Know About Floating-Point Arithmetic", **ACM Computing Surveys**, Vol. 23, No. 1, pp. 5-48 [Online], Available : https://doi.org/10.1145/103162.103163 [2026, November 10].

17. Unity Technologies, n.d., **Event function execution order**, Unity 6.3 User Manual [Online], Available : https://docs.unity.com/en-us/engine/6000.3/manual/scripting/managing-update-order/execution-order [2026, November 10].

18. Meszaros, G., 2007, **xUnit Test Patterns: Refactoring Test Code**, Addison-Wesley, Boston, MA, USA [Online], Available : https://www.oreilly.com/library/view/xunit-test-patterns/9780131495050/ [2026, November 10].

19. Unity Technologies, n.d., **Unity Test Framework**, Test Framework Package Documentation, version 1.6.0 [Online], Available : https://docs.unity3d.com/Packages/com.unity.test-framework@1.6/manual/index.html [2026, November 10].

20. Ford, N., Parsons, R. and Kua, P., 2017, **Building Evolutionary Architectures: Support Constant Change**, O'Reilly Media, Sebastopol, CA, USA [Online], Available : https://www.oreilly.com/library/view/building-evolutionary-architectures/9781491986356/ [2026, November 10].

21. Unity Technologies, n.d., **Introduction to the Universal Render Pipeline**, Unity 6.3 User Manual [Online], Available : https://docs.unity.com/en-us/engine/6000.3/manual/render-pipelines/universal-render-pipeline/urp-introduction [2026, November 10].

22. Unity Technologies, n.d., **Prefabs**, Unity 6.3 User Manual [Online], Available : https://docs.unity.com/en-us/engine/6000.3/manual/working-with-gameobjects/prefabs [2026, November 10].

23. Unity Technologies, n.d., **ScriptableObject data assets**, Unity 6.3 User Manual [Online], Available : https://docs.unity.com/en-us/engine/6000.3/manual/scripting/get-started/fundamental-unity-types/class-scriptable-object [2026, November 10].

24. Unity Technologies, n.d., **Referencing assemblies**, Unity 6.3 User Manual [Online], Available : https://docs.unity.com/en-us/engine/6000.3/manual/programming-environment/script-compilation/assembly-definition-files/assembly-definitions-referencing [2026, November 10].

25. Unity Technologies, n.d., **Input System**, Input System Package Documentation, version 1.17.0 [Online], Available : https://docs.unity3d.com/Packages/com.unity.inputsystem@1.17/manual/index.html [2026, November 10].

26. NUnit Project, n.d., **NUnit - a unit-testing framework for all .NET languages** [Online], Available : https://nunit.org [2026, November 10].

27. Hendrikx, M., Meijer, S., Van Der Velden, J. and Iosup, A., 2013, "Procedural Content Generation for Games: A Survey", **ACM Transactions on Multimedia Computing, Communications, and Applications**, Vol. 9, No. 1, Article 1, pp. 1-22 [Online], Available : https://doi.org/10.1145/2422956.2422957 [2026, November 10].

28. Shaker, N., Togelius, J. and Nelson, M. J., 2016, **Procedural Content Generation in Games**, Springer, Cham, Switzerland [Online], Available : https://link.springer.com/doi/10.1007/978-3-319-42716-4 [2026, November 10].

ภาคผนวก

ภาคผนวก ก.
รายงานปฏิบัติงานประจำสัปดาห์

[แทรกรายงานปฏิบัติงานประจำสัปดาห์ที่ส่งศูนย์สหกิจศึกษา หน้าละสัปดาห์ เรียงตามลำดับ แล้วลบหน้านี้]

ประวัติผู้จัดทำโครงงานสหกิจศึกษา

ชื่อ – นามสกุล	อวกาศ ดีคำ
วัน เดือน ปี เกิด	9 พฤษภาคม 2545
ประวัติการศึกษา	
ระดับมัธยมศึกษา	มัธยมศึกษาตอนต้น พ.ศ. 2561 กรรณสูตศึกษาลัย จังหวัดสุพรรณบุรี มัธยมศึกษาตอนปลาย พ.ศ. 2564 กรรณสูตศึกษาลัย จังหวัดสุพรรณบุรี
ระดับอุดมศึกษา	วิศวกรรมศาสตรบัณฑิต สาขาคอมพิวเตอร์และปัญญาประดิษฐ์ สถาบันเทคโนโลยีไทย-ญี่ปุ่น พ.ศ. 2569
ทุนการศึกษา	-
ประวัติการฝึกอบรม	-
ผลงานที่ได้รับการตีพิมพ์	-